

***CODA-EDA: COMPACT DEPENDENCY-AWARE ESTIMATION
OF DISTRIBUTION ALGORITHM UNTUK POLICY MINING
ATTRIBUTE-BASED ACCESS CONTROL (ABAC) PADA
EDGE-IOT***

TESIS

**Karya tulis sebagai salah satu syarat
untuk memperoleh gelar Magister dari
Institut Teknologi Bandung**

**Oleh:
ZIADAN QOWI
NIM: 23524019
Program Studi Magister Informatika**



**INSTITUT TEKNOLOGI BANDUNG
September 2026**

ABSTRAK

CODA-EDA: COMPACT DEPENDENCY-AWARE ESTIMATION OF DISTRIBUTION ALGORITHM UNTUK POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) PADA EDGE-IOT

Oleh
Ziadan Qowi
NIM: 23524019
Program Studi Magister Informatika

Kontrol akses berbasis atribut (ABAC) memerlukan penambangan kebijakan (*policy mining*) dari log akses karena penyusunan mahal dan rawan galat. Pada IoT, penambangan idealnya berjalan langsung pada perangkat *edge* bersumber daya terbatas sekaligus diperbarui secara berkala. Algoritma yang ada umumnya boros memori atau mengabaikan dependensi antar atribut. Penelitian ini merancang dan menguji CoDA-EDA (*Compact Dependency-Aware Estimation of Distribution Algorithm*) yang menyatukan pembaruan probabilistik tanpa populasi eksplisit, pemodelan dependensi orde satu melalui pohon Chow-Liu, dan pemisahan pembaruan struktur yang dipicu *drift* dari pembaruan parameter dan memiliki jejak memori $O(D)$. Dengan paradigma *Design Science Research*, CoDA-EDA diuji terhadap enam *base-line* pada Raspberry Pi 5 atas lima dataset ABAC Lab dan validasi antar-generator *Smart Building IoT* menggunakan uji Friedman dan *post-hoc* Wilcoxon berkoreksi Holm-Bonferroni. Hasilnya, kualitas mengungguli keluarga *compact* univariat yang menguat pada dimensi tinggi, setara iBOA/BOA, namun tertinggal moderat dari ACO/UBK. Jejak memorinya datar dengan rasio 1,00 berbanding 6,2-9,2 kali pada algoritma berpopulasi eksplisit. Pemisahan pembaruan berhasil menghemat waktu saat data stabil, serta strategi *warm-start* 2,0-2,6 kali lebih cepat dengan kualitas non-inferior. Kontribusinya adalah pertukaran rancangan yang koheren bagi *gateway edge-IoT*.

Kata kunci: ABAC, CoDA-EDA, dependensi, *edge-IoT*, inkremental, memori, penambangan kebijakan.

ABSTRACT

CODA-EDA: COMPACT DEPENDENCY-AWARE ESTIMATION OF DISTRIBUTION ALGORITHM FOR POLICY MINING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) IN EDGE-IOT

By

Ziadan Qowi

NIM: 23524019

Master's Program in Informatics

Attribute-Based Access Control (ABAC) requires policy mining from access logs because manual policy construction is costly and error-prone. In IoT environments, policy mining should ideally be performed directly on resource-constrained edge devices while being periodically updated. Existing algorithms are generally memory-intensive or fail to account for dependencies among attributes. This study designs and evaluates CoDA-EDA (Compact Dependency-Aware Estimation of Distribution Algorithm), which integrates population-free probabilistic updates, first-order dependency modeling through a Chow-Liu tree, and the separation of drift-triggered structure updates from parameter updates, while maintaining an $O(D)$ memory footprint. Following the Design Science Research paradigm, CoDA-EDA is evaluated against six baselines on a Raspberry Pi 5 using five ABAC Lab datasets, together with cross-generator validation on a Smart Building IoT dataset. Statistical significance is assessed using the Friedman test followed by post-hoc Wilcoxon tests with Holm-Bonferroni correction. The results show that CoDA-EDA achieves higher solution quality than the univariate compact family, with the advantage becoming more pronounced at higher dimensionalities. Its performance is comparable to iBOA/BOA, although it remains moderately behind ACO/UBK. Its memory footprint remains effectively constant, with a ratio of 1.00 compared with 6.2-9.2 times higher memory usage for explicit population-based algorithms. The separation of update mechanisms successfully reduces computation time when the data distribution is stable, while the warm-start strategy is 2.0-2.6 times faster while maintaining non-inferior solution quality. The main contribution is a coherent design trade-off for policy mining on edge-IoT gateways.

Keywords: ABAC, CoDA-EDA, dependency, edge-IoT, incremental, memory, policy mining.

**CODA-EDA: *COMPACT DEPENDENCY-AWARE ESTIMATION
OF DISTRIBUTION ALGORITHM* UNTUK *POLICY MINING
ATTRIBUTE-BASED ACCESS CONTROL (ABAC)* PADA
*EDGE-IOT***

Oleh
Ziadan Qowi
NIM 23524019
(Program Studi Magister Informatika)

Institut Teknologi Bandung

Menyetujui
Tim Pembimbing
Tanggal 4 September 2026

Ketua



(Ir. Robithoh Annur, S.T., M.Eng.,
Ph.D.)
NIP. 122110015

Anggota



(Dr. Phil. Eng. Hari Purnama, S.Si.,
M.Si)
NIP. 118110072

DAFTAR ISI

ABSTRAK.....	i
<i>ABSTRACT</i>	ii
HALAMAN PENGESAHAN	iii
DAFTAR ISI.....	iv
DAFTAR LAMPIRAN.....	vi
DAFTAR GAMBAR DAN ILUSTRASI.....	vii
DAFTAR TABEL	viii
DAFTAR SINGKATAN DAN LAMBANG	ix
 Bab I	
Pendahuluan	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah.....	3
I.3 Tujuan Penelitian	4
I.4 Batasan Masalah	4
I.5 Premis dan Hipotesis.....	5
I.6 Metodologi Penelitian	6
I.7 Sistematika Penulisan.....	8
 Bab II	
Tinjauan Pustaka dan Landasan Teori	9
II.1 <i>Internet of Things</i> dan Keterbatasan Sumber Daya Perangkat <i>Edge</i>	9
II.2 Kontrol Akses dan <i>Attribute-Based Access Control</i> (ABAC) .	10
II.3 <i>Policy Mining</i> pada ABAC.....	12
II.4 Algoritma Metaheuristik untuk Optimasi Kombinatorial	15
II.5 <i>Estimation of Distribution Algorithm</i>	16
II.6 EDA Ringan/ <i>Compact</i>	18
II.7 Pemodelan Dependensi	20
II.8 Penelitian Terdahulu	22
II.9 Posisi dan Kebaruan Penelitian.....	24
II.10 Kerangka Berpikir	29
 Bab III	
Metodologi Penelitian	31
III.1 Analisis Masalah	31
III.2 Analisis Kebutuhan	33
III.3 Variabel Penelitian	34
III.4 Rancangan Algoritma CoDA-EDA	37
III.5 Konstruksi Dataset Penelitian	45
III.6 Algoritma Pembandingan	52
III.7 Perangkat dan Lingkungan Eksperimen	55
III.8 Desain Eksperimen.....	57
III.9 Metrik dan Instrumen Evaluasi.....	61
III.10 Teknik Analisis Data	65
III.11 Ketertelusuran Rancangan Penelitian	67

III.12	Studi Pendahuluan	69
Bab IV	Hasil dan Pembahasan	76
IV.1	Kualitas Kebijakan	76
IV.2	Jejak Memori	82
IV.3	Efisiensi Pembaruan Kebijakan	84
IV.4	Validasi Eksternal	86
IV.5	Sintesis, Keterbatasan, dan Jawaban Rumusan Masalah	87
Bab V	Kesimpulan dan Saran	90
V.1	Kesimpulan	90
V.2	Saran	91
DAFTAR PUSTAKA.....		93
LAMPIRAN		97

DAFTAR LAMPIRAN

Lampiran A	Contoh Kebijakan Hasil Mining Lengkap	98
A.1	CoDA-EDA pada Dataset <i>Workforce</i> ($D = 436$) . . .	98
A.2	ACO pada Dataset <i>Workforce</i> (Pembanding)	99
A.3	CoDA-EDA pada Dataset <i>Smart Building IoT</i> ($D = 428$)	100
Lampiran B	Tabel Hasil Eksperimen Lengkap	101
B.1	Kualitas Kebijakan (H1) – Rerata Per Dataset dan Tingkat Korelasi	101
B.2	Jejak Memori (H2) – Rasio Penskalaan $\text{alloc}(NP = 400)/\text{alloc}(NP = 25)$ Per Dataset	103
B.3	Efisiensi Pembaruan (H3 dan H4) – Waktu Per Siklus dan Kualitas	104
B.4	Validasi Eksternal <i>Smart Building IoT</i> (H1-H4) . . .	106
Lampiran C	Hasil Uji Statistik Lengkap	107
C.1	Uji H1 (Kualitas Kebijakan) Per Dataset	107
C.2	Uji H3 dan H4 (Efisiensi Pembaruan) Per Dataset dan Skenario	110
C.3	Uji Validasi Eksternal <i>Smart Building IoT</i> (H1) . . .	112
Lampiran D	Analisis Karakteristik Dataset (Pra-Eksperimen)	113
Lampiran E	Konfigurasi dan Kalibrasi Algoritma	115
E.1	Parameter Tiap Algoritma	115
E.2	Anggaran Evaluasi	116
E.3	Parameter Siklus Pembaruan (H3 dan H4)	116
E.4	Ringkasan Uji Monotonisitas (Kalibrasi)	117
Lampiran F	Rincian Konstruksi Dataset	119
F.1	Parameter Konstruksi	119
F.2	Struktur Manifest	119
F.3	Laporan Validasi — Kalibrasi Korelasi	121
F.4	Laporan Validasi — Lantai Derau, Drift Bersih, dan Duplikasi	121
Lampiran G	Kutipan Kode Kunci	123
G.1	Fungsi Fitness Selaras-F1	123
G.2	Pohon Chow-Liu order-1 via Algoritma Prim (Memori $O(D)$)	124
G.3	Mekanisme Deteksi Drift	125

DAFTAR GAMBAR DAN ILUSTRASI

I.1	Enam Aktivitas DSRM dan Pemetaannya terhadap Struktur Penelitian	8
II.1	Arsitektur Keputusan Akses ABAC	12
II.2	Siklus Kerja Algoritma <i>Compact</i> dan Mekanisme Pembaruan PV ...	19
II.3	Spektrum Pemodelan Dependensi pada EDA dan Posisi CoDA-EDA	21
II.4	Kerangka Berpikir Penelitian	30
III.1	Diagram Alir Arsitektur CoDA-EDA	41
III.2	<i>Pipeline</i> Konstruksi Dataset Penelitian	49
III.3	Peta Eksperimen: Pemetaan Hipotesis H1-H4 terhadap Perbandingan, Metrik, dan Uji Statistik	61
IV.1	Jejak Memori CoDA-EDA Linear Terhadap Dimensi Ruang Solusi D	83

DAFTAR TABEL

II.1	Contoh potongan log akses ($\mathcal{L}$)	13
II.2	Ringkasan Posisi Penelitian Terdahulu	23
II.3	Perbedaan CoDA-EDA terhadap Algoritma EDA yang Sudah Ada ..	27
III.1	Analisis Masalah pada Algoritma Pembandingan terhadap Kebutuhan Inti Edge-IoT	33
III.2	Pemetaan Analisis Masalah — Kebutuhan — Keputusan Rancangan	34
III.3	Definisi Operasional Variabel Penelitian	35
III.4	Perbandingan kompleksitas waktu dan memori per siklus/generasi ..	44
III.5	Ilustrasi distribusi bersama dua atribut sebelum injeksi korelasi	46
III.6	Ilustrasi distribusi bersama dua atribut setelah injeksi korelasi	46
III.7	Skema Dataset Penelitian	50
III.8	Algoritma Pembandingan dan Perlakuan Kesetaraan Implementasi	53
III.9	Spesifikasi Perangkat dan Lingkungan Eksperimen	56
III.10	Ringkasan Desain Eksperimen	60
III.11	Definisi Operasional Metrik dan Instrumen Evaluasi.....	63
III.12	Rancangan Uji Statistik per Hipotesis	67
III.13	Desain artefak dan pemetaan kebutuhan	68
III.14	Karakteristik Dataset dan Kecocokannya dengan Hipotesis (Analisis Pra-Eksperimen).....	69
III.15	Hasil Studi Pendahuluan: Kualitas Kebijakan (F-score) pada Dua Dataset Utama	73
IV.1	Rerata <i>F-score</i> Ketujuh Algoritma pada Lima Dataset ABAC Lab...	76
IV.2	Rincian Kualitas H1 pada e-document ($D = 1.907$; rerata lintas korelasi; post-hoc vs CoDA-EDA)	77
IV.3	Efek Anggaran Jumlah Aturan terhadap Kualitas CoDA-EDA pada <i>e-document</i>	78
IV.4	Contoh Kebijakan Hasil Mining (cuplikan aturan terbaca).....	81
IV.5	Penskalaan Jejak Memori terhadap Ukuran Populasi NP (rerata lintas lima dataset).....	82
IV.6	Hasil Pengujian H3 dan H4 pada Lima Dataset (rerata; warm/always/cold)	84
IV.7	Validasi Eksternal Smart Building IoT — Kualitas ($D = 428$; garis sepele $F1=0,858$; 5 seed)	86
IV.8	Sintesis Status Hipotesis H1-H4 Lintas Dataset	88

DAFTAR SINGKATAN DAN LAMBANG

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
ABAC	<i>Attribute-Based Access Control</i>	1
ACL	<i>Access Control List</i>	10
ACO	<i>Ant Colony Optimization</i>	1
ARM	Arsitektur/prosesor ARM	??
BIC	<i>Bayesian Information Criterion</i>	20
BOA	<i>Bayesian Optimization Algorithm</i>	20
CoAP	<i>Constrained Application Protocol</i>	??
CoDA-EDA	<i>Compact Dependency-Aware Estimation of Distribution Algorithm</i>	4
compact-GA	<i>Compact Genetic Algorithm</i>	??
compact-PSO	<i>Compact Particle Swarm Optimization</i>	??
CPU	<i>Central Processing Unit</i>	56
DAC	<i>Discretionary Access Control</i>	1
DSR	<i>Design Science Research</i>	??
DSRM	<i>Design Science Research Methodology</i>	??
EDA	<i>Estimation of Distribution Algorithm</i>	5
F1	<i>F-measure (F-score)</i>	38
FN	<i>False Negative</i>	62
FP	<i>False Positive</i>	38
H1	Hipotesis Penelitian 1	6
H2	Hipotesis Penelitian 2	6
H3	Hipotesis Penelitian 3	6
H4	Hipotesis Penelitian 4	6
HAT	<i>Hardware Attached on Top</i>	57
iBOA	<i>Incremental Bayesian Optimization Algorithm</i>	??
IoT	<i>Internet of Things</i>	1
JSD	<i>Jensen-Shannon Divergence</i>	69
LLM	<i>Large Language Model</i>	??
MAC	<i>Mandatory Access Control</i>	10
MGGO	<i>Modified Greylag Goose Optimization</i>	2
MQQT	<i>Message Queuing Telemetry Transport</i>	??
NIST	<i>National Institute of Standards and Technology</i>	11
NMI	<i>Normalized Mutual Information</i>	45
OS	<i>Operating System</i>	57
PCIe	<i>Peripheral Component Interconnect Express</i>	57
PEP	<i>Policy Enforcement Point</i>	??
PMBGA	<i>Probabilistic Model-Building Genetic Algorithm</i>	16
PV	<i>Probability Vector</i>	18

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
RAM	<i>Random Access Memory</i>	??
RBAC	<i>Role-Based Access Control</i>	1
rcGA	<i>Real-Valued Compact Genetic Algorithm</i>	19
RSS	<i>Resident Set Size</i>	55
SDRAM	<i>Synchronous Dynamic Random-Access Memory</i>	57
SoC	<i>System on Chip</i>	??
SPEA2	<i>Strength Pareto Evolutionary Algorithm 2</i>	22
SSD	<i>Solid-State Drive</i>	57
TP	<i>True Positive</i>	38
UBK	<i>Upgraded Black-Winged Kite</i>	2
UcGA	<i>Uniform Compact Genetic Algorithm</i>	19
VmHWM	<i>Virtual Memory High Water Mark</i>	55
WSC	<i>Weighted Structural Complexity</i>	14
LAMBANG		
α	Parameter ACO; simbol ini kemudian juga digunakan sebagai parameter adaptif UBK dan taraf signifikansi statistik	54
β	Parameter/bobot heuristik ACO	54
<i>Coverage</i>	Proporsi permintaan akses yang dicakup oleh setidaknya satu aturan	62
D	Dimensi ruang solusi, yaitu jumlah pasangan atribut-nilai	4
δ	Margin non-inferioritas praktis	40
Δ	Besar pergeseran distribusi atau <i>drift</i> antar-jendela	66
Δ Quality	Selisih kualitas antara <i>warm-start</i> dan <i>re-mining</i> penuh	63
$f(x)$	Nilai fungsi <i>fitness</i> kandidat aturan x	38
i	Indeks pasangan atribut-nilai atau variabel keputusan	38
k	Orde dependensi yang diizinkan pada model	??
L	Log akses yang menjadi masukan <i>policy mining</i>	13
M	Ukuran populasi pada EDA kanonik	16
<i>MemPeak</i>	Nilai maksimum RSS selama eksekusi	62
MemPeak_alloc	Puncak alokasi struktur data algoritma yang diukur dengan <i>tracemalloc</i>	??
MemPeak_sys	Puncak penggunaan memori pada tingkat sistem operasi	60

LAMBANG	Nama	Pemakaian pertama kali pada halaman
n	Banyaknya dimensi/variabel dalam formulasi umum; kemudian juga digunakan untuk jumlah pasangan/blok statistik	20
N	Jumlah individu terbaik yang dipilih pada tahap seleksi EDA	16
NP	Ukuran populasi virtual	4
$O(D)$	Kompleksitas memori linear terhadap dimensi D	18
$O(D^2)$	Kompleksitas perhitungan <i>mutual information</i> seluruh pasangan variabel	43
$O(D^2 \log D)$	Kompleksitas yang dinyatakan untuk konstruksi pohon rentang maksimum	43
$O(n^k)$	Kompleksitas jumlah tabel probabilitas marginal iBOA pada orde dependensi k	21
$O(NP \times D)$	Kompleksitas memori algoritma berpopulasi eksplisit terhadap ukuran populasi dan dimensi	18
p	Nilai-p pada pengujian statistik	65
p_{adj}	Nilai-p yang telah dikoreksi/di- <i>adjust</i> dengan prosedur Holm-Bonferroni	74
p_i	Probabilitas bit ke- i bernilai 1 pada PV	62
p_0	Probabilitas awal/inisialisasi PV	39
π	Kebijakan yang dihasilkan	62
$Precision$	Presisi keputusan kebijakan, $TP/(TP + FP)$	62
r	Aturan yang menjadi anggota kebijakan π ; simbol ini kemudian juga digunakan untuk ukuran efek dan korelasi Pearson	62
$Recall$	$Recall$ keputusan kebijakan, $TP/(TP + FN)$	62
R^2	Koefisien determinasi pada regresi linear	83
$RSS(t)$	<i>Resident Set Size</i> proses pada waktu <i>sampling</i> t	62
t	Indeks/waktu <i>sampling</i> ; kemudian juga digunakan sebagai indeks iterasi pada parameter adaptif UBK	62
T	Struktur dependensi berupa pohon Chow-Liu	39
T_{init}	Waktu pembelajaran/inisialisasi awal hingga elit pertama terbentuk	63
T_{param}	Waktu siklus yang hanya menjalankan pembaruan parameter	63
T_{reopt}	Waktu re-optimalisasi setelah <i>drift</i> terdeteksi hingga konvergensi	63

LAMBANG	Nama	Pemakaian pertama kali pada halaman
T_{struct}	Waktu satu siklus yang turut melakukan pembelajaran ulang struktur	63
τ	Ambang divergensi Jensen-Shannon untuk memicu pembelajaran ulang struktur	37
w_{fp}	Bobot penalti <i>false positive</i> pada fungsi <i>fitness</i>	75
w_{wsc}	Bobot penalti kompleksitas struktural WSC pada fungsi <i>fitness</i>	38
x	Vektor biner kandidat aturan ABAC	38
$ \text{attr}(r) $	Banyaknya pasangan atribut-nilai yang menjadi syarat pada aturan r	62
$\emptyset$	Struktur/himpunan kosong pada inisialisasi	42

Bab I Pendahuluan

I.1 Latar Belakang

Perkembangan *Internet of Things* (IoT) telah mengubah cara perangkat, sensor, dan sistem saling terhubung dalam skala yang belum pernah terjadi sebelumnya. Ahsan dan Pathan (2025) mencatat bahwa jumlah perangkat IoT yang saling terhubung diproyeksikan melampaui 60 miliar unit pada tahun 2030, mencakup perangkat dengan karakteristik yang sangat heterogen baik dari sisi kapabilitas komputasi, sumber daya energi, maupun konteks penggunaannya. Pertumbuhan ini membawa konsekuensi serius pada aspek keamanan. Dritsas dan Trigka (2025) mengidentifikasi bahwa kerentanan pada lapisan autentikasi, integritas data, dan kontrol akses menjadi titik lemah utama ekosistem IoT, yang mana sebagian besar disebabkan oleh keterbatasan sumber daya perangkat yang tidak memungkinkan penerapan mekanisme keamanan konvensional secara langsung.

Salah satu aspek keamanan yang paling krusial dalam ekosistem IoT adalah kontrol akses, yaitu mekanisme yang menentukan entitas mana yang berhak mengakses sumber daya tertentu dalam kondisi apa. Model kontrol akses konvensional seperti *Role-Based Access Control* (RBAC) dan *Discretionary Access Control* (DAC) terbukti kurang memadai untuk lingkungan IoT yang bersifat dinamis, berskala besar, dan melibatkan konteks yang terus berubah (Ragothaman dkk., 2023). Sebagai responnya, *Attribute-Based Access Control* (ABAC) banyak diadopsi karena kemampuannya membuat keputusan akses berdasarkan kombinasi atribut subjek, objek, dan lingkungan secara fleksibel, sehingga lebih sesuai dengan karakteristik IoT yang heterogen dan dinamis (Ahsan dan Pathan, 2025).

Meskipun demikian, penerapan ABAC menghadapi tantangan tersendiri. Perancangan kebijakan (*policy*) ABAC secara manual merupakan proses yang kompleks dan mahal, sehingga banyak organisasi memilih bertahan dengan mekanisme kontrol akses lama meskipun kurang optimal, alih-alih menanggung biaya migrasi (Díaz-Rodríguez dan Díaz-Pérez, 2025). Untuk mengatasi hal ini, *policy mining*, proses ekstraksi kebijakan ABAC dari *log* akses dan data atribut yang sudah ada, menjadi pendekatan yang banyak diteliti. Dalam beberapa tahun terakhir, algoritma metaheuristik terbukti efektif untuk tugas ini. Siyuan dkk. (2025) mengusulkan pendekatan berbasis *Ant Colony Optimization* (ACO) untuk memigrasikan kebijakan

lintas sistem heterogen, sementara Li dkk. (2026) mengusulkan algoritma *Upgrade Black-Winged Kite* (UBK) yang mengungguli sejumlah metaheuristik lain dalam mengoptimalkan kebijakan ABAC pada konteks berbagi data. Namun, kedua pendekatan tersebut dirancang untuk dijalankan secara *batch/offline* pada lingkungan komputasi dengan sumber daya memadai (*server*), bukan pada perangkat *edge*-IoT yang terbatas.

Keterbatasan sumber daya pada perangkat *edge*-IoT bukanlah persoalan sepele. Canavese dkk. (2024) menegaskan bahwa mayoritas perangkat IoT beroperasi dengan keterbatasan memori, daya komputasi, dan energi yang signifikan, sehingga mekanisme keamanan, termasuk kontrol akses, harus dirancang secara ringan agar dapat dijalankan langsung pada perangkat tersebut tanpa bergantung sepenuhnya pada komputasi awan. Di ranah optimasi metaheuristik untuk komputasi *edge* secara umum, kebutuhan akan algoritma yang ringan dan adaptif ini sudah mulai direspons, misalnya melalui algoritma *Modified Greylag Goose Optimization* (MGGO) yang dirancang khusus untuk alokasi layanan IoT pada *edge computing* dengan mekanisme adaptif tanpa memerlukan pelatihan atau pemodelan eksplisit yang berat (Khosrowshahi dkk., 2025).

Namun demikian, prinsip serupa belum diterapkan pada *policy mining* ABAC. Algoritma metaheuristik ABAC yang ada saat ini, seperti ACO dan UBK, bersifat *population-based* dan tidak dirancang untuk berjalan berulang secara ringan pada perangkat berdaya rendah. Sementara upaya menjadikan *mining* kebijakan lebih efisien secara inkremental masih mengandalkan pendekatan berbasis aturan, bukan metaheuristik (Batra dkk., 2021 dan Bamberger dan Fernández, 2024), sehingga kehilangan kemampuan eksplorasi kombinatorial yang justru menjadi kekuatan utama metaheuristik.

Sebagai ilustrasi konkret, bayangkan sebuah *edge gateway* di sebuah *smart building* berbasis IoT yang harus memberlakukan kontrol akses ABAC untuk ratusan sumber daya, seperti sensor, aktuator, dan data, kepada berbagai pengguna serta perangkat. Selama ini, kebijakan akses pada sistem seperti itu biasanya dirancang manual oleh administrator. Cara tersebut mahal, lambat, subjektif, dan mudah menimbulkan kesalahan. Oleh karena itu, banyak organisasi lebih memilih bertahan dengan RBAC/DAC daripada menanggung biaya migrasi (Díaz-Rodríguez dan Díaz-Pérez, 2025). Mengabaikan kontrol akses juga berakibat nyata secara finansial. Studi IBM-Ponemon pada tahun 2024 yang dikutip oleh Díaz-Rodríguez dan Díaz-Pérez (2025) memperkirakan rata-rata biaya satu pelanggaran data mencapai USD 4,88 juta. *Policy mining* memang hadir untuk mengotomatisasi penyusunan kebijakan

dari log akses, tetapi pendekatan yang ada masih menghadapi dua keterbatasan di konteks *edge*. Pertama, dari segi skala, metode berbasis *association rule* dan sejenisnya mengalami penurunan kinerja pada log berukuran besar serta lonjakan jumlah aturan ketika atribut banyak (Xu dan Stoller, 2015 dan Cotrini Jimenez dkk., 2018). Akibatnya, bahkan metode mutakhir dirancang dan diuji untuk dijalankan pada server dengan sumber daya yang memadai. Kedua, dari segi dinamika, penambangan umumnya dilakukan secara *batch* atau sekali jalan. Padahal, lingkungan IoT terus berubah karena muncul perangkat, peran, dan pola akses baru, sehingga kebijakan menjadi cepat usang dan perlu diperbarui secara berkala. Menjalankan penambangan konvensional di server berarti log akses harus dikirim keluar perangkat yang mengakibatkan bertambahnya beban jaringan, latensi, dan risiko privasi. Sedangkan algoritma berpopulasi eksplisit yang jejak memorinya membesar tidak muat pada *edge gateway* yang terbatas. Kesenjangan inilah yang mendasari adanya penelitian ini untuk merancang sebuah algoritma yang secara khusus ringan atau *compact*, sadar terhadap korelasi antar-atribut ABAC, dan mampu memperbarui kebijakan secara inkremental, agar *policy mining* ABAC dapat dijalankan langsung pada perangkat *edge*-IoT tanpa mengorbankan kualitas kebijakan yang dihasilkan.

I.2 Rumusan Masalah

Berdasarkan kesenjangan penelitian yang teridentifikasi, rumusan masalah yang ada pada penelitian ini disusun sebagai berikut:

1. Mekanisme pemodelan probabilistik dan pembelajaran struktur dependensi seperti apa yang memungkinkan korelasi antar-atribut ABAC ditangkap tanpa menyimpan populasi solusi eksplisit, sehingga jejak memorinya tidak tumbuh terhadap ukuran populasi?
2. Bagaimana kualitas kebijakan ABAC yang dihasilkan algoritma usulan dibandingkan terhadap algoritma berpopulasi (BOA, ACO, UBK) dan *compact* murni (*compact-GA/PSO*) pada berbagai tingkat korelasi atribut, dan *trade-off* apa yang muncul antara kualitas kebijakan dan kehematan sumber daya?
3. Sejauh mana pemisahan frekuensi pembaruan struktur dependensi dari pembaruan parameter, termasuk strategi *warm-start* setelah *drift*, memengaruhi waktu pembaruan dan kualitas kebijakan pada berbagai tingkat *drift* data akses, dibandingkan pembaruan struktur setiap siklus dan penambangan ulang dari awal?
4. Bagaimana jejak memori algoritma usulan berperilaku terhadap ukuran popu-

lasi virtual (NP) dan dimensi ruang solusi (D) ketika dijalankan pada perangkat *edge* target, dan bagaimana kelayakan sumber dayanya dibandingkan algoritma berpopulasi eksplisit (BOA, ACO, UBK)?

Ketersediaan dataset yang memenuhi karakteristik pada rumusan masalah di atas dibahas dan dikonstruksi pada metodologi penelitian.

I.3 Tujuan Penelitian

Secara umum, penelitian ini bertujuan untuk merancang dan mengevaluasi sebuah algoritma metaheuristik yang ringan, sadar terhadap korelasi antar-atribut, dan mampu diperbarui secara inkremental yang diberi nama CoDA-EDA (*Compact Dependency-Aware Estimation of Distribution Algorithm*), untuk mendukung *policy mining Attribute-Based Access Control* (ABAC) pada perangkat *edge*-IoT berdaya rendah. Secara khusus, tujuan penelitian ini adalah untuk:

1. Menghasilkan mekanisme CoDA-EDA yang memodelkan dependensi antar-atribut ABAC tanpa menyimpan populasi solusi eksplisit.
2. Memperoleh bukti mengenai kualitas kebijakan CoDA-EDA beserta *trade-off*-nya terhadap algoritma berpopulasi (BOA, ACO, UBK) dan *compact* pada berbagai tingkat korelasi atribut.
3. Memperoleh bukti efektivitas mekanisme pembaruan kebijakan berkala terhadap waktu pembaruan dan kualitas kebijakan pada berbagai tingkat *drift*.
4. Memperoleh karakterisasi jejak memori CoDA-EDA terhadap ukuran populasi virtual (NP) dan dimensi ruang solusi (D), serta kelayakan sumber dayanya pada perangkat *edge* target dibanding algoritma berpopulasi eksplisit.

I.4 Batasan Masalah

Batasan berikut menetapkan ruang lingkup persoalan yang dikaji pada penelitian ini, antara lain:

1. Penelitian ini dibatasi pada tahap penambangan kebijakan (*policy mining*), yaitu mengekstraksi himpunan aturan ABAC dari log akses yang telah tersedia. Tahap penegakan kebijakan (*Policy Enforcement Point*), instrumentasi/pengumpulan log, dan administrasi siklus hidup kebijakan berada di luar lingkup persoalan.

2. Persoalan dibatasi pada penemuan aturan ABAC berbentuk konjungsi dari kondisi atribut subjek, objek, dan lingkungan yang memetakan permintaan akses ke keputusan izin/tolak. Bentuk kebijakan lain seperti kebijakan berprioritas/berbobot tidak termasuk persoalan yang dikaji.
3. Persoalan pemodelan ketergantungan antar-atribut yang ditelaah dibatasi pada keterkaitan berpasangan (orde satu). Ketergantungan orde tinggi berada di luar persoalan yang dijawab oleh penelitian ini.
4. Kualitas kebijakan dimaknai sebagai fidelitas rekonstruksi, yaitu seberapa tepat himpunan aturan hasil penambangan memetakan kembali keputusan izin/tolak pada log yang menjadi sumbernya. Kemampuan generalisasi ke permintaan akses yang belum pernah muncul maupun kebenaran semantik kebijakan terhadap maksud organisasi berada di luar lingkup persoalan.
5. Persoalan dibatasi pada konteks *edge-IoT* bersumber daya terbatas yang menuntut penambangan berjalan langsung pada perangkat dan kebijakan diperbarui secara berkala mengikuti perubahan distribusi data *drift*. Konteks komputasi server atau penambangan *batch/offline* tanpa kendala sumber daya tidak menjadi persoalan yang dikaji.

I.5 Premis dan Hipotesis

Hipotesis penelitian ini disusun secara deduktif dari sejumlah premis yang berakar pada Latar Belakang Masalah dan Rumusan Masalah. Premis-premis tersebut adalah sebagai berikut:

1. **Premis 1:** Perangkat *edge-IoT* beroperasi dengan keterbatasan memori, daya komputasi, dan energi yang signifikan, sehingga mekanisme keamanan perlu dirancang secara ringan agar dapat dijalankan langsung pada perangkat tersebut (Canavese dkk., 2024).
2. **Premis 2:** Algoritma *compact* yang ada saat ini (*compact-GA*, *compact-PSO*) mencapai efisiensi memori dengan mengasumsikan independensi penuh antar-atribut, sedangkan algoritma yang mampu menangkap dependensi antar-atribut (BOA, iBOA) tidak dibatasi kebutuhan memorinya sehingga tidak sesuai untuk perangkat berdaya rendah. Dalam teori *Estimation of Distribution Algorithm* (EDA) secara umum, kemampuan memodelkan dependensi antar-variabel diketahui meningkatkan kualitas solusi pada domain dengan korelasi antar-variabel yang tinggi dibandingkan model yang mengasumsikan independensi

penuh.

3. **Premis 3:** Lingkungan IoT bersifat dinamis, berskala besar, dan melibatkan konteks yang terus berubah (Ragothaman dkk., 2023), sehingga kebijakan ABAC hasil *mining* berpotensi memerlukan pembaruan berkala agar tetap relevan terhadap kondisi akses terkini. Pembaruan struktur dependensi secara menyeluruh pada setiap perubahan data, sebagaimana pendekatan BOA/iBOA konvensional, membutuhkan biaya komputasi yang signifikan dan tidak efisien apabila dilakukan berulang pada perangkat berdaya rendah.
4. **Premis 4:** Kualitas kebijakan hasil *policy mining* bergantung pada kesesuaian model probabilistik yang digunakan dengan struktur dependensi sesungguhnya pada data akses. Dengan demikian, algoritma yang tetap mempertahankan pemodelan dependensi diperkirakan dapat menghasilkan kebijakan yang lebih mendekati kualitas algoritma *population-based* tanpa batasan orde (ACO, UBK) dibandingkan algoritma yang mengasumsikan independensi penuh.

Berdasarkan keempat premis tersebut, hipotesis penelitian yang diajukan adalah sebagai berikut:

1. **H1** – CoDA-EDA diharapkan dapat menghasilkan kualitas kebijakan (*F-score*) yang kompetitif dibandingkan dengan keenam algoritma pembanding lainnya (ACO, UBK, BOA, iBOA, *compact-GA*, *compact-PSO*).
2. **H2** – Jejak memori CoDA-EDA tetap datar terhadap ukuran populasi virtual (NP), sehingga diperkirakan secara signifikan lebih rendah dibanding algoritma berpopulasi pada NP besar.
3. **H3** – Pemisahan pembaruan struktur dari pembaruan parameter mengurangi total waktu komputasi dengan kualitas kebijakan yang non-inferior.
4. **H4** – Pada skenario *drift* rendah hingga sedang, CoDA-EDA dengan *warm-start* mencapai kualitas kebijakan non-inferior terhadap *re-mining* penuh dalam waktu re-optimasi yang jauh lebih singkat.

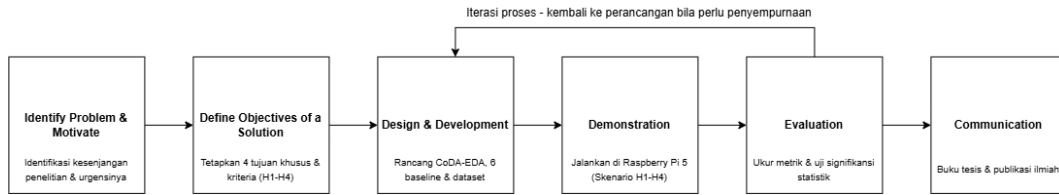
I.6 Metodologi Penelitian

Penelitian ini menggunakan pendekatan *Design Science Research Methodology* (DSRM), yaitu paradigma yang berorientasi pada perancangan dan evaluasi artefak untuk menyelesaikan permasalahan yang teridentifikasi, alih-alih sekadar menjelaskan atau memprediksi fenomena yang telah berlangsung (Peppers dkk., 2007). Titik masuk yang diikuti adalah *problem-centered initiation*, karena penelitian ini bermula

dari kesenjangan yang teridentifikasi pada tinjauan pustaka. DSRM menjadi payung paradigma, sedangkan aktivitas *Demonstration* dan *Evaluation* dioperasionalkan melalui desain eksperimen kuantitatif terkontrol. Jadi artefak akan dibandingkan dengan enam algoritma *baseline* pada dataset yang dikonstruksi, diukur dengan metrik yang ditetapkan, dan diuji signifikansinya secara statistik. Proses penelitian mengikuti enam aktivitas DSRM (Peffer dkk., 2007) berikut, yang alurnya divisualisasikan pada Gambar I.1:

1. *Problem identification and motivation* – Mengidentifikasi kesenjangan penelitian, yaitu belum tersedianya algoritma metaheuristik untuk *policy mining* ABAC yang sekaligus ringan, sadar dependensi, dan dapat diperbarui inkremental pada perangkat *edge-IoT* beserta urgensinya.
2. *Define objectives of a solution* – Menetapkan empat tujuan khusus beserta kriteria keberhasilan terukur yang dirumuskan sebagai Hipotesis H1-H4.
3. *Design and development* – Merancang dan mengimplementasikan artefak penelitian. Aktivitas ini paling ekstensif dan mencakup tiga komponen: (a) perancangan algoritma CoDA-EDA itu sendiri – representasi model probabilistik, mekanisme pembaruan kompetitif tanpa populasi eksplisit, dan strategi pembaruan struktur dependensi berbasis delta; (b) implementasi enam algoritma pembanding secara setara agar perbandingan pada tahap *Evaluation* tidak bias oleh perbedaan kualitas implementasi; dan (c) konstruksi dataset ABAC-Lab dengan tingkat korelasi atribut dan *drift* terkontrol, mengingat dataset semacam ini belum tersedia sebagai sumber daya siap pakai.
4. *Evaluation* – Mengukur dan membandingkan hasil demonstrasi terhadap metrik (kualitas kebijakan/*F-score* dan WSC, jejak memori, serta waktu komputasi), disertai uji signifikansi statistik perbandingan antar-algoritma terhadap H1-H4.
5. *Communication* – Mengomunikasikan permasalahan, artefak CoDA-EDA, dan hasil evaluasinya melalui buku tesis serta naskah publikasi ilmiah terkait.

Meskipun keenam aktivitas disajikan berurutan, proses DSRM bersifat iteratif, sehingga penulis dimungkinkan kembali ke aktivitas *Design and Development* apabila hasil *Demonstration* dan *Evaluation* menunjukkan kebutuhan penyempurnaan rancangan (Peffer dkk., 2007).



Gambar I.1 Enam Aktivitas DSRM dan Pemetaannya terhadap Struktur Penelitian

I.7 Sistematika Penulisan

Penulisan tesis ini disusun dalam lima bab sebagai berikut.

Bab I Pendahuluan membahas latar belakang, rumusan masalah, tujuan penelitian, hipotesis, batasan masalah, metodologi penelitian secara ringkas, dan sistematika penulisan.

Bab II Tinjauan Pustaka dan Landasan Teori membahas landasan teori mengenai *Internet of Things* dan keterbatasan sumber daya perangkat *edge*, kontrol akses berbasis atribut (ABAC) beserta penambangan kebijakannya, algoritma metaheuristik dan *Estimation of Distribution Algorithm* (EDA) beserta variannya yang ringan dan sadar dependensi, posisi penelitian terhadap literatur terdahulu, argumentasi kebaruan, serta kerangka berpikir penelitian.

Bab III Metodologi Penelitian menjelaskan pendekatan *Design Science Research* yang digunakan, tahapan penelitian, variabel penelitian, rancangan algoritma CoDA-EDA yang diusulkan, konstruksi dataset, algoritma pembandingan, perangkat dan lingkungan eksperimen, desain eksperimen, metrik dan instrumen evaluasi, teknik analisis data, serta studi pendahuluan yang mencakup analisis karakteristik dataset dan kalibrasi rancangan.

Bab IV Hasil dan Pembahasan menyajikan hasil pengujian empiris keempat hipotesis pada kelima dataset ABAC Lab, validasi eksternal pada dataset [Smart Building IoT], contoh kebijakan hasil *mining*, serta sintesis temuan, keterbatasan, dan jawaban atas rumusan masalah.

Bab V Kesimpulan dan Saran memuat memuat kesimpulan penelitian yang menjawab tujuan dan rumusan masalah beserta kontribusi yang dicapai, serta saran penerapan praktis dan pengembangan lanjutan.

Bab II Tinjauan Pustaka dan Landasan Teori

II.1 *Internet of Things* dan Keterbatasan Sumber Daya Perangkat *Edge*

Internet of Things (IoT) merujuk pada jaringan objek fisik, mulai dari sensor, aktuator, hingga perangkat pintar yang saling terhubung dan mampu bertukar data melalui internet tanpa memerlukan interaksi manusia secara langsung. Ahsan dan Pathan (2025) mencatat bahwa jumlah perangkat IoT yang saling terhubung diproyeksikan melampaui 60 miliar unit pada tahun 2030 dan mencakup perangkat dengan karakteristik yang sangat heterogen, baik dari sisi kapabilitas komputasi, sumber daya energi, maupun konteks penggunaannya.

Guna mengelola kompleksitas jaringan IoT yang heterogen tersebut, berbagai arsitektur aplikasi telah dikembangkan. Dauda dkk. (2024) mengklasifikasikan arsitektur aplikasi IoT berdasarkan model *deployment*-nya menjadi tiga kategori utama, yakni *cloud*, *edge*, dan *fog computing*. Arsitektur *cloud* memanfaatkan pemrosesan dan penyimpanan data terpusat untuk mendukung aplikasi IoT berskala besar, tetapi kerap menghadapi latensi tinggi dan keterbatasan *bandwidth* akibat jarak komunikasi antara perangkat dan pusat data. Arsitektur *edge computing* mengatasi persoalan ini dengan memindahkan komputasi lebih dekat ke sumber data, sehingga meningkatkan pemrosesan *real-time* dan mengurangi kepadatan jaringan, sementara arsitektur *fog* menggabungkan kekuatan kedua pendekatan tersebut untuk lingkungan IoT yang lebih kompleks.

Namun demikian, pemindahan komputasi ke *edge* tidak serta-merta menghilangkan tantangan sumber daya. Ia justru memindahkan tantangan tersebut ke perangkat itu sendiri. Canavese dkk. (2024) menegaskan bahwa mayoritas perangkat IoT yang beroperasi di *edge* memiliki keterbatasan memori, daya komputasi, dan energi yang signifikan, sehingga mekanisme apa pun yang dijalankan di atasnya harus dirancang secara ringan agar dapat berjalan langsung pada perangkat tersebut tanpa bergantung sepenuhnya pada komputasi awan.

Kebutuhan akan pendekatan yang ringan dan adaptif ini sudah mulai direspons dalam ranah optimasi metaheuristik untuk *edge computing* secara umum. Khosrowshahi dkk. (2025) misalnya, mengusulkan algoritma *Modified Greylag Goose Optimization* (MGGO) yang dirancang khusus untuk alokasi layanan IoT pada *edge computing*

dengan mekanisme adaptif tanpa memerlukan pelatihan atau pemodelan eksplisit yang berat. Hal tersebut menunjukkan bahwa keterbatasan sumber daya *edge* dapat diatasi melalui desain algoritma yang tepat, bukan hanya melalui penambahan kapasitas perangkat keras.

Ketiga dimensi keterbatasan sumber daya perangkat *edge* tersebut menjadi premis dasar penelitian ini dan melandasi kebutuhan akan algoritma metaheuristik yang secara khusus ringan untuk tugas *policy mining* ABAC.

II.2 Kontrol Akses dan *Attribute-Based Access Control* (ABAC)

Kontrol akses adalah mekanisme keamanan yang menentukan entitas mana yang berhak melakukan operasi tertentu terhadap suatu sumber daya dan dalam kondisi apa. Model kontrol akses klasik yang paling banyak diterapkan mencakup *Discretionary Access Control* (DAC), *Mandatory Access Control* (MAC), dan *Role-Based Access Control* (RBAC). Pada DAC, pemilik sumber daya atau *data owner* memiliki kendali penuh untuk memberikan atau mencabut hak akses kepada pengguna lain, biasanya melalui *Access Control List* (ACL) yang harus diperbarui secara manual oleh administrator setiap kali terjadi perubahan (Ragothaman dkk., 2023). Karena sifatnya yang statis ini, DAC dinilai tidak sesuai untuk lingkungan yang dinamis seperti IoT (Ragothaman dkk., 2023 dan Ahsan dan Pathan, 2025).

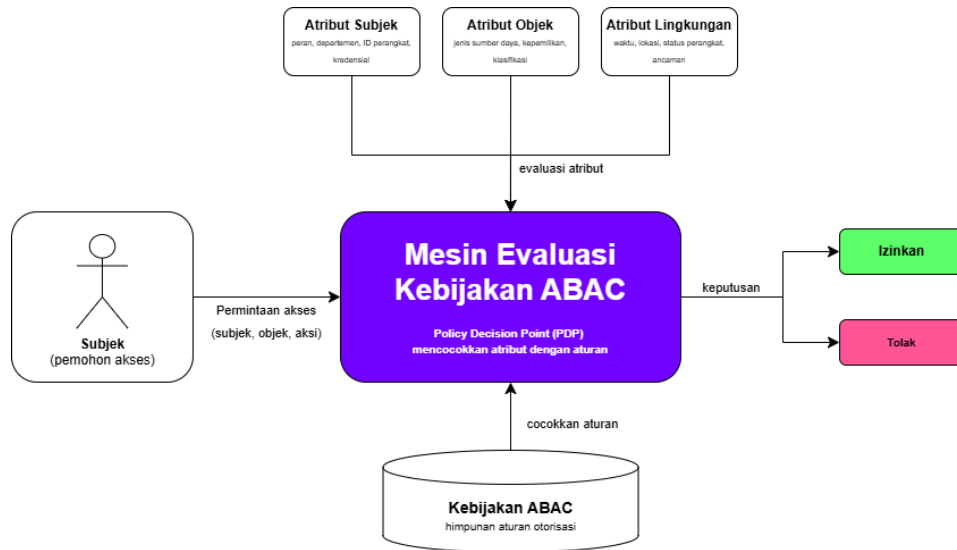
Berbeda dari DAC, *Mandatory Access Control* (MAC) menempatkan keputusan akses pada kebijakan terpusat yang ditetapkan oleh sistem melalui label keamanan, misalnya tingkat klasifikasi pada objek dan tingkat kewenangan *clearance* pada subjek, alih-alih pada kebijaksanaan pemilik sumber daya. Model ini memberikan kontrol yang ketat dan konsisten sehingga banyak dipakai pada lingkungan militer dan sistem *multi-level security*. Namun, sifatnya yang kaku dan ketergantungannya pada pengelolaan label secara terpusat membuatnya sulit menyesuaikan diri dengan lingkungan yang heterogen, berskala besar, dan konteksnya terus berubah seperti IoT, karena setiap penambahan atau perubahan label harus dikelola oleh otoritas pusat (Ragothaman dkk., 2023).

Role-Based Access Control (RBAC) berupaya mengatasi keterbatasan kedua model tersebut dengan mengaitkan hak akses pada peran atau *role* yang disandang pengguna, alih-alih pada identitas individu sebagaimana DAC maupun label keamanan yang kaku sebagaimana MAC. Namun, pendekatan berbasis peran ini kembali menemui hambatan pada lingkungan IoT yang sangat heterogen dan berskala besar, yakni jumlah kombinasi peran yang dibutuhkan untuk merepresentasikan seluruh konteks

akses yang mungkin dapat membengkak tak terkendali atau disebut *role explosion*. Sementara peran itu sendiri tidak cukup fleksibel untuk merepresentasikan kondisi kontekstual seperti waktu akses, lokasi, atau status perangkat (Ahsan dan Pathan, 2025).

Sebagai respons atas keterbatasan tersebut, *Attribute-Based Access Control* (ABAC) dikembangkan sebagai model kontrol akses alternatif. Menurut definisi resmi *National Institute of Standards and Technology* (NIST) dalam *Special Publication* 800-162, ABAC adalah metodologi kontrol akses logis di mana otorisasi untuk melakukan sekumpulan operasi ditentukan dengan mengevaluasi atribut-atribut yang terkait dengan subjek, objek, operasi yang diminta, dan kondisi lingkungan, terhadap kebijakan, aturan, atau relasi yang menjelaskan operasi yang diizinkan untuk kombinasi atribut tertentu (Hu dkk., 2014). NIST juga menegaskan bahwa ABAC pada dasarnya mampu mengakomodasi baik konsep DAC maupun MAC sekaligus, sembari menyediakan jumlah kombinasi input keputusan akses yang jauh lebih besar dibandingkan model berbasis peran.

Secara operasional, Díaz-Rodríguez dan Díaz-Pérez (2025) menjabarkan tiga komponen utama ABAC. Yang pertama adalah atribut (*attribute*), yaitu karakteristik yang melekat pada suatu entitas, baik subjek (pemohon akses), objek (sumber daya yang diakses), maupun lingkungan (*environment*). Yang kedua adalah aturan (*rule*) ABAC, yang menentukan apakah suatu objek dapat mengakses suatu sumber daya dalam kondisi lingkungan tertentu. Dan yang terakhir adalah kebijakan (*policy*) ABAC, yaitu himpunan aturan otorisasi yang didefinisikan atas ketiga jenis atribut tersebut. Ketika permintaan akses diterima, mesin ABAC mengevaluasi atribut subjek, atribut objek, dan kondisi lingkungan terhadap kebijakan yang berlaku untuk memutuskan apakah akses diizinkan atau ditolak. Alur pengambilan keputusan akses berdasarkan ketiga komponen tersebut diilustrasikan pada Gambar II.1



Gambar II.1 Arsitektur Keputusan Akses ABAC

Gambar II.1 memperlihatkan bahwa keputusan akses pada ABAC tidak ditentukan oleh identitas subjek secara langsung, melainkan oleh hasil evaluasi kombinasi atribut subjek, objek, dan lingkungan terhadap himpunan aturan pada kebijakan ABAC. Struktur inilah yang memberi ABAC fleksibilitas untuk merepresentasikan konteks akses yang kompleks, sekaligus menegaskan bahwa kualitas keputusan akses sangat bergantung pada kualitas himpunan aturan yang menyusun kebijakan tersebut.

Karakteristik ABAC ini menjadikannya sangat sesuai untuk ekosistem IoT yang heterogen dan dinamis. Keputusan akses dapat mempertimbangkan atribut perangkat (mis. jenis, lokasi, status baterai), atribut pengguna, dan kondisi lingkungan (mis. waktu, tingkat ancaman) tanpa perlu mendefinisikan peran secara eksplisit untuk setiap kombinasi konteks yang mungkin (Ahsan dan Pathan, 2025 dan Ragothaman dkk., 2023). Meskipun fleksibel, penerapan ABAC menghadapi tantangan operasionalnya sendiri, yaitu proses spesifikasi kebijakan yang kompleks dan mahal apabila dilakukan secara manual (Díaz-Rodríguez dan Díaz-Pérez, 2025).

II.3 *Policy Mining* pada ABAC

Sebagaimana yang sudah disinggung, spesifikasi kebijakan ABAC secara manual merupakan proses yang mahal dan sulit diskalakan pada sistem berskala besar. *Policy mining* hadir sebagai solusi otomatis atas persoalan ini. Perez-Haro dan Diaz-Perez (2024) mendefinisikan *policy mining* sebagai prosedur otomatis untuk menghasilkan aturan akses dengan menambang pola atribut-nilai dari *log* akses, yaitu catatan permintaan akses yang telah diberikan izin atau ditolak oleh sistem. Secara lebih

formal, Díaz-Rodríguez dan Díaz-Pérez (2025) merumuskan masalah *policy mining* ABAC sebagai berikut: diberikan sebuah *log akses L*, tujuan *policy mining* ABAC adalah menghasilkan kebijakan ABAC yang secara akurat merepresentasikan pola perizinan yang terdapat pada *L*.

Sebagai ilustrasi, berikut merupakan contoh potongan log akses *L* pada sistem rekam medis rumah sakit. Setiap baris adalah satu permintaan akses beserta nilai atribut subjek dan objek, aksi yang diminta, dan keputusan yang tercatat.

Tabel II.1 Contoh potongan log akses ($\mathcal{L}$)

#	peran (subjek)	unit (subjek)	jenis objek	unit (objek)	aksi	keputusan
1	perawat	ICU	rekam_medis	ICU	baca	permit
2	dokter	ICU	rekam_medis	ICU	baca	permit
3	perawat	UGD	rekam_medis	UGD	baca	permit
4	perawat	ICU	rekam_medis	UGD	baca	deny
5	staf_admin	ICU	rekam_medis	ICU	baca	deny
6	dokter	ICU	resep_obat	ICU	tulis	permit
7	perawat	ICU	resep_obat	ICU	tulis	deny

Dari pola perizinan pada *L*, *policy mining* berupaya menemukan kebijakan berupa himpunan aturan ringkas yang memetakan setiap permintaan ke keputusan yang sama seperti tercatat. Untuk potongan di atas, kebijakan berikut sudah memadai:

R1. (baca rekam medis oleh klinisi pada unit yang sama). JIKA peran $\in \{\text{perawat, dokter}\}$ DAN jenis objek = *rekam_medis* DAN aksi = *baca* DAN $\text{unit(subjek)} = \text{unit(objek)} \rightarrow \text{izinkan}$.

R2. (penulisan resep oleh dokter). JIKA peran = *dokter* DAN jenis objek = *resep_obat* DAN aksi = *tulis* DAN $\text{unit(subjek)} = \text{unit(objek)} \rightarrow \text{izinkan}$.

Permintaan yang tidak dicakup aturan mana pun ditolak secara baku (*default-deny*).

Kebijakan R1, R2 mengizinkan tepat keempat permintaan ber-*permit* dan menolak

seluruh permintaan ber-*deny*. Karena semua *permit* tercakup dan tak ada *deny* yang keliru diizinkan, kebijakan ini merekonstruksi L secara sempurna. Inilah yang diukur sebagai *F-score* (rata-rata harmonik presisi dan *recall* aturan terhadap L). Adapun *Weighted Structural Complexity* (WSC) kebijakan ini dihitung dari banyaknya pasangan atribut-nilai yang menyusun kedua aturan. Kebijakan dengan WSC lebih rendah lebih ringkas dan lebih mudah dikelola administrator. Kedua metrik inilah yang dipakai untuk menilai kualitas kebijakan pada Hipotesis H1.

Berbagai pendekatan telah dikembangkan untuk menyelesaikan masalah ini. Díaz-Rodríguez dan Díaz-Pérez (2025) merangkum bahwa pendekatan-pendekatan tersebut umumnya berbasis salah satu dari beberapa strategi ekstraksi aturan. Pendekatannya antara lain adalah *association rule mining* yang dipelopori oleh Xu dan Stoller (2015) yang membentuk aturan secara progresif dari *log permission* namun kinerjanya menurun pada *log* berukuran besar. Pendekatan lain yang digunakan seperti pembelajaran mesin, seperti yang dilakukan oleh Iyer dan Masoumzadeh (2018) dengan melakukan pendekatan berbasis algoritma PRISM, atau kombinasi *data mining*-statistik yang dilakukan oleh Abu Jabal dkk. (2020), dan deteksi kluster dengan mengelompokkan permintaan akses sebelum aturan dibentuk. Perez-Haro dan Diaz-Perez (2024) menyoroti bahwa mayoritas pendekatan tersebut mengandalkan strategi berbasis frekuensi, yaitu menetapkan ambang dukungan minimum untuk membentuk aturan. Namun pendekatan tersebut menghadapi *trade-off* yang nyata. Ambang tinggi dapat menyisakan banyak sumber daya tanpa aturan, khususnya yang jarang diakses. Sedangkan ambang rendah dapat memicu ledakan aturan atau *rule explosion* yang tidak andal. Sebagai alternatif, mereka mengusulkan pemodelan *log* akses sebagai jaringan afiliasi dan menerapkan analisis *biclique* untuk mengekstraksi aturan tanpa memerlukan ambang frekuensi.

Kualitas kebijakan hasil *policy mining* lazim diukur melalui dua dimensi utama, yakni akurasi dan kompleksitas struktural. Akurasi biasanya diukur menggunakan *F-score*, yaitu rata-rata harmonik antara presisi dan *recall* aturan yang diekstraksi terhadap *log* akses asli (Díaz-Rodríguez dan Díaz-Pérez, 2025). Kompleksitas struktural diukur melalui *Weighted Structural Complexity* (WSC), sebuah metrik yang awalnya dirumuskan oleh Molloy dan Reed (1995) untuk *role mining* pada RBAC dan kemudian diadaptasi oleh Xu dan Stoller (2015) untuk konteks ABAC (Perez-Haro dan Diaz-Perez, 2024). WSC pada dasarnya menjumlahkan banyaknya pasangan atribut-nilai yang digunakan untuk mendeskripsikan seluruh aturan dalam suatu kebijakan, sehingga kebijakan dengan WSC rendah lebih ringkas dan lebih mudah dikelola oleh administrator.

Kedua metrik ini, akurasi dan WSC, konsisten dengan metrik kualitas kebijakan yang telah ditetapkan pada Hipotesis 1 untuk mengevaluasi CoDA-EDA terhadap *baseline compact-GA/compact-PSO*. Adapun pendekatan metaheuristik *population-based* seperti ACO (Siyuan dkk., 2025) dan UBK (Li dkk., 2026) yang menjadi *baseline* penelitian ini merupakan varian lain dari strategi ekstraksi aturan berbasis pencarian (*search-based*), berbeda dari strategi *association-rule*, *machine learning*, kluster, maupun *graph-based* yang sudah diuraikan di atas.

II.4 Algoritma Metaheuristik untuk Optimasi Kombinatorial

Optimasi kombinatorial merujuk pada persoalan pencarian solusi terbaik dari ruang solusi berukuran diskret dan sangat besar, yang pada umumnya tergolong *NP-hard* sehingga tidak dapat diselesaikan secara eksak dalam waktu yang wajar untuk instansi berskala besar. Algoritma metaheuristik hadir sebagai pendekatan pencarian heuristik tingkat tinggi yang mampu mendekati solusi optimal tanpa jaminan optimalitas formal, dengan biaya komputasi yang jauh lebih rendah dibanding pendekatan eksak. Rajwar dkk. (2023) mencatat bahwa lebih dari 500 algoritma metaheuristik baru telah dikembangkan hingga saat studi mereka dilakukan, dengan lebih dari 350 di antaranya muncul dalam dekade terakhir. Angka tersebut menandakan betapa pesatnya pertumbuhan di bidang ini.

Berdasarkan sumber inspirasinya, Rajwar dkk. (2023) mengklasifikasikan algoritma metaheuristik ke dalam empat kategori utama. Kategori pertama adalah algoritma evolusioner (*evolutionary algorithms*) yang terinspirasi oleh seleksi alam Darwin, misal *Genetic Algorithm* dan *Differential Evolution*. Kategori kedua adalah algoritma kecerdasan (*swarm intelligence*) yang terinspirasi oleh perilaku kolektif kelompok organisme, misal *Particle Swarm Optimization* dan *Ant Colony Optimization*. Kategori yang ketiga adalah algoritma berbasis hukum fisika atau kimia, seperti *Simulated Annealing* dan *Gravitational Search Algorithm*. Kategori terakhir adalah kategori lain-lain yang mencakup algoritma yang terinspirasi dari perilaku manusia, strategi permainan, maupun teorema matematis. Jakšić dkk. (2023) menegaskan bahwa klasifikasi semacam ini penting namun relatif jarang dan kerap tidak konsisten antar-penulis, sehingga kajian taksonomi seperti ini tetap perlu terus diperbarui seiring pesatnya publikasi metode baru.

Selain berdasarkan sumber inspirasi, Rajwar dkk. (2023) turut mengklasifikasikan algoritma metaheuristik berdasarkan ukuran populasi pencarian menjadi dua kategori, yakni algoritma berbasis lintasan (*trajectory-based*) yang menggunakan satu agen pencarian tunggal yang bergerak secara bertahap melalui ruang solusi (mis. *Simula-*

ted Annealing dan *Tabu Search*), dan algoritma berbasis populasi (*population-based*) yang menggunakan banyak agen pencarian secara simultan sehingga memungkinkan eksplorasi ruang solusi yang jauh lebih luas (mis. *Genetic Algorithm*, *Particle Swarm Optimization*, *Ant Colony Optimization*).

Kedua algoritma metaheuristik yang menjadi *baseline population-based* penelitian ini, yakni ACO dan UBK, tergolong dalam kategori algoritma kecerdasan kawanan berbasis populasi menurut taksonomi di atas. ACO terinspirasi oleh perilaku koloni semut dalam mencari jalur makanan terpendek, sementara UBK terinspirasi oleh strategi berburu burung *black-winged kite*. Keduanya memanfaatkan banyak agen pencarian yang berinteraksi untuk mengeksplorasi ruang kebijakan ABAC secara paralel, sebagaimana ciri umum algoritma *population-based* yang diuraikan Rajwar dkk. (2023).

Namun demikian, kemampuan eksplorasi luas melalui banyak agen pencarian yang menjadi kekuatan utama algoritma *population-based* ini juga menjadi sumber utama kebutuhan memori. Jadi setiap agen dalam populasi umumnya perlu direpresentasikan dan disimpan secara eksplisit sepanjang proses optimasi, sehingga jejak memori bertumbuh sebanding dengan ukuran populasi. Pada perangkat *edge-IoT* dengan keterbatasan memori, karakteristik ini menjadi kendala tersendiri.

II.5 *Estimation of Distribution Algorithm*

Estimation of Distribution Algorithm (EDA) merupakan kelas algoritma evolusioner yang berbeda secara mendasar dari algoritma genetika maupun algoritma kecerdasan kawanan seperti ACO dan UBK. Alih-alih menggunakan operator pindah silang (*crossover*) dan mutasi untuk menghasilkan generasi baru, EDA, atau juga dikenal sebagai *probabilistic model-building genetic algorithm* (PMBGA), membangun model probabilistik eksplisit dari individu-individu terbaik pada setiap generasi, kemudian men-*sampling* model tersebut untuk menghasilkan populasi generasi berikutnya (Larrañaga dan Bielza, 2024 dan Lemtenneche dkk., 2023). Dengan demikian, proses eksplorasi ruang solusi dipandu langsung oleh distribusi probabilitas yang dipelajari dari data, bukan oleh operator variasi yang bersifat implisit.

Secara umum, EDA bekerja melalui tiga langkah yang diulang pada setiap generasi (Larrañaga dan Bielza, 2024). Pertama seleksi, yaitu memilih sejumlah N individu terbaik dari populasi berukuran M berdasarkan nilai *fitness*. Kedua estimasi, yaitu membangun distribusi probabilitas yang merepresentasikan individu-individu terpilih tersebut. Pada persoalan kombinatorial/diskret, distribusi ini umumnya direpresenta-

sikan melalui jaringan Bayes (*Bayesian network*), yakni graf asiklik berarah yang merepresentasikan struktur (in)dependensi bersyarat antar-variabel secara ringkas dan dengan jumlah parameter yang jauh lebih sedikit dibanding tabel probabilitas gabungan penuh. Dan ketiga adalah *sampling*, yaitu membangkitkan M individu populasi baru dengan men-*sampling* dari distribusi yang telah diestimasi. Ketiga langkah ini diulang hingga kriteria pemberhentian terpenuhi.

Berdasarkan derajat dependensi antar-variabel yang diperbolehkan dalam pemodelan probabilistiknya, Larrañaga dan Bielza (2024) mengkalsifikasikan EDA ke dalam tiga kategori. Yang pertama adalah EDA univariat yang mengasumsikan seluruh variabel saling independen. Yang kedua adalah EDA bivariat, yang hanya memperhitungkan dependensi berpasangan antar-variabel. Dan yang terakhir adalah EDA multivariat, yang tidak membatasi derajat dependensi antar-variabel dan umumnya mempelajari struktu jaringan Bayes secara penuh.

EDA telah banyak diterapkan pada persoalan optimasi kombinatorial, termasuk persoalan berbasis permutasi yang menghadirkan tantangan tersendiri akibat besarnya ruang solusi dan batasan *mutual exclusivity* antar-elemen (Lemtenneche dkk., 2023). Meskipun ruang kebijakan ABAC yang menjadi objek penelitian ini tidak berbasis permutasi, karakteristiknya tetap tergolong kombinatorial. Setiap kandidat kebijakan merupakan kombinasi diskret dari nilai-nilai atribut subjek, objek, aksi, dan lingkungan. Prinsip yang sama yakni dengan memodelkan distribusi probabilitas atas konfigurasi-konfigurasi yang menjanjikan, sebagaimana EDA memodelkan distribusi solusi permutasi yang menjanjikan pada persoalan penjadwalan *flow-shop* pada Lemtenneche dkk. (2023), berpotensi diadaptasi untuk memandu pencarian aturan ABAC secara terarah.

Namun demikian, skema umum EDA di atas tetap mensyaratkan penyimpanan populasi sebanyak M individu secara eksplisit pada setiap generasi sehingga EDA dalam bentuk kanoniknya belum secara otomatis menjawab persoalan keterbatasan memori pada perangkat *edge-IoT*. Perbedaan mendasarnya terletak pada apa yang direpresentasikan sebagai keadaan pencarian, yakni bukan pada operator variasi implisit, melainkan parameter model probabilistik itu sendiri. Karakteristik inilah yang membuka peluang bagi varian EDA ringan untuk menggantikan penyimpanan populasi eksplisit dengan sebuah vektor probabilitas tunggal yang diperbarui secara inkremental, sehingga jejak memori algoritma dapat ditekan di bawah kebutuhan populasi penuh.

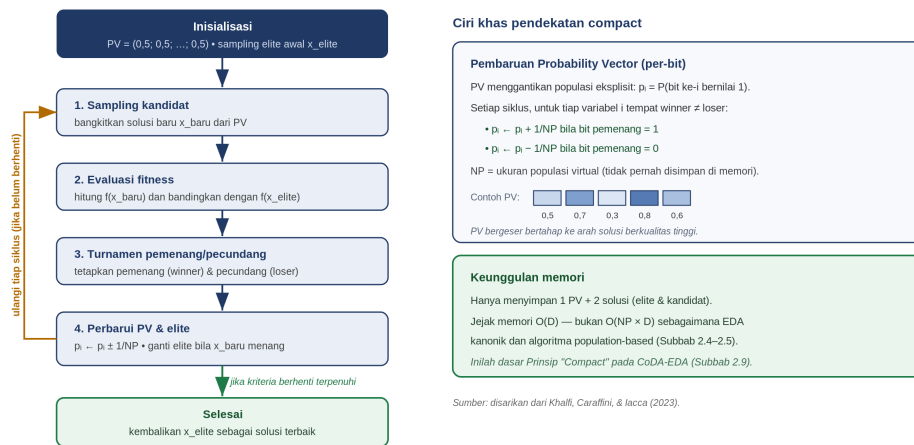
II.6 EDA Ringan/*Compact*

Sebagaimana yang sudah diuraikan, skema EDA kanonik tetap mensyaratkan penyimpanan populasi penuh pada setiap generasi dan celah inilah yang dijawab oleh kelas algoritma *compact*. Khalfi dkk. (2023) mendefinisikan algoritma *compact* sebagai salah satu ekspresi paling sederhana dari algoritma EDA, yang menggantikan penyimpanan populasi eksplisit dengan sebuah *Probability Vector* (PV) tunggal yang merepresentasikan model probabilistik dari sebuah populasi virtual berukuran NP , tanpa pernah benar-benar menyimpan NP individu tersebut di memori. Signifikansi persoalan ini dikonkretkan Khalfi dkk. (2023) melalui contoh nyata. Pada mikrokontroler ATmega328P dengan RAM 2 KB, sebuah algoritma genetika standar dengan populasi 128 individu ber-*encoding* 32-bit menghabiskan sekitar 80% dari seluruh memori yang tersedia dan tidak menyisakan ruang bagi proses lain yang berjalan pada perangkat yang sama. Ilustrasi ini secara langsung menggambarkan tantangan memori pada perangkat *edge* IoT.

Secara umum, algoritma *compact* bekerja melalui *template* berikut (Khalfi dkk., 2023):

- PV diinisialisasi, kemudian sebuah solusi elit awal disampling darinya.
- Pada setiap iterasi, sebuah solusi kandidat baru disampling dari PV.
- *Fitness*-nya dibandingkan dengan solusi elit saat ini melalui skema turnamen pemenang/pecundang.
- PV kemudian diperbarui berdasarkan selisih antara nilai pemenang dan pecundang pada tiap variabel, dengan besaran pembaruan berbanding terbalik dengan ukuran populasi virtual NP .
- Elit digantikan oleh kandidat baru jika kondisi elitisme terpenuhi.

Dengan skema ini, hanya dua solusi (elit dan kandidat) serta satu PV yang perlu disimpan di memori pada satu waktu dan bukan NP individu penuh sebagaimana EDA kanonik atau algoritma *population-based* lain, sehingga jejak memori akan turun dari $O(NP \times D)$ menjadi $O(D)$, dengan D adalah dimensi atau jumlah variabel keputusan. Siklus kerja algoritma *compact* beserta mekanisme pembaruan PV dan keunggulan memorinya diilustrasikan pada Gambar II.2.



Gambar II.2 Siklus Kerja Algoritma *Compact* dan Mekanisme Pembaruan PV

Gambar II.2 memperlihatkan bahwa algoritma *compact* bekerja dalam satu putaran iteratif yang diulang hingga kriteria terpenuhi. Karena hanya satu PV beserta dua solusi yang perlu disimpan, jejak memorinya tetap $O(D)$ tanpa pernah menyimpan populasi virtual berukuran NP secara eksplisit. Prinsip pembaruan hemat memori inilah yang akan diadopsi pada solusi yang diusulkan sebagai komponen *compact*-nya.

Untuk ruang pencarian diskret/biner, *compact-GA* merepresentasikan PV sebagai vektor probabilitas per-bit, yang diperbarui sebesar $\pm \frac{1}{NP}$ berdasarkan nilai bit pemenang dan pecundang pada setiap posisi (Khalfi dkk., 2023). Representasi biner/kategorikal semacam ini relevan bagi ruang kebijakan ABAC yang bersifat diskret. Sejumlah varian dikembangkan dari *compact-GA*, antara lain varian elitis yang terbukti mengungguli *compact-GA* nonelitis orisinal, serta *uniform compact Genetic Algorithm* (UcGA) yang memperkenalkan ukuran populasi virtual berbeda per variabel.

Untuk ruang pencarian kontinu, PV pada *real-valued compact Genetic Algorithm* (rcGA) menyimpan parameter distribusi Gaussian per dimensi, alih-alih probabilitas per-bit, namun tetap mengikuti logika pembaruan pemenang/pecundang yang sama. Prinsip ini kemudian diperluas ke berbagai logika pencarian metaheuristik lain, salah satunya *compact-PSO* yang mereformulasikan mekanisme pembaruan kecepatan-posisi PSO ke dalam kerangka probabilistik *compact* (Khalfi dkk., 2023). Keberadaan varian *compact* untuk beragam metaheuristik menunjukkan bahwa prinsip penggantian populasi eksplisit dengan model probabilistik ringan bersifat umum dan dapat diterapkan lintas logika pencarian dan bukan eksklusif milik satu keluarga

algoritma tertentu.

Meskipun demikian, Khalfi dkk. (2023) mencatat bahwa algoritma *compact* pada umumnya melakukan pencarian tak berkorelasi yang tidak menyimpan nilai korelasi silang antar-variabel keputusan. Dengan kata lain, PV pada *compact-GA*, *rcGA*, maupun *compact-PSO* standar mengasumsikan independensi antar-variabel. Asumsi independensi ini efisien dari segi memori, tetapi mengabaikan potensi dependensi antar-atribut yang lazim ditemukan pada kebijakan ABAC dunia nyata. Celah antara efisiensi memori dan kemampuan memodelkan dependensi inilah yang menjadi motivasi utama pada penelitian ini.

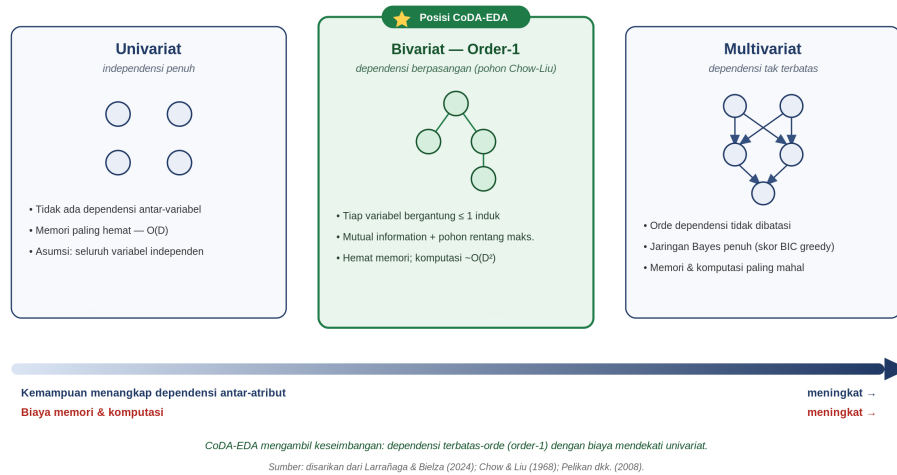
II.7 Pemodelan Dependensi

Kesenjangan yang teridentifikasi menuntut adanya mekanisme pemodelan dependensi yang tetap hemat memori. *Bayesian Optimization Algorithm* (BOA) adalah EDA yang mampu mempelajari jaringan Bayes tanpa batasan derajat dependensi (Pelikan dkk., 2008). Struktur jaringan berupa graf asiklik berarah yang dipelajari secara *greedy*, dimulai dari graf kosong, dengan menambahkan satu *edge* pada setiap langkah yang paling meningkatkan skor *Bayesian Information Criterion* (BIC) hingga tidak ada lagi penambahan yang meningkatkan skor. Parameter jaringan diestimasi melalui estimasi kemungkinan maksimum dari populasi solusi terpilih. Soloviev dkk. (2024) mencatat bahwa BOA tetap menjadi salah satu modul EDA multivariat yang aktif diimplementasikan hingga saat ini, dengan varian jaringan Bayes diskret, Gaussian, semiparametrik, maupun berbasis [kernel density].

Solusi baru pada BOA disampling dari jaringan Bayes yang telah dipelajari melalui *logic sampling* dua tahap (Pelikan dkk., 2008). Pertama, menentukan urutan ancestral sedemikian rupa sehingga setiap variabel didahului oleh seluruh variabel induknya. Kedua, membangkitkan nilai tiap variabel secara berurutan mengikuti urutan tersebut, berdasarkan probabilitas bersyarat terhadap nilai induknya yang telah dibangkitkan lebih dulu. Dengan mekanisme ini, BOA mampu merepresentasikan dependensi multivariat kompleks antar-variabel keputusan, namun dengan konsekuensi biaya komputasi dan memori yang meningkat seiring kompleksitas struktur jaringan yang dipelajari.

Di antara kutub independensi penuh (univariat) dan dependensi tak terbatas (BOA), terdapat pendekatan dependensi berpasangan (bivariat) yang berlandaskan metode Chow dan Liu (1968). Metode ini adalah metode *dependency tree* yang mengaproksimasi distribusi probabilitas gabungan n -dimensi secara optimal melalui produk

distribusi berpasangan yang membentuk struktur pohon. Chow dan Liu (1968) membuktikan bahwa dengan menghitung *mutual information* antar setiap pasangan variabel, lalu mencari pohon rentang maksimum atas nilai *mutual information* tersebut, diperoleh aproksimasi pohon dengan divergensi Kullback-Leibler minimum terhadap distribusi sebenarnya.



Gambar II.3 Spektrum Pemodelan Dependensi pada EDA dan Posisi CoDA-EDA

Gambar II.3 memperlihatkan bahwa kemampuan menangkap dependensi antar-variabel dan biaya memori/komputasi meningkat secara bersamaan dari kiri ke kanan. CoDA-EDA sengaja ditempatkan pada titik tengah spektrum yang menangkap korelasi antar-atribut terpenting namun dengan biaya yang mendekati pendekatan univariat.

Sebagaimana EDA kanonik dan algoritma *population-based* lain, BOA standar tetap menyimpan populasi penuh pada setiap generasi untuk mempelajari struktur maupun parameter jaringan Bayes-nya. Pelikan dkk. (2008) mengusulkan *incremental BOA* (iBOA) untuk menjawab keterbatasan ini. Alih-alih menyimpan populasi eksplisit, iBOA memperbarui struktur dan parameter jaringan Bayes secara inkremental menggunakan skema turnamen pemenang/pecundang yang sama dengan *compact-GA*. iBOA menunjukkan bahwa pemodelan dependensi multivariat dan pembaruan model tanpa populasi eksplisit bukanlah dua tujuan yang saling eksklusif.

Namun demikian, Pelikan dkk. (2008) sendiri mengakui tantangan skalabilitas iBOA yaitu mempertahankan seluruh kemungkinan tabel probabilitas marginal untuk struktur dependensi yang belum diketahui sebelumnya berpotensi membutuhkan hingga $O(n^k)$ tabel untuk k orde dependensi tertinggi yang diizinkan. Sehingga tanpa

pembatasan tambahan, iBOA berisiko kehilangan sebagian keunggulan memori yang menjadi ciri khas pendekatan *compact*. Celah inilah yang menjadi ruang rancangan CoDA-EDA dengan menggabungkan mekanisme pembaruan inkremental tanpa populasi eksplisit ala *compact-GA/iBOA* dengan struktur dependensi yang dibatasi secara eksplisit dan murah secara komputasi.

II.8 Penelitian Terdahulu

Pendekatan berbasis pencarian evolusioner untuk *ABAC policy mining* diperkenalkan oleh Medvet dkk. (2015), jauh sebelum ACO (Shang dkk., 2024) dan UBK (Li dkk., 2026) yang menjadi *baseline* penelitian ini. Medvet dkk. (2015) mengusulkan pendekatan evolusioner multi-objektif dengan representasi fenotipik dan operator genetik khusus untuk aturan ABAC, strategi *separate-and-conquer*, inisialisasi populasi kustom, skema promosi diversitas, dan kriteria penghentian dini yang dioptimasi terhadap tiga objektif sekaligus, yakni konsistensi terhadap data permintaan akses, kompleksitas kebijakan yang rendah, dan penghindaran atribut yang secara unik merepresentasikan identitas subjek/objek. Medvet dkk. (2015) sendiri mencatat bahwa penerapan teknik evolusioner pada kebijakan keamanan telah dieksplorasi sebelumnya secara terbatas. Misalnya, Lim dkk. (2008) menggunakan *genetic programming* untuk kebijakan keamanan multi-level berskala sangat kecil sebagai bukti konsep, dan Lim dkk. (2009) menggunakan algoritma evolusioner multi-objektif SPEA2 untuk pembelajaran kebijakan keamanan dinamis.

Perkembangan berikutnya bergeser ke metaheuristik *population-based* berbasis kecerdasan kawanan. Shang dkk. (2024) menggunakan *Ant Colony Optimization* (ACO) dan Li dkk. (2026) menggunakan *Upgraded Black-winged Kite* (UBK), keduanya dievaluasi dengan metrik akurasi dan *Weighted Structural Complexity* (WSC). Di sisi lain, penelitian non-evolusioner pada *policy mining* juga terus berkembang. Guo dkk. (2025) mengusulkan kerangka *on-the-fly* yang memformulasikan *policy mining* sebagai persoalan optimasi aproksimasi submodular, dipandu oleh teknik pemangkasan ruang pencarian, dengan fokus pada usabilitas kebijakan bagi manusia maupun LLM. Kerangka tersebut dievaluasi pada kontrol akses secara umum, bukan ABAC secara spesifik, dan mencapai penurunan redundansi 93,2% dibanding metode pembandingan. Pendekatan ini bersifat deterministik/*greedy*, bukan *population-based* maupun EDA, dan sebagaimana Medvet dkk. (2015), Shang dkk. (2024), serta Li dkk. (2026), tidak membahas keterbatasan memori perangkat *edge-IoT* sebagai pertimbangan desain.

Tabel II.2 Ringkasan Posisi Penelitian Terdahulu

No	Peneliti (Tahun)	Metode	Fokus/Kontribusi	Celah terhadap CoDA-EDA
1	Medvet dkk. (2015)	<i>Genetic programming</i> multi-objektif (representasi fenotipik kustom)	Salah satu pendekatan evolusioner paling awal untuk ABAC policy mining; 3 objektif: konsistensi, kompleksitas rendah, hindari atribut identitas unik	Population-based (menyimpan populasi pohon ekspresi penuh); tidak dirancang untuk perangkat memori terbatas
2	Shang dkk. (2024)	<i>Ant Colony Optimization</i> (ACO)	Baseline population-based penelitian ini; dievaluasi dengan akurasi & WSC	Population-based — jejak memori (matriks feromon) tumbuh dengan ukuran populasi/masalah
3	Li dkk. (2026)	<i>Upgraded Black-winged Kite</i> (UBK)	Baseline population-based penelitian ini; kecerdasan kawanan berbasis populasi	Population-based — kebutuhan memori sama dengan algoritma population-based lain
4	Guo dkk. (2025)	Optimasi submodular/aproksimasi (deterministik, non-evolusioner)	Kontrol akses umum (bukan ABAC spesifik); fokus usabilitas kebijakan bagi manusia/LLM	Bukan pendekatan population-based/EDA; tidak membahas keterbatasan memori perangkat edge-IoT

Lanjut ke halaman berikutnya

Tabel II.2 – lanjutan dari halaman sebelumnya

No	Peneliti (Tahun)	Metode	Fokus/Kontribusi	Celah terhadap CoDA-EDA
5	— (celah penelitian)	EDA / <i>compact-EDA</i> / <i>dependency-aware</i> EDA	Belum ditemukan penelitian yang menerapkan kelas algoritma ini pada ABAC policy mining	Celah inilah yang diisi CoDA-EDA, bersama Premis 1-4 dan Hipotesis H1-H4

Berdasarkan ringkasan pada Tabel II.2, tampak bahwa seluruh penelitian terdahulu pada *ABAC policy mining* belum mempertimbangkan keterbatasan memori perangkat *edge-IoT* sebagai bagian dari rancangan algoritmanya. Selain itu, sepanjang penelusuran literatur yang dilakukan, belum ditemukan penelitian yang menerapkan kelas algoritma EDA pada persoalan ini. Kekosongan inilah yang menjadi dasar argumentasi kebaruan penelitian yang akan diuraikan lebih rinci pada Subbab selanjutnya.

II.9 Posisi dan Kebaruan Penelitian

Tabel II.2 memperlihatkan bahwa dua jalur penelitian telah berkembang secara paralel tanpa pernah saling bersinggungan. Jalur pertama adalah penelitian *ABAC policy mining* berbasis pencarian evolusioner atau metaheuristik (Medvet dkk., 2015, Shang dkk., 2024 dan Li dkk., 2026) maupun non-evolusioner (Guo dkk., 2025), yang secara konsisten dievaluasi pada lingkungan komputasi tanpa batasan sumber daya dan tidak menjadikan keterbatasan memori perangkat sebagai pertimbangan rancangan. Jalur kedua adalah penelitian algoritma EDA yang secara eksplisit dirancang untuk efisiensi memori, mulai dari EDA kanonik, algoritma *compact* seperti *compact-GA* dan *compact-PSO* (Khalfi dkk., 2023), hingga algoritma pemodelan dependensi seperti BOA, iBOA, dan pohon Chow-Liu (Pelikan dkk., 2008 dan Chow dan Liu, 1968). Namun, berdasarkan penelusuran literatur yang dilakukan, belum ditemukan satu pun di antaranya yang diterapkan atau dievaluasi pada persoalan *ABAC policy mining*. Penelitian ini, melalui algoritma yang diusulkan bernama *Compact Dependency-Aware EDA* (CoDA-EDA), diposisikan pada titik temu kedua jalur penelitian tersebut.

Nama CoDA-EDA sendiri merefleksikan tiga prinsip rancangan yang secara langsung merespons kebutuhan-kebutuhan dasar yang telah diidentifikasi dari sintesis

literatur. Prinsip pertama, *compact*, mengadopsi mekanisme pembaruan hemat memori ala *compact-GA*, yaitu kompetisi pemenang dan pecundang yang memperbarui sebuah PV tanpa menyimpan populasi eksplisit (Khalifi dkk., 2023). Hal ini merupakan jawaban langsung atas keterbatasan memori, daya komputasi, dan energi pada perangkat *edge-IoT* (Canavese dkk., 2024). Prinsip kedua, *dependency-aware*, mempertahankan pemodelan dependensi antar-atribut yang dibatasi pada orde tertentu, alih-alih mengasumsikan independensi penuh sebagaimana *compact* standar atau mempelajari jaringan Bayes tak terbatas sebagaimana BOA maupun iBOA (Pelikan dkk., 2008). Dengan demikian, pemodelan dependensi tetap meningkatkan kualitas kebijakan namun tidak harus dilakukan tanpa batasan orde agar tetap hemat memori. Prinsip ketiga, EDA, menempatkan kedua mekanisme tersebut dalam kerangka *Estimation of Distribution Algorithm* sehingga CoDA-EDA tetap dapat diperlakukan dan dibandingkan secara setara dengan EDA lain dalam hal skema seleksi, estimasi, dan pencuplikan.

Lingkungan IoT yang dinamis (Ragothaman dkk., 2023) menuntut pembaruan kebijakan secara berskala, sementara pembaruan struktur dependensi secara menyeluruh pada setiap perubahan data, sebagaimana dilakukan iBOA, mahal secara komputasi. Kebutuhan ini dijawab melalui rancangan yang memisahkan frekuensi pembaruan struktur dari pembaruan parameter, sebagaimana telah ditetapkan dalam aktivitas rancang bangun metodologi penelitian ini. Pembaruan parameter dilakukan setiap siklus sebagaimana lazimnya EDA, sedangkan pembaruan struktur dependensi hanya dipicu ketika terdeteksi pergeseran *drift* data yang signifikan. Pemisahan ini berbeda dari iBOA yang memperbarui struktur maupun parameter secara inkremental dalam skema yang sama (Pelikan dkk., 2008), dan menjadi dasar bagi pengujian efisiensi waktu komputasi serta kualitas *warm-start* pada skenario pergeseran data.

Pemicu pembaruan struktur pada CoDA-EDA memerlukan ukuran yang dapat membedakan perubahan distribusi antar-jendela secara stabil. Penelitian ini menggunakan *Jensen–Shannon divergence* (JSD) sebagai ukuran pergeseran distribusi. Secara umum, untuk dua distribusi probabilitas P dan Q dengan bobot yang sama, JSD dapat dinyatakan sebagai

$$\text{JSD}(P \parallel Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), \quad M = \frac{1}{2}(P + Q), \quad (\text{II.1})$$

dengan KL menyatakan *Kullback–Leibler divergence*. Pemilihan JSD didasarkan pada sifat teoretisnya sebagai ukuran perbedaan distribusi. Lin (1991) menunjukkan bahwa kelas divergensi berbasis entropi Shannon yang mencakup JSD tidak memerlukan syarat kontinuitas absolut sebagaimana divergensi Kullback. Sifat ini relevan

untuk distribusi diskret karena suatu nilai atribut dapat memiliki probabilitas nol pada satu distribusi tetapi muncul pada distribusi lain. Pada divergensi Kullback, keadaan tersebut dapat menyebabkan nilai divergensi tidak terdefinisi ketika distribusi perbandingan bernilai nol, sedangkan konstruksi JSD melalui distribusi campuran M tetap menghasilkan nilai yang terdefinisi dan berhingga.

Selain itu, JSD bersifat non-negatif dan, untuk dua distribusi dengan bobot sama serta logaritma berbasis dua, memiliki rentang nilai yang terbatas antara 0 dan 1 (Lin, 1991). Nilai 0 menunjukkan bahwa kedua distribusi sama, sedangkan nilai yang semakin besar menunjukkan ketidakmiripan distribusi yang semakin tinggi. Sifat terbatas tersebut memberikan skala numerik yang lebih mudah digunakan sebagai dasar penetapan ambang perubahan distribusi dibandingkan divergensi yang tidak terbatas. JSD dengan bobot yang sama juga bersifat simetris, sehingga perbandingan antara distribusi jendela sebelumnya dan jendela berjalan tidak bergantung pada arah perbandingan. Lin (1991) juga menunjukkan hubungan formal JSD dengan batas atas dan bawah probabilitas kesalahan klasifikasi Bayes. Hubungan tersebut memberikan interpretasi bahwa JSD tidak hanya mengukur perbedaan numerik antar-distribusi, tetapi juga berkaitan dengan tingkat keterbedaan informasi yang dikandung distribusi tersebut.

Penggunaan JSD sebagai dasar deteksi perubahan distribusi juga telah diterapkan secara langsung pada permasalahan *concept drift*. Yang dkk. (2022) mengusulkan CDJD (*Concept Drift Detection based on Jensen–Shannon Divergence*) untuk klasifikasi trafik jaringan. Pendekatan tersebut mempertahankan dua jendela yang mewakili distribusi lama dan distribusi berjalan, kemudian menghitung JSD antara distribusi fitur pada kedua jendela. Perubahan dinyatakan sebagai *concept drift* ketika nilai JSD melampaui ambang yang ditetapkan. Mekanisme yang sama juga digunakan untuk membandingkan konsep yang baru terdeteksi dengan konsep terdahulu sehingga model yang telah tersedia dapat digunakan kembali ketika distribusinya serupa. Hasil eksperimen Yang dkk. (2022) menunjukkan bahwa pendekatan berbasis JSD tersebut mampu mendeteksi perubahan distribusi dan mempertahankan kinerja klasifikasi pada aliran data yang mengalami *concept drift*.

Prinsip tersebut sejalan dengan kebutuhan CoDA-EDA, meskipun objek yang dipantau berbeda. CoDA-EDA tidak menggunakan JSD untuk menentukan perubahan model klasifikasi, tetapi untuk mengukur pergeseran distribusi pasangan atribut–nilai antar-jendela log akses. Struktur dependensi Chow–Liu hanya perlu dipelajari ulang apabila pergeseran tersebut melampaui ambang τ , sedangkan parameter probabilistik tetap dapat diperbarui pada setiap siklus. Dengan demikian, JSD berperan sebagai

mekanisme pemicu yang memisahkan perubahan distribusi yang cukup besar untuk memerlukan pembaruan struktur dari variasi antar-jendela yang masih dapat ditangani melalui pembaruan parameter. Pemilihan ini sekaligus mendukung tujuan rancangan CoDA-EDA untuk menghindari pembelajaran ulang struktur yang tidak diperlukan pada perangkat *edge* dengan sumber daya terbatas.

Diposisikan terhadap Tabel II.2 secara baris per baris, CoDA-EDA berbeda dari Medvet dkk. (2015), Shang dkk. (2024), dan Li dkk. (2026) karena meninggalkan paradigma berbasis populasi secara keseluruhan demi kerangka EDA yang kompak. Perbedaan ini mendasari perbandingan jejak memori terhadap BOA, iBOA, ACO, dan UBK. CoDA-EDA juga berbeda dari Guo dkk. (2025). Meskipun sama-sama termotivasi oleh efisiensi, Guo dkk. (2025) mengandalkan pencarian deterministik atau *greedy* berbasis optimasi submodular untuk meningkatkan kemudahan penggunaan kebijakan bagi manusia dan LLM, sedangkan CoDA-EDA mengandalkan pencarian probabilistik berbasis model yang secara eksplisit dirancang untuk keterbatasan sumber daya perangkat, bukan untuk kemudahan penggunaan bagi manusia. Kedua pendekatan ini bersifat komplementer, bukan saling menggantikan.

Pertanyaan yang lebih tajam adalah di bagian mana persisnya CoDA-EDA berbeda dari algoritma EDA yang sudah ada. Perbedaan itu tidak terletak pada satu komponen tunggal, melainkan pada kombinasi spesifik empat sumbu rancangan yang tidak ditempati oleh EDA mana pun sebelumnya. Tabel II.3 memetakan CoDA-EDA terhadap empat keluarga EDA terdekat menurut empat sumbu tersebut: (1) model dependensi antar-atribut, (2) penyimpanan populasi dan perilaku memori terhadap ukuran populasi NP, (3) mekanisme pembaruan struktur model, dan (4) domain penerapan asal.

Tabel II.3 Perbedaan CoDA-EDA terhadap Algoritma EDA yang Sudah Ada

Algoritma EDA	Model dependensi antar-atribut	Populasi & memori terhadap NP	Mekanisme pembaruan struktur	Diterapkan pada ABAC?
<i>compact-GA / compact-PSO</i>	Tidak ada — univariat, independensi penuh	Tanpa populasi eksplisit; <i>Probability Vector</i> O(D), memori datar terhadap NP	Tidak ada struktur untuk diperbarui	Belum

Lanjut ke halaman berikutnya

Tabel II.3 – lanjutan dari halaman sebelumnya

Algoritma EDA	Model dependensi antar-atribut	Populasi & memori terhadap NP	Mekanisme pembaruan struktur	Diterapkan pada ABAC?
BOA	Jaringan Bayes tanpa batasan orde (multivariat penuh)	Populasi eksplisit; memori tumbuh terhadap NP	Membangun ulang jaringan Bayes tiap generasi (mahal secara komputasi)	Belum
iBOA	Pohon dependensi (Chow-Liu), inkremental	Statistik/populasi inkremental; tidak menanggung transien populasi penuh	Struktur dan parameter diperbarui pada skema inkremental yang sama, tiap langkah	Belum
CoDA-EDA (diusulkan)	Pohon dependensi Chow-Liu dibatasi order-1 (terbatas, bukan tanpa batas)	Tanpa populasi eksplisit; <i>Probability Vector</i> + pohon O(D), memori datar terhadap NP	Struktur dipisah dari parameter: struktur jarang (dipicu <i>drift</i>), parameter tiap siklus	Ya — penelitian ini (Raspberry Pi 5)

Membaca Tabel II.3 secara kolom menjelaskan letak perbedaannya. Pada sumbu model dependensi, CoDA-EDA berada di antara dua ekstrem. Ia tidak mengasumsikan independensi penuh sebagaimana *compact*, namun juga tidak memodelkan dependensi tanpa batasan orde sebagaimana BOA. Melainkan membatasi diri pada pohon Chow-Liu order 1, mengambil sebagian besar manfaat pemodelan dependensi dengan ongkos yang jauh lebih kecil. Pada sumbu memori, hanya CoDA-EDA dan keluarga *compact* yang datar terhadap NP, tetapi keluarga *compact* membayar kedataran itu dengan membuang pemodelan dependensi sepenuhnya, sedangkan CoDA-EDA mempertahankannya. Pada sumbu pembaruan struktur, inilah pembeda paling tajam terhadap kerabat terdekatnya, iBOA. iBOA memperbarui struktur dan parameter pada satu skema inkremental yang sama pada setiap langkah, sementara CoDA-EDA sengaja memisahkan keduanya. Struktur hanya disegarkan ketika *drift* terdeteksi, parameter diperbarui tiap siklus. Sel yang ditempati baris terakhir Tabel II.3 tidak ditempati oleh satupun baris di atasnya, di sanalah kontribusi metodologis

penelitian ini berada.

Kontribusi tersebut karenanya bukan sekadar penerapan algoritma yang sudah ada pada domain baru, melainkan penggabungan tiga mekanisme yang selama ini terpisah pada literatur EDA menjadi satu rancangan yang koheren: pembaruan tanpa populasi ala *compact* yang selama ini hanya univariat, pemodelan dependensi berorde terbatas ala *dependency-tree* yang selama ini menyertai populasi eksplisit atau skema inkremental penuh, dan pemisahan frekuensi pembaruan struktur-parameter yang tidak ada pada EDA mana pun. Ketiga penggabungan inilah yang diterjemahkan ke dalam tiga pilar kebaruan penelitian ini. Kebaruan pertama terletak pada penerapan kelas algoritma EDA, termasuk varian ringannya dan varian yang peka terhadap dependensi pada permasalahan *ABAC policy mining*. Berdasarkan penelusuran literatur yang dilakukan, kelas algoritma ini belum pernah diterapkan pada domain tersebut. Kebaruan kedua adalah kombinasi antara pemodelan dependensi terbatas orde pertama menggunakan pendekatan pohon Chow-Liu dengan mekanisme pembaruan tanpa populasi eksplisit ala *compact*. Kombinasi ini belum ditemukan pada varian EDA maupun *compact* yang ada saat ini. Kebaruan ketiga adalah pemisahan yang jelas antara frekuensi pembaruan struktur dependensi yang dilakukan secara jarang dan hanya saat terjadi pergeseran data dengan pembaruan parameter yang dilakukan setiap siklus. Pengujian dilakukan langsung pada perangkat *edge-IoT* nyata, yaitu Raspberry Pi 5. Hal ini berbeda dari seluruh studi yang tercantum pada Tabel II.2 yang umumnya dievaluasi pada lingkungan komputasi tanpa pembatasan sumber daya yang dinyatakan secara eksplisit.

II.10 Kerangka Berpikir

Kerangka berpikir penelitian ini merangkum secara visual alur logis yang menghubungkan premis masalah, hasil sintesis tinjauan pustaka, rancangan solusi yang diusulkan, hipotesis yang akan diuji, hingga aktivitas metodologi yang telah ditetapkan. Alur tersebut disajikan pada Gambar II.4.



Gambar II.4 Kerangka Berpikir Penelitian

Diagram pada Gambar II.4 menunjukkan bahwa Premis 1-4, yang berakar dari kesenjangan pada kajian pustaka, secara langsung membentuk tiga prinsip rancangan CoDA-EDA. Ketiga prinsip tersebut dioperasionalkan melalui Hipotesis H1-H4 dan diuji secara empiris mengikuti keenam aktivitas DSRM (Pefferers dkk., 2007), menghasilkan luaran berupa algoritma yang tervalidasi pada perangkat *edge-IoT* nyata. Kerangka berpikir ini menjadi acuan utama dalam merancang detail algoritma, instrumen evaluasi, dan desain eksperimen.

Bab III Metodologi Penelitian

III.1 Analisis Masalah

Sebelum artefak dirancang, persoalan penambangan kebijakan ABAC pada *edge-IoT* diuraikan menjadi sejumlah *design tension* yang harus dijawab. Analisis ini menjadi dasar penurunan kebutuhan dan keputusan rancangan dari algoritma CoDA-EDA yang diusulkan.

1. Keterbatasan sumber daya perangkat *edge* – Perangkat *edge-IoT* beroperasi dengan memori dan daya komputasi yang terbatas. Algoritma metaheuristik berpopulasi eksplisit, seperti BOA, ACO, dan UBK, memelihara sekumpulan kandidat solusi yang jejak memorinya tumbuh seiring ukuran populasi, sehingga tidak layak dijalankan langsung pada perangkat semacam itu. Dari masalah ini muncullah persoalan bagaimana mencari kebijakan berkualitas tanpa memelihara populasi solusi yang membengkak.
2. Ketegangan antara kesadaran dependensi dan hemat memori – Atribut ABAC di dunia nyata itu saling berkorelasi (mis. peran-unit). Keluarga *compact* univariat hemat memori tetapi mengasumsikan independensi penuh antar-atribut, sehingga kualitasnya menurun pada ruang berdimensi tinggi. Sebaliknya, algoritma yang memodelkan dependensi penuh (jaringan bayes pada BOA/iBOA) memerlukan representasi struktur yang mahal (hingga $O(D^2)$). Dari masalah ini muncullah persoalan bagaimana menangkap keterkaitan antar-atribut dengan biaya memori yang tetap hemat.
3. Dinamika kebijakan dan biaya pembaruan – Lingkungan IoT bersifat dinamis sehingga kebijakan perlu diperbarui berkala mengikuti perubahan distribusi data (*drift*). Menambang ulang dari nol, atau membangun ulang struktur dependensi pada setiap siklus sebagaimana pendekatan konvensional akan memakan waktu dan energi yang tidak efisien pada perangkat *edge*, sehingga muncul suatu persoalan bagaimana memperbarui kebijakan secara berkala dengan biaya komputasi minimal tanpa menurunkan kualitas.
4. Kompromi kualitas yang proporsional – Pada konteks *edge*, sedikit kompromi kualitas penambangan statis dapat diterima asalkan memori stabil dan pembaruan dilakukan secara efisien. Namun, kualitas kebijakan yang dihasilkan

harus tetap kompetitif terhadap algoritma sekelas agar artefak tetap bermanfaat. Dari situlah muncul persoalan yakni bagaimana menjaga kualitas tetap setara sambil memenuhi ketiga kendala di atas.

Keempat persoalan di atas dapat ditelusuri secara konkret pada enam algoritma pembandingan yang telah diidentifikasi, yang mana masing-masing memenuhi sebagian kebutuhan tetapi gagal pada sebagian lain, sehingga tidak ada satu pun yang sekaligus ringan, sadar-dependensi, dan inkremental untuk *edge*. Pertama, pada keluarga algoritma *compact*, seperti *compact-GA* dan *compact-PSO*. Keduanya memenuhi kebutuhan memori (jejak $O(D)$ yang datar terhadap populasi), tetapi mengasumsikan independensi penuh antar-atribut. Pada ruang kebijakan berdimensi tinggi yang kaya dengan keterkaitan antar-atribut, asumsi ini dapat menurunkan kualitas karena kombinasi atribut yang selaras tidak terwakili. Masalah utama yang ada pada keluarga algoritma *compact* ini selaras dengan persoalan kedua, yakni buta terhadap dependensi.

Yang kedua, pada kelompok algoritma EDA yang sadar dependensi, seperti BOA dan iBOA. Keduanya sadar dependensi, tetapi bermasalah pada sisi yang berbeda. BOA memelihara populasi eksplisit dan membangun ulang jaringan bayes pada setiap generasi, sehingga jejak memorinya tumbuh dan biaya pembaruannya menjadi tinggi, sehingga menjadikannya tidak layak untuk perangkat *edge*. iBOA memperbaiki sisi memori (jejaknya datar melalui statistik inkremental) dan tetap sadar dependensi, sehingga menjadi pembandingan terdekat dengan algoritma yang diusulkan. Namun ia memperbarui struktur secara terus-menerus pada setiap langkah pencarian, tidak dipicu oleh perubahan data atau *drift*, dan tanpa mekanisme *warm-start*. Akibatnya, pada pembaruan berkala, iBOA melakukan pembelajaran ulang struktur yang tidak perlu ketika data sebenarnya stabil.

Yang ketiga adalah keluarga algoritma metaheuristik berpopulasi eksplisit, seperti ACO dan UBK. Keduanya mencapai kualitas tinggi pada penambangan statis, tetapi berbasis populasi eksplisit/feromon yang jejak memorinya membengkak seiring ukuran populasi, dan dirancang untuk eksekusi *batch/offline* pada sumber daya yang memadai, bukan pembaruan inkremental di *edge*.

Dari ketiga keluarga algoritma tersebut, bisa disimpulkan bahwa tidak ada satu pun algoritma pembandingan yang memenuhi ketiga kebutuhan inti secara serentak. iBOA sebenarnya adalah algoritma yang paling mendekati, tetapi gagal pada pembaruan berkala yang efisien. Kesenjangan inilah yang secara langsung memotivasi rancangan CoDA-EDA. Ringkasannya disajikan pada Tabel III.1

Tabel III.1 Analisis Masalah pada Algoritma Pembandingan terhadap Kebutuhan Inti Edge-IoT

Algoritma (keluarga)	Sadar de- pendensi	Memori datar thd populasi	Pembaruan inkremental efisien	Masalah utama di edge-IoT
compact-GA (compact)	tidak	ya	tidak	Buta dependensi → kualitas turun pada dimensi tinggi
compact-PSO (compact)	tidak	ya	tidak	Independensi penuh antar-atribut, sama seperti compact-GA
BOA (EDA penuh)	ya	tidak	tidak	Populasi eksplisit + bangun ulang struktur tiap generasi → boros
iBOA (EDA inkremental)	ya	ya	tidak	Relearn struktur kontinu, tak sadar-drift, tanpa warm-start
ACO (berpopulasi)	ya*	tidak	tidak	Populasi/feromon tumbuh; batch/offline, bukan edge
UBK (berpopulasi)	ya*	tidak	tidak	Berpopulasi eksplisit → memori tumbuh; tidak inkremental
CoDA-EDA (usulan)	ya (orde-1)	ya	ya	memenuhi ketiga kebutuhan inti

ya* = keterkaitan atribut ditangani secara implisit melalui jejak feromon/konstruksi solusi, bukan model dependensi eksplisit.

III.2 Analisis Kebutuhan

Dari analisis masalah di atas, diturunkan kebutuhan rancangan yang harus dipenuhi artefak yang dibedakan menjadi kebutuhan fungsional dan kebutuhan non-fungsional.

Kebutuhan fungsional (KF):

KF1. Menambang himpunan aturan ABAC berbentuk konjungsi kondisi atribut dari log akses, yang merekonstruksi keputusan izin/tolak.

KF2. Menangkap keterkaitan berpasangan antar-atribut saat membangkitkan kandidat aturan.

KF3. Memperbarui kebijakan secara inkremental ketika distribusi data berubah, termasuk mendeteksi kapan perubahan (*drift*) tersebut terjadi.

Kebutuhan non-fungsional (KNF):

KNF1 (memori) — Jejak memori bersifat $O(D)$ dan tidak tumbuh terhadap ukuran populasi, agar layak pada perangkat *edge*.

KNF2 (efisiensi pembaruan) — Pembaruan berkala hemat waktu; operasi mahal (pembelajaran struktur) hanya dijalankan bila diperlukan.

KNF3 (kualitas) — Kualitas kebijakan non-inferior terhadap algoritma baseline

sekelas.

Keterkaitan antara persoalan, kebutuhan, dan keputusan rancangan yang menjawabnya dirangkum pada Tabel III.2. Dengan demikian, setiap komponen rancangan CoDA-EDA bukan merupakan pilihan sembarangan, melainkan jawaban langsung atas kebutuhan yang diturunkan dari analisis masalah.

Tabel III.2 Pemetaan Analisis Masalah — Kebutuhan — Keputusan Rancangan

Persoalan	Kebutuhan	Keputusan rancangan
1. Sumber daya <i>edge</i> terbatas	KNF1 (memori $O(D)$, tanpa populasi)	<i>Probability Vector</i> (PV) tanpa populasi eksplisit; pembaruan kompetitif $\pm 1/NP$
2. Dependensi vs. hemat memori	KF2 + KNF1	Pohon Chow-Liu orde-1 ($O(D)$), bukan jaringan dependensi penuh ($O(D^2)$)
3. Dinamika & biaya pembaruan	KF3 + KNF2	Pemisahan ritme: struktur dipicu- <i>drift</i> ($JSD > \tau$) + parameter tiap siklus + <i>warm-start</i>
4. Kompromi kualitas proporsional	KNF3	Fungsi <i>fitness</i> selaras-F1 (cakupan + presisi) + penambangan <i>separate-and-conquer</i>

III.3 Variabel Penelitian

Penetapan variabel penelitian secara eksplisit diperlukan agar aktivitas *Demonstration* dan *Evaluation* dapat dioperasikan secara tidak ambigu, serta agar pengujian hipotesis dapat dilakukan secara terkontrol. Penelitian ini melibatkan tiga kelompok variabel, antara lain variabel bebas, variabel terikat, dan variabel kontrol.

Variabel bebas pada penelitian ini terdiri atas tiga hal. Pertama, jenis algoritma yang dibandingkan, mencakup tujuh kondisi antara lain CoDA EDA, *compact-GA* dan *compact-PSO*, BOA dan iBOA, serta *Ant Colony Optimization* dan *Upgraded Black-winged Kite* sebagai *baseline population-based*. Kedua, tingkat korelasi antar-atribut pada dataset pengujian, yang dikendalikan secara eksplisit pada tingkat (rendah, sedang, dan tinggi) melalui perluasan generator ABAC Lab. Ketiga, skenario *drift* data akses, yaitu besaran perubahan (delta) pada log akses antar-siklus pembaruan kebijakan, yang menentukan kapan pembaruan struktur dependensi dipicu pada CoDA-EDA.

Variabel terikat pada penelitian ini mencakup empat kelompok ukuran kinerja, antara lain (1) kualitas kebijakan hasil *policy mining*, terdiri atas akurasi/*f-score*, *Weighted Structural Complexity* (WSC), dan cakupan (*coverage*) aturan terhadap log akses, (2) jejak memori, yaitu konsumsi memori puncak selama algoritma berjalan pada perangkat target, (3) waktu komputasi, mencakup waktu pembelajaran awal dan

waktu satu siklus pembaruan inkremental, dan (4) efisiensi serta kualitas re-optimalisasi pada skenario *drift*, yaitu perbandingan waktu dan kualitas kebijakan antara strategi *warm-start* dan *re-mining* penuh dari awal.

Agar perbandingan antar-algoritma pada aktivitas *Evaluation* tetap adil, sejumlah variabel dikendalikan. Variabel kontrolnya antara lain adalah seluruh algoritma dijalankan pada perangkat keras yang sama (Raspberry Pi 5), Seluruh algoritma diuji pada dataset yang identik dalam satu skenario pengujian, kriteria pemberhentian disetarakan antar-algoritma, dan setiap kombinasi algoritma-skenario dijalankan dengan replikasi berulang menggunakan *random seed* yang berbeda.

Ringkasan definisi operasional atas seluruh variabel di atas, beserta keterkaitannya dengan hipotesis disajikan pada Tabel III.3.

Tabel III.3 Definisi Operasional Variabel Penelitian

No	Variabel	Jenis	Definisi Operasional	Terkait Hipotesis
1	Algoritma yang dibandingkan	Bebas	Tujuh kondisi: CoDA-EDA (diusulkan) dan enam algoritma pembanding (<i>compact-GA</i> , <i>compact-PSO</i> , BOA, iBOA, ACO, UBK). Dari keenam pembanding, tiga berpopulasi eksplisit (BOA, ACO, UBK), sedangkan <i>compact-GA/PSO</i> dan iBOA berjejak memori datar terhadap NP.	H1 (vs <i>compact-GA/PSO</i>); H2 (vs BOA, ACO, UBK); H3 (vs iBOA)
2	Tingkat korelasi atribut	Bebas	Tiga tingkat terkontrol (rendah, sedang, tinggi) yang disuntikkan pada generator ABAC Lab yang diperluas.	H1

Lanjut ke halaman berikutnya

Tabel III.3 – lanjutan dari halaman sebelumnya

No	Variabel	Jenis	Definisi Operasional	Terkait Hipotesis
2b	Dimensi ruang solusi (D)	Bebas	Banyaknya pasangan atribut-nilai yang membentuk vektor keputusan: $D = 436$ pada <i>workforce</i> dan $D = 1.907$ pada <i>e-document</i> . Ditetapkan sebagai variabel bebas berdasarkan studi pendahuluan yang menunjukkan dimensi soal sebagai faktor penentu keunggulan pemodelan dependensi.	H1
3	Skenario <i>drift</i> data akses	Bebas	Besaran perubahan (delta) pada log akses antar-siklus pembaruan yang memicu pembaruan struktur dependensi (tidak ada/rendah/sedang).	H3, H4
4	Kualitas kebijakan	Terikat	Akurasi/ <i>F-score</i> , <i>Weighted Structural Complexity</i> (WSC), dan cakupan (<i>coverage</i>) aturan terhadap log akses; formula rinci pada bagian metode evaluasi.	H1, H3, H4
5	Jejak memori	Terikat	Konsumsi memori puncak (<i>peak resident set size</i>) selama algoritma berjalan pada perangkat target; prosedur pengukuran pada bagian metode evaluasi.	H2

Lanjut ke halaman berikutnya

Tabel III.3 – lanjutan dari halaman sebelumnya

No	Variabel	Jenis	Definisi Operasional	Terkait Hipotesis
6	Waktu komputasi	Terikat	Waktu pembelajaran awal (<i>initial training</i>) dan waktu satu siklus pembaruan inkremental; prosedur pengukuran pada bagian metode evaluasi.	H3
7	Efisiensi re-optimalisasi pada <i>drift</i>	Terikat	Perbandingan waktu dan kualitas kebijakan antara strategi <i>warm-start</i> dan <i>re-mining</i> penuh dari awal pasca- <i>drift</i> .	H4

III.4 Rancangan Algoritma CoDA-EDA

Berdasarkan tiga prinsip rancangan yang telah diuraikan sebelumnya, subbab ini menjabarkan rancangan algoritmik CoDA-EDA secara konkret yang mencakup representasi solusi dan model probabilistik, mekanisme inisialisasi dan *sampling*, mekanisme pembaruan parameter, mekanisme pembaruan struktur dependensi, algoritma keseluruhan, serta analisis kompleksitas memori dan waktu. Rancangan ini merupakan hasil awal aktivitas *Design and Development* yang dapat disempurnakan lebih lanjut apabila hasil *Demonstration* dan *Evaluation* menunjukkan kebutuhan penyesuaian yang sejalan dengan sifat iteratif DSRM.

Agar letak setiap unsur kebaruan terhadap EDA yang sudah ada dapat ditelusuri pada level rancangan, pemetaannya ditegaskan di muka. Unsur *compact* (pembaruan PV tanpa populasi eksplisit) diwujudkan pada mekanisme pembaruan parameter. Inilah yang membedakan CoDA-EDA dari BOA yang menyimpan populasi eksplisit. Unsur *dependency-aware* berorde terbatas diwujudkan pada komponen model probabilistik kedua, yakni pohon Chow-Liu order-1 yang disusun dengan algoritma Prim agar jejaknya tetap $O(D)$. Inilah yang membedakannya dari sumsi independensi penuh *compact-GA/PSO* yang tidak punya komponen ini, sekaligus dari jaringan Bayes tanpa batas orde BOA. Unsur pemisahan frekuensi pembaruan diwujudkan pada perbedaan eksplisit antara pembaruan parameter tiap siklus dan pembaruan struktur berbasis pemicu *drift* (delta terhadap ambang tau (τ)). Inilah pembeda paling tajam terhadap iBOA yang memadukan keduanya pada satu skema inkremental.

Sejalan dengan definisi *policy mining ABAC* sebagai penambangan pola atribut-nilai dari log akses, setiap kandidat aturan ABAC direpresentasikan sebagai vektor biner $x = (x_1, x_2, \dots, x_D)$ berdimensi D , dengan D adalah banyaknya kombinasi pasangan atribut-nilai subjek, objek, dan lingkungan yang mungkin pada domain data. Bit $x_i = 1$ menandakan pasangan atribut-nilai ke- i disertakan sebagai syarat pada aturan, sedangkan $x_i = 0$ menandakan sebaliknya. Sebuah kebijakan ABAC yang lengkap terdiri atas satu atau lebih aturan. Berdasarkan hasil implementasi, representasi-multi-aturan ditetapkan mengikuti strategi *separate-and-conquer*. Metaheuristik mencari satu aturan pada satu waktu, *permit* yang telah tercakup aturan tersebut dikeluarkan dari himpunan sasaran, lalu proses diulang hingga anggaran evaluasi habis atau tidak ada lagi aturan yang memberi tambahan cakupan. Semantik pencocokan mengikuti bahasa kebijakan ABAC Lab (Bui dkk., 2025), dimana nilai-nilai terpilih dalam satu atribut bersifat disjungtif, sedangkan antar-atribut bersifat konjungtif, dan atribut tanpa nilai terpilih berarti tidak dibatasi. Dengan demikian D mencakup pasangan atribut-nilai subjek dan objek beserta himpunan aksi, dan WSC sebuah aturan sama dengan banyaknya pasangan atribut-nilai yang aktif pada vektor tersebut.

Kualitas satu aturan kandidat dinilai melalui fungsi *fitness* yang menyeimbangkan cakupan dan presisi. Misalkan TP menyatakan banyaknya *permit* yang belum tercakup dan berhasil dicocokkan aturan, FP banyaknya permintaan berstatus *deny* yang keliru diizinkan, serta $WSC(x)$ kompleksitas struktural aturan, maka $f(x) = \frac{TP^2}{TP + w_{fp} \cdot FP} - w_{wsc} \cdot WSC(x)$. Suku pertama setara dengan hasil kali TP dengan presisi aturan, sehingga menghargai cakupan dan presisi secara simultan dan selaras dengan metrik F1 yang menjadi ukuran evaluasi akhir. Aturan yang tidak mencakup satu pun *permit* baru ($TP = 0$) didominasi secara mutlak agar tidak pernah terpilih. Perumusan ini merupakan koreksi atas bentuk penalti linier $f(x) = TP - w_{fp}$. Ketika FP diuji lebih dulu, bentuk linier tersebut terbukti memiliki optimum pada aturan yang sangat spesifik dengan cakupan mendekati nol. Pencarian yang lebih intensif justru menurunkan F1 kebijakan yang merupakan gejala ketidakselarasan antara fungsi objektif dan metrik evaluasi. Ketergantungan hasil pendekatan *separate-and-conquer* pada pemilihan ukuran kualitas aturan yang mengarahkan induksi merupakan temuan mapan pada literatur pembelajaran aturan (Gudyś dkk., 2024), sehingga penyelarasan fungsi *fitness* terhadap metrik evaluasi akhir menjadi keputusan rancangan yang berdasar dan bukan sekadar penyetelan teknis.

Dua ketetapan implementasi lain turut memengaruhi kesahihan rancangan. Pertama, inisialisasi *Probability Vector* harus sadar-dimensi. Aturan ABAC secara struktural bersifat jarang atau umumnya hanya memuat dua hingga lima pasangan atribut-nilai.

Kesederhanaan struktural yang justru menjadi salah satu indikator kualitas kebijakan hasil *mining* pada literatur ABAC (Perez-Haro dan Diaz-Perez, 2024 dan Quan dkk., 2023), sehingga inisialisasi seragam $p_i = 0,5$ sebagaimana lazim pada *compact-GA* kanonik menghasilkan aturan yang terlalu spesifik dan tidak mencocokkan satu entri pun. Karena itu, PV diinisialisasi pada $p_0 = \frac{k}{D}$ dengan $k \approx 3$, sehingga jumlah pasangan atribut-nilai aktif di awal pencarian tetap konstan meskipun D berbeda antar-dataset. Kedua, pencarian dibenihkan dari satu contoh *permit* yang belum tercakup, mengikuti prinsip pembelajaran aturan berbasis contoh positif pada kerangka *sequential covering/separate-and-conquer* (Gudyś dkk., 2024). Karena aturan acak nyaris tidak pernah mencocokkan entri mana pun (terukur hanya 2% dari aturan acak), sehingga tanpa benih tersebut fungsi *fitness* membentuk dataran datar yang justru mengarahkan pencarian menuju aturan kosong.

Model probabilistik CoDA-EDA terdiri atas dua komponen. Komponen pertama adalah *Probability Vector* (PV), vektor probabilitas marginal per-bit yang merepresentasikan populasi virtual berukuran NP , mengikuti definisi algoritma *compact* (Khalfi dkk., 2023): $PV = (p_1, p_2, \dots, p_D)$ dengan p_i menyatakan probabilitas bit ke- i bernilai 1. Komponen kedua adalah struktur dependensi T , sebuah pohon yang menghubungkan pasangan variabel dengan *mutual information* tertinggi dan dipelajari melalui metode Chow-Liu (Chow dan Liu, 1968) dan dibatasi pada orde-1 (setiap variabel bergantung pada paling banyak satu variabel induk), alih-alih jaringan Bayes tak terbatas ala BOA/iBOA (Pelikan dkk., 2008). Kombinasi PV (parameter) dan T (struktur) inilah yang membedakan CoDA-EDAA dari *compact* standar yang murni univariat sekaligus dari BOA/iBOA yang tidak dibatasi ordenya.

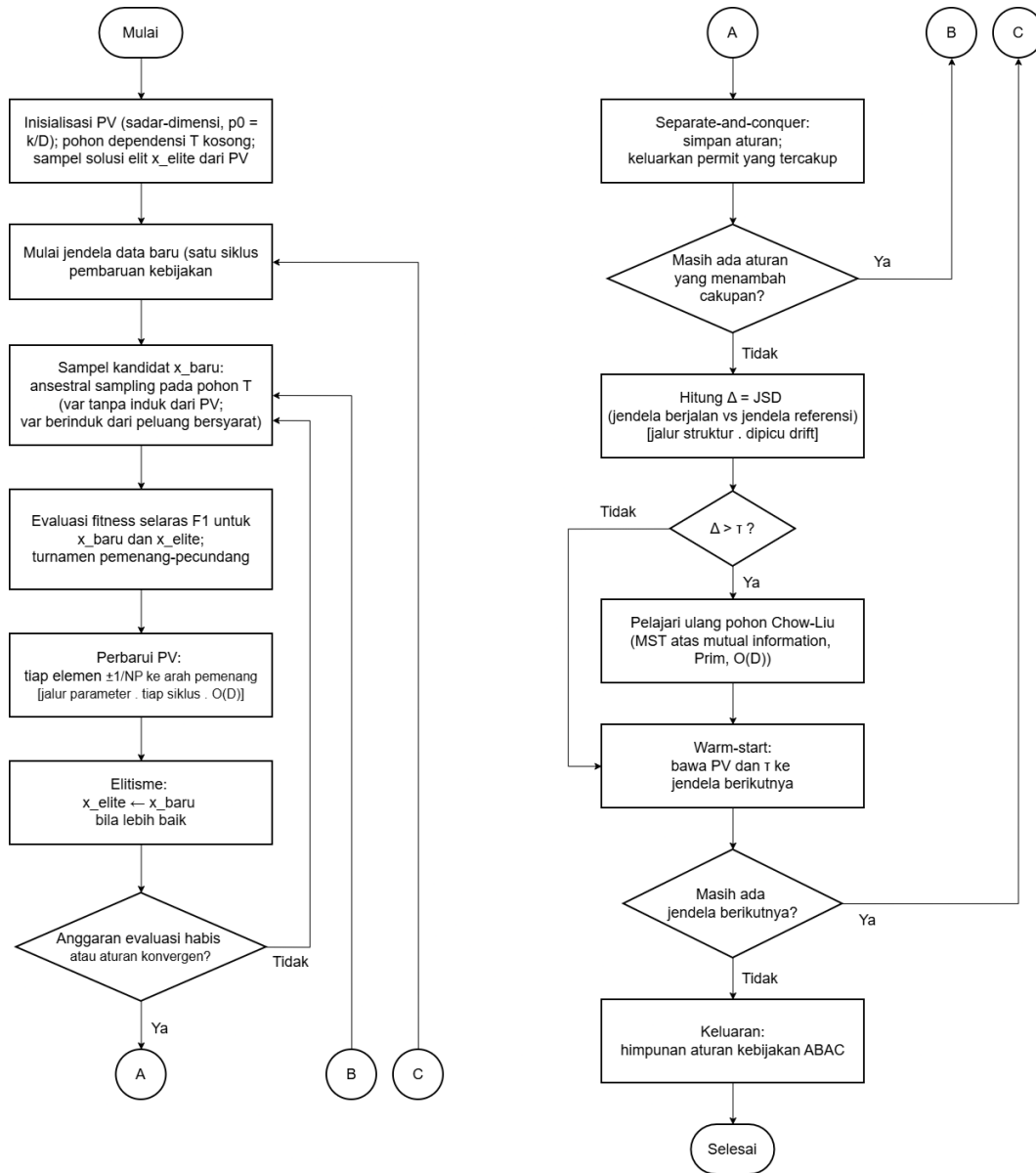
Pada inisialisasi, PV ditetapkan *uniform* ($p_i = 0,5$ untuk seluruh i) dan struktur T diinisialisasi kosong, mencerminkan tidak adanya asumsi dependensi awal. Sebuah solusi elit awal x_{elite} kemudian disampling dari PV. Pada setiap iterasi berikutnya, sebuah solusi kandidat baru x_{baru} disampling mengikuti urutan anestral struktur T (Pelikan dkk., 2008). Variabel tanpa induk disampling langsung dari PV, sedangkan variabel dengan induk disampling dari probabilitas bersyarat terhadap nilai induknya yang telah dibangkitkan lebih dulu.

Pembaruan parameter dilakukan pada setiap siklus tanpa terkecuali, mengikuti skema kompetisi pemenang/pecundang yang sama dengan *compact-GA* (Khalfi dkk., 2023) dan iBOA (Pelikan dkk., 2008). x_{baru} dan x_{elite} dievaluasi *fitness*-nya, ditentukan mana yang menang dan yang kalah, kemudian setiap elemen PV diperbarui sebesar $\pm 1/NP$ bergantung pada nilai pemenang dan pecundang pada variabel bersangkutan. Elit digantikan oleh x_{baru} apabila kondisi elitisme terpenuhi (*fitness* x_{baru} lebih baik).

Skema ini menjaga jejak memori tetap $O(D)$ tanpa menyimpan populasi eksplisit.

Berbeda dari pembaruan parameter, pembaruan struktur dependensi T tidak dilakukan pada setiap siklus, melainkan hanya dipicu ketika terdeteksi pergeseran *drift* signifikan pada distribusi data akses. Deteksi *drift* dilakukan dengan memantau ukuran pergeseran (delta) antara distribusi frekuensi pasangan atribut-nilai pada jendela data akses berjalan terhadap jendela referensi sebelumnya. Ketika Δ melampaui ambang τ yang ditetapkan, struktur T dipelajari ulang melalui perhitungan *mutual information* berpasangan dan pohon rentang maksimum. (Chow dan Liu, 1968), sedangkan PV tetap diperbarui seperti biasa. Pembelajaran struktur ini wajib diimplementasikan secara hemat memori agar klaim *compact* tidak gugur. Penyusunan pohon rentang maksimum dilakukan dengan algoritma Prim (Cormen dkk., 2022) yang hanya menyimpan larik jarak sepanjang D dan menghitung *mutual information* baris demi baris, sehingga kebutuhan memorinya tetap $O(D)$. Implementasi naif yang lebih dulu diuji, yakni dengan membentuk matriks *mutual information* berukuran $D \times D$, membutuhkan sekitar 29 MB pada $D = 1.907$ dan justru meniadakan keunggulan memori yang menjadi inti rancangan. Perhitungan *mutual information* juga harus dilakukan dalam presisi ganda, karena pada presisi tunggal suku $(1 - p_i)$ membulat menjadi nol ketika p_i mendekati satu sehingga pohon gagal terbentuk sama sekali. Pemisahan frekuensi pembaruan ini merupakan mekanisme inti yang membedakan CoDA-EDA dari iBOA konvensional dan menjadi dasar pengujian hipotesis H3 dan H4.

Arsitektur keseluruhan CoDA-EDA, mencakup model probabilistik dua komponen, jalur pembaruan parameter yang dijalankan setiap siklus, serta jalur pembaruan struktur yang hanya dipicu oleh *drift*, diilustrasikan pada Gambar III.1, sedangkan rincian langkah per-baris diberikan pada Algoritma III.1.



Gambar III.1 Diagram Alir Arsitektur CoDA-EDA

Algoritma III.1 merangkum keseluruhan rancangan di atas. Baris 1-2 merupakan inisialisasi. Baris 4-12 merupakan siklus sampling dan pembaruan parameter yang dilakukan pada setiap iterasi. Baris 13-17 merupakan mekanisme deteksi *drift* dan pembaruan struktur bersyarat yang menjadi kebaruan utama CoDA-EDA.

Analisis kompleksitas berikut bersifat *by-construction* serta akan mendapatkan validasi empiris melalui eksperimen, sejalan dengan lingkup penelitian yang tidak mencakup pembuktian konvergensi formal secara matematis mendalam. Kompleksitas memori CoDA-EDA tetap $O(D)$, terdiri atas $O(D)$ untuk PV dan $O(D)$ untuk struktur T berorde-1 dengan setiap variabel menyimpan satu referensi induk dan

Algoritma III.1 CoDA-EDA (Compact Dependency-Aware EDA)

Input: NP (ukuran populasi virtual), τ (ambang delta pemicu pembaruan struktur), kriteria pemberhentian

Output: kebijakan ABAC elite ($\mathbf{x}_{\text{elite}}$)

```
1: Inisialisasi  $\mathbf{PV} \leftarrow (0,5, 0,5, \dots, 0,5)$ ;  $T \leftarrow \emptyset$  (struktur kosong)
2:  $\mathbf{x}_{\text{elite}} \leftarrow \text{SAMPLE}(\mathbf{PV})$ 
3: while kriteria pemberhentian belum terpenuhi do
4:    $\mathbf{x}_{\text{baru}} \leftarrow \text{SAMPLE}(\mathbf{PV}, T)$   $\triangleright$  urutan ancestral mengikuti  $T$ 
5:   evaluasi  $f(\mathbf{x}_{\text{baru}})$  dan  $f(\mathbf{x}_{\text{elite}})$ 
6:   (winner, loser)  $\leftarrow \text{TURNAMEN}(\mathbf{x}_{\text{baru}}, \mathbf{x}_{\text{elite}})$ 
7:   for  $i \leftarrow 1$  to  $D$  do
8:      $\mathbf{PV}[i] \leftarrow \mathbf{PV}[i] \pm \frac{1}{NP}$  berdasarkan winner[i], loser[i]
9:   end for
10:  if  $f(\mathbf{x}_{\text{baru}})$  lebih baik dari  $f(\mathbf{x}_{\text{elite}})$  then
11:     $\mathbf{x}_{\text{elite}} \leftarrow \mathbf{x}_{\text{baru}}$ 
12:  end if
13:   $\Delta \leftarrow \text{UKUR\_DRIFT}(\text{jendela akses berjalan}, \text{jendela referensi})$ 
14:  if  $\Delta > \tau$  then
15:     $T \leftarrow \text{CHOW-LIU}(\text{mutual information berpasangan}, \text{pohon rentang maksimum})$ 
16:    perbarui jendela referensi
17:  end if
18: end while
19: return  $\mathbf{x}_{\text{elite}}$ 
```

tabel probabilitas bersyarat berukuran tetap. Angka ini setara dengan varian *compact* standar, tetapi jauh lebih rendah daripada $O(n^k)$ yang berpotensi dikonsumsi oleh iBOA tanpa pembatasan orde (Pelikan dkk., 2008). Untuk waktu komputasi, pembaruan parameter memerlukan $O(D)$ per siklus, sedangkan pembaruan struktur yang diaktifkan hanya saat terjadi *drift* membutuhkan $O(D^2)$ untuk perhitungan *mutual information* antar seluruh pasangan variabel, ditambah $O(D^2 \log D)$ untuk konstruksi pohon rentang maksimum (Chow dan Liu, 1968). Biaya ini secara signifikan lebih ringan dibandingkan biaya struktur pada BOA yang tidak dibatasi ordenya. Karena pembaruan struktur tidak dilakukan pada setiap siklus, biaya rata-rata per siklus menjadi lebih rendah daripada iBOA konvensional yang justru berpotensi memperbarui struktur pada tiap iterasi.

Untuk menurunkan kompleksitas waktu CoDA-EDA, tinjau notasi berikut terlebih dahulu. D menyatakan dimensi ruang solusi atau banyaknya pasangan atribut-nilai, n jumlah entri log per jendela, E anggaran evaluasi *fitness* per jendela, W jumlah jendela, δ fraksi jendela yang memicu pembaruan struktur yang bernilai antara 0 dan 1 dan ditentukan oleh *drift*, dan k rata-rata jumlah kondisi aktif per aturan yang bernilai kecil ($k \approx 3$) karena aturan ABAC bersifat jarang. Inti biaya algoritma terletak pada satu kali evaluasi kandidat, yang berlangsung dalam tiga langkah berurutan. Kandidat baru mula-mula disampel melalui *ancestral sampling* pada pohon T sehingga menelan waktu $O(D)$. Kandidat itu kemudian dievaluasi *fitness*-nya dengan memindai seluruh jendela untuk menghitung TP dan FP. Langkah ini berbiaya $O(n.D)$ pada kasus terburuk, meski dalam praktik menyusut menjadi $O(n.k)$ yang mendekati $O(n)$ karena aturan hanya memuat sedikit kondisi aktif. Terakhir, PV diperbarui sebesar $\pm 1/NP$ dengan biaya $O(D)$. Karena pemindaian jendela mendominasi dua langkah lainnya, biaya satu evaluasi secara keseluruhan adalah $O(n.D)$.

Biaya per-evaluasi ini terakumulasi secara berbeda pada dua jalur pembaruan CoDA-EDA. Pada jalur parameter yang dijalankan setiap siklus, seluruh anggaran E evaluasi per jendela menghasilkan biaya $O(E.n.D)$ per jendela. Pada jalur struktur, setiap jendela terlebih dahulu menjalani deteksi *drift*. Distribusi frekuensi pasangan atribut-nilai dihitung lalu dibandingkan terhadap jendela referensi melalui $\Delta = \text{JSD}$ dengan biaya $O(n.D)$. Baru apabila Δ melampaui ambang τ , pohon dependensi dipelajari ulang dengan menghitung *mutual information* berpasangan atas $O(D^2)$ pasangan yang diakumulasi dari n baris, lalu membentuk pohon rentang maksimum melalui algoritma Prim yang juga berorde $O(D^2)$, sehingga pembelajaran ulang berbiaya $O(n.D^2)$, namun tetap hemat memori $O(D)$ karena perhitungan dilakukan baris demi baris. Dengan menjumlahkan kedua jalur lintas W jendela, kompleksitas waktu

total menjadi $T = O(W \cdot E \cdot n \cdot D) + O(W \cdot n \cdot D) + O(\delta \cdot W \cdot n \cdot D^2)$ yang dapat disederhanakan menjadi $O(W \cdot E \cdot n \cdot D + \delta \cdot W \cdot n \cdot D^2)$. Suku pertama, yang mewakili penambahan per siklus, berskala linear terhadap D , sedangkan suku kedua yang kuadratik terhadap D hanya muncul sebanding dengan fraksi *drift* δ . Konsekuensinya, ketika data relatif stabil dan δ mengecil, operasi mahal $O(n \cdot D^2)$ jarang terpicu sehingga waktu total mendekati algoritma yang tidak memodelkan struktur sama sekali. Justru di sinilah dasar analitis Hipotesis H3 terletak. Pemisahan frekuensi pembaruan memindahkan operasi mahal $O(n \cdot D^2)$ keluar dari jalur panas per siklus dan menjadikannya bersyarat pada kemunculan *drift*.

Tabel III.4 Perbandingan kompleksitas waktu dan memori per siklus/generasi

Algoritma	Sampling 1 kandidat	Evaluasi <i>fitness</i>	Pembaruan model per siklus	Pembaruan struktur	Memori
compact-GA / compact- PSO	$O(D)$	$O(n \cdot D)$	$O(D)$ (PV)	— (tanpa struktur)	$O(D)$
CoDA-EDA	$O(D)$ (ancestral pd pohon)	$O(n \cdot D)$	$O(D)$ (PV)	$O(n \cdot D^2)$ — hanya saat <i>drift</i>	$O(D)$
iBOA	$O(D)$	$O(n \cdot D)$	$O(D)$	$O(D^2)$ tiap langkah (kontinu)	$O(D)$
BOA	$O(NP \cdot D)$	$O(NP \cdot n \cdot D)$	—	$O(NP \cdot D^2)$ bangun ulang tiap generasi	$O(NP \cdot D)$
ACO / UBK	$O(NP \cdot D)$ (konstruk- si)	$O(NP \cdot n \cdot D)$	$O(NP \cdot D)$ (feromon)	— (tak inkremental)	$O(NP \cdot D)$

Terbaca bahwa jalur per siklus CoDA-EDA satu orde dengan keluarga *compact* ($O(n \cdot D)$), sedangkan operasi struktur $O(n \cdot D^2)$ -nya hanya dibayar saat *drift*. Bandingkan dengan iBOA yang membayar biaya struktur $O(D^2)$ pada setiap langkah (kontinu) dan dengan BOA/ACO/UBK yang seluruh biayanya diperbesar faktor populasi NP. Perbedaan inilah yang membuat CoDA-EDA lebih efisien pada pembaruan berkala sekaligus tetap hemat memori.

III.5 Konstruksi Dataset Penelitian

Sebagaimana yang sudah disinggung sebelumnya, dataset ABAC dengan tingkat korelasi antar-atribut yang bervariasi dan berdomain IoT belum tersedia sebagai sumber daya siap pakai. Penelusuran terhadap ABAC Lab (Bui dkk., 2025) menunjukkan bahwa lima dataset *ground-truth* yang tersedia (*healthcare*, *project-management*, *university*, *workforce*, *e-document*) seluruhnya berdomain organisasi/*enterprise*, dan generator log sintetisnya belum mendukung kontrol eksplisit terhadap tingkat korelasi atribut. Subbab ini menguraikan rancangan konstruksi dataset yang dibutuhkan untuk menguji hipotesis sebagai bagian dari aktivitas *Design and Development*.

Konstruksi dataset dimulai dari dataset *ground-truth* ABAC Lab beserta generator log sintetis *open-source*-nya (Bui dkk., 2025) yang saat ini mendukung parameter ukuran log, rasio *permit/deny*, dan injeksi *noise*. Generator tersebut diperluas dengan menambahkan mekanisme injeksi korelasi atribut terkontrol. Jadi, alih-alih *men-sampling* nilai setiap atribut subjek dan objek secara independen, pasangan atribut yang relevan pada tiap domain (mis. departemen dan peran pengguna pada dataset *workforce*) disampling dari distribusi bersama yang dikalibrasi untuk mencapai target *Normalized Mutual Information* (NMI) tertentu, sehingga tingkat korelasi yang di-*inject* dapat dibandingkan langsung dengan struktur yang berhasil dipelajari algoritma. Tiga tingkat korelasi ditetapkan sebagai variabel bebas (rendah, sedang, dan tinggi) dengan target NMI berturut-turut 0,05, 0,35, dan 0,65. Batas bawah sengaja ditetapkan 0,05 dan bukan 0,00 karena entri berstatus *permit* secara inheren berkorelasi sehingga memaksa proporsi tertentu pada log akan selalu menyuntikkan korelasi dasar yang tidak dapat dihilangkan.

Sebagai ilustrasi mengenai mekanisme penyuntikan korelasi, tinjau dua atribut pada domain *workforce*, yaitu atribut subjek *position* dan atribut sumber daya *associatedContract*. Misalkan untuk suatu pasangan nilai tertentu berlaku probabilitas marginal $P(\text{position} = p) = 0,30$ dan $P(\text{associatedContract} = c) = 0,20$. Apabila kedua atribut tersebut independen, probabilitas keduanya muncul secara bersamaan hanya sebesar $0,30 \times 0,20 = 0,06$. Dalam kondisi ini, mengetahui nilai *position* tidak memberikan informasi tambahan mengenai *associatedContract*, karena probabilitas bersyarat $P(\text{associatedContract} = c \mid \text{position} = p)$ tetap sebesar 0,20. Pada proses penyuntikan korelasi, sebagian massa probabilitas dipindahkan menuju kombinasi nilai yang dibuat selaras tanpa mengubah distribusi marginal masing-masing atribut. Sebagai contoh ilustratif, apabila probabilitas gabungan pasangan tersebut dinaikkan dari 0,060 menjadi 0,151, maka

$P(\text{associatedContract} = c \mid \text{position} = p)$ meningkat dari 0,20 menjadi sekitar 0,503. Dengan demikian, frekuensi kemunculan masing-masing nilai atribut secara individual dapat tetap sama, tetapi pola kemunculan bersama keduanya menjadi tidak acak. Perubahan pada distribusi bersama inilah yang dimaksud sebagai peningkatan korelasi antar-atribut dalam konstruksi dataset penelitian ini.

Tabel III.5 Ilustrasi distribusi bersama dua atribut sebelum injeksi korelasi

	$\text{associatedContract} = c$	$\text{associatedContract} \neq c$	Total
$\text{position} = p$	0,060	0,240	0,300
$\text{position} \neq p$	0,140	0,560	0,700
Total	0,200	0,800	1,000

Tabel III.6 Ilustrasi distribusi bersama dua atribut setelah injeksi korelasi

	$\text{associatedContract} = c$	$\text{associatedContract} \neq c$	Total
$\text{position} = p$	0,151	0,149	0,300
$\text{position} \neq p$	0,049	0,651	0,700
Total	0,200	0,800	1,000

Tabel III.5 dan Tabel III.6 memperlihatkan bahwa penyuntikan korelasi tidak mengubah probabilitas marginal kedua atribut. Proporsi $\text{position} = p$ tetap 0,30 dan proporsi $\text{associatedContract} = c$ tetap 0,20. Perubahan hanya terjadi pada distribusi bersama keduanya. Massa probabilitas pada kombinasi $\text{position} = p$ dan $\text{associatedContract} = c$ meningkat dari 0,060 menjadi 0,151, sedangkan massa pada sel silang berkurang secara bersesuaian. Akibatnya, probabilitas bersyarat berubah dari

$$P(\text{associatedContract} = c \mid \text{position} = p) = \frac{0,060}{0,300} = 0,20 \quad (\text{III.1})$$

menjadi

$$P(\text{associatedContract} = c \mid \text{position} = p) = \frac{0,151}{0,300} \approx 0,503. \quad (\text{III.2})$$

Dengan kata lain, setelah korelasi disuntikkan, mengetahui nilai position memberikan informasi mengenai kemungkinan nilai $\text{associatedContract}$. Karakteristik inilah yang kemudian diukur melalui *Normalized Mutual Information* (NMI) dan

dapat dimanfaatkan oleh model yang mempertahankan dependensi antar-variabel.

Perbedaan tersebut penting bagi perbandingan algoritma dalam penelitian ini. Model univariat seperti *compact-GA* dan *compact-PSO* hanya mempertahankan probabilitas marginal masing-masing variabel sehingga tidak merepresentasikan secara eksplisit bahwa kemunculan suatu nilai `position` dapat meningkatkan kemungkinan kemunculan nilai `associatedContract` tertentu. Sebaliknya, CoDA-EDA mempelajari keterkaitan antar-variabel melalui *mutual information* dan merepresentasikan dependensi terpilih sebagai sisi pada pohon Chow–Liu. Oleh karena itu, penyuntikan korelasi tidak sekadar mengubah frekuensi nilai atribut secara individual, tetapi membentuk struktur dependensi pada distribusi bersama yang dapat dimanfaatkan oleh model *dependency-aware*. Nilai numerik pada ilustrasi ini hanya digunakan untuk memperjelas mekanisme. Pada dataset eksperimen, tingkat korelasi tidak ditetapkan langsung berdasarkan probabilitas gabungan pada ilustrasi tersebut, melainkan melalui proses kalibrasi *closed-loop* hingga NMI empiris mendekati target 0,05, 0,35, dan 0,65.

Kalibrasi korelasi dilakukan secara *closed-loop*. Parameter penggandeng distribusi bersama dicari melalui pencarian biner hingga NMI yang terukur pada log final mendekati target, bukan hingga NMI distribusi teoretis mencapai target. Prosedur ini diperlukan karena estimator NMI dan divergensi Jensen-Shannon sama-sama memiliki bias sampel-hingga yang membesar seiring rasio antara banyaknya sel kontingensi dan banyaknya sampel. Pada pelaksanaan awal, kalibrasi yang memakai log rintisan berukuran kecil (800 entri) sementara log final berukuran 10.000 entri menghasilkan galat NMI sampai -0,11 pada dataset *e-document* yang memiliki 379 sel kontingensi. Galat tersebut menyusut menjadi -0,005 ketika ukuran log rintisan disamakan dengan ukuran log final. Karena itu ukuran log kalibrasi ditetapkan sama dengan ukuran log final.

Bias serupa juga memengaruhi pengukuran *drift* dan menuntut penyesuaian cara pelaporan. Lantai derau divergensi Jensen-Shannon, yakni nilai yang terukur antar-jendela ketika distribusi sesungguhnya tidak berubah, bergantung pada rasio sel/sampel dan berbeda tajam antar-dataset. Pada jendela rintisan berukuran kecil yang dipakai diagnostik awal, lantai derau terukur 0,017 pada *workforce* (45 sel) tetapi mencapai 0,132-0,145 pada *e-document* (379 sel). Inilah yang memaksa pembesaran ukuran jendela. Nilai ini bergantung ukuran jendela. Pada ukuran jendela final yang lebih besar lantai derau *e-document* menyusut ke kisaran 0,014-0,0333, namun tetap taktrivial (di atas $\tau = 0,01$) sehingga pelaporan *drift* bersih tetap diperlukan. Sifat sebagai properti struktural data ditegaskan oleh replikasinya yang

konsisten pada dua lingkungan komputasi yang berbeda. Konsekuensinya, target *drift* 0,10 tidak mungkin tercapai dalam nilai mentah pada *e-document*. Tingkat *drift* karena itu dilaporkan sebagai nilai bersih, yaitu divergensi terukur dikurangi lantai derau yang diperoleh dari kondisi stasioner. Dengan koreksi ini, *drift* terukur mendekati target pada kedua dataset (galat -0,006 hingga -0,022). Implikasi metodologisnya, kondisi stasioner naik status. Ia bukan sekadar kondisi pembandingan dalam desain eksperimen, melainkan instrumen kalibrasi yang wajib dijalankan agar angka *drift* dapat ditafsirkan.

Untuk menguji Hipotesis H3 dan H4 mengenai efisiensi dan kualitas pembaruan pada skenario *drift*, log akses yang dihasilkan generator dibagi menjadi rangkaian jendela berurutan yang mensimulasikan siklus pembaruan kebijakan. Antar-jendela, distribusi bersama atribut disesuaikan dengan besaran perubahan (Δ) tertentu, yang diukur melalui divergensi Jensen-Shannon antara distribusi jendela berjalan dan jendela referensi sebelumnya, konsisten dengan definisi delta pada mekanisme deteksi *drift* CoDA-EDA.

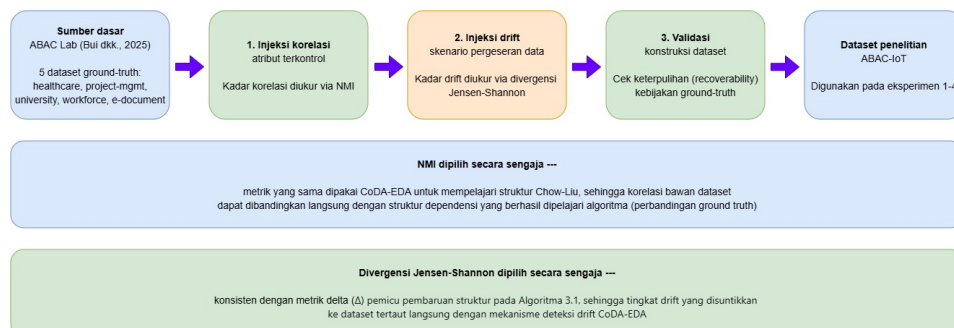
Sebelum digunakan pada aktivitas *Demonstration*, setiap varian dataset yang dikonstruksi akan diverifikasi kesesuaiannya dengan target rancangan: tingkat korelasi aktual (diukur via NMI pada data yang dihasilkan) dibandingkan terhadap target yang ditetapkan, dan proporsi keputusan *permit/deny* diperiksa agar tetap berada pada rentang wajar (tidak timpang secara ekstrem) sebagaimana disyaratkan oleh parameter rasio *permit/deny* bawaan ABAC Lab. Keseluruhan alur konstruksi dataset, dari sumber dasar hingga dataset penelitian akhir, dirangkum pada Gambar III.2.

Pelaksanaan awal konstruksi dataset pada perangkat target menghasilkan tiga temuan yang mengubah rancangan pada subbab ini, dan karenanya dicatat secara eksplisit demi keterulangan. Temuan pertama menyangkut kelangkaan *permit*. Enumerasi penuh ruang ($subjek \times objek \times aksi$) menunjukkan bahwa proporsi permintaan berstatus *permit* hanya berkisar 2-5% pada seluruh dataset ABAC Lab (*workforce* 1,99%; *university* 2,50%; *project-management* 3,32%; *healthcare* 4,27%; *e-document* 5,34%). Konsekuensinya, log tidak dapat dibangkitkan melalui sampling seragam atas ruang permintaan, melainkan harus melalui stratifikasi *permit/deny* dengan himpunan *permit* dipra-komputasi terlebih dahulu.

Temuan kedua menyangkut kelayakan dataset. Seluruh dataset ABAC Lab yang digunakan bersifat sintesis yang dibangkitkan oleh generator log ABAC Lab (Bui dkk., 2025) yang diperluas sehingga karakteristik yang dilaporkan berikut merupakan properti konstruksi, bukan properti data operasional. Setelah dibatasi pada entitas yang memiliki pasangan atribut terkorelasi, jumlah *triple permit* unik yang

tersedia berbeda tajam antar-dataset (*healthcare* hanya 14, *project-management* 17, dan *university* 90, dibandingkan *workforce* 14.513 dan *e-document* 32.961). Pada ketiga dataset kecil, pembangkitkan log berukuran memadai menghasilkan tingkat duplikasi entri 48-93%, sehingga tugas *mining* menjadi *degenerate*, yakni algoritma dihadapkan pada pengulangan pola yang sama alih-alih keragaman pola yang wajar. Karena itu penelitian ini menetapkan *workforce* dan *e-document* sebagai dataset utama untuk pengujian Hipotesis H1, sebab *permit* uniknya memberi keragaman pola yang memadai. Ketiga dataset kecil tetap disertakan pada pengujian seluruh hipotesis, namun dengan peran yang disesuaikan menurut karakteristiknya. Untuk H1, hasil pada ketiganya diperlakukan sebagai konteks pelengkap dan bukan tumpuan vonis, karena tingginya duplikasi entri menumpulkan pembeda antar-algoritma. Sedangkan untuk H2 serta H3 dan H4, ketiganya sepenuhnya sah, bahkan lantai derau divergensi informasinya yang rendah menjadikannya lingkungan yang justru layak untuk menunjukkan mekanisme H3. Dimensi ruang solusi, yaitu banyaknya pasangan atribut-nilai yang membentuk vektor keputusan adalah $D = 436$ untuk *workforce* dan $D = 1.907$ untuk *e-document*, serta $D = 46$, 51 , dan 54 untuk *healthcare*, *project-management*, dan *university*.

Temuan ketiga menyangkut hubungan antara korelasi dan duplikasi. Tingkat duplikasi entri meningkat seiring naiknya target korelasi (*workforce* 17% $\rightarrow$ 37% $\rightarrow$ 51%; *e-document* 4% $\rightarrow$ 14% $\rightarrow$ 30% untuk korelasi rendah $\rightarrow$ sedang $\rightarrow$ tinggi). Peningkatan ini bersifat struktural, bukan cacat implementasi. Korelasi tinggi secara matematis memusatkan sampling pada sedikit sel distribusi bersama, sehingga entri yang sama lebih sering terpilih. Duplikasi semacam ini juga tidak menyalahi realisme, karena log akses nyata memang memuat permintaan berulang. Meskipun demikian, tingkat duplikasi tetap dilaporkan pada setiap konfigurasi agar pembaca dapat menilai keragaman efektif data yang dihadapi algoritma.



Gambar III.2 *Pipeline* Konstruksi Dataset Penelitian

Gambar III.2 menegaskan satu keputusan rancang yang penting, yakni metrik yang dipakai untuk menyuntikkan korelasi dan *drift* ke dataset sengaja dipilih identik dengan metrik yang dipakai CoDA-EDA untuk mempelajari struktur dependensi dan mendeteksi *drift*. Dengan demikian, kadar korelasi dan *drift* yang tertanam pada dataset dapat dibandingkan langsung dengan yang berhasil ditangkap algoritma, sehingga evaluasi H1, H3, dan H4 memiliki acuan *ground-truth* yang jelas.

Di luar dataset yang dikonstruksi sendiri, penelitian ini turut memakai dataset *Smart Building IoT* (Díaz-Rodríguez dan Díaz-Pérez, 2025) sebagai validasi eksternal, yakni memeriksa apakah temuan bertahan pada data yang dibangkitkan pihak lain dengan generator, kebijakan, dan distribusi yang berbeda. Dataset ini digunakan apa adanya tanpa injeksi korelasi atau *drift* buatan. Karena berukuran 5,7 juta permintaan akses dengan rasio *permit* 0,75 dan dimensi penuh sekitar 1.164, ia disubsampel secara terstratifikasi (mempertahankan rasio *permit* asli) dan nilai atribut ber-*support* rendah disatukan, menghasilkan dimensi efektif sekitar 428. Perlu ditegaskan bahwa dataset tersebut bersifat sintesis sebagaimana dinyatakan penulisnya. Nilai validasinya terletak pada asal generatornya yang independen, bukan pada keasliannya sebagai data operasional.

Tabel III.7 Skema Dataset Penelitian

No	Dataset Dasar	Dimensi D	Permit Unik	Perluasan yang Diterapkan	Peran dalam Penelitian
1	<i>Workforce</i> (ABAC Lab)	436	14.513	3 tingkat korelasi (NMI rendah/sedang/tinggi) $\times$ 3 skenario drift (stasio-ner/rendah/sedang)	EKSPERIMEN UTAMA — pengujian H1 (korelasi) dan H3-H4 (drift)
2	<i>E-document</i> (ABAC Lab)	1.907	32.961	3 tingkat korelasi $\times$ 3 skenario drift	EKSPERIMEN UTAMA — pengujian H1 dan H3-H4 pada dimensi lebih besar

Lanjut ke halaman berikutnya

Tabel III.7 – lanjutan dari halaman sebelumnya

No	Dataset Dasar	Dimensi D	Permit Unik	Perluasan yang Diterapkan	Peran dalam Penelitian
3	<i>University</i> (ABAC Lab)	54	90	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 27-57% pada log memadai)
4	<i>Project-management</i> (ABAC Lab)	51	17	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 48-69%)
5	<i>Healthcare</i> (ABAC Lab)	46	14	3 tingkat korelasi $\times$ 3 skenario drift	KONTEKS PELENGKAP — H2/H3/H4 sah; vonis H1 terbatas (duplikasi 58-93%)
6	<i>Smart Building IoT</i> (Díaz-Rodríguez dan Díaz-Pérez, 2025)	428*	—	Digunakan apa adanya; subsampel terstratifikasi dari 5,7 juta baris, tanpa injeksi korelasi/drift	VALIDASI EKSTERNAL — generalisasi H1-H4 pada dataset berdomain IoT yang dibangkitkan pihak lain

Dataset-dataset pada Tabel III.7 menjadi lingkungan pengujian bersama bagi CoDA-EDA dan seluruh algoritma pembandingan. Tanda (*) pada dataset *Smart Building IoT* menunjukkan dimensi efektif setelah subsampel dan penyatuan nilai bersupport rendah yang mana berkas asalnya memuat 5,7 juta permintaan akses dengan rasio *permit* 0,75.

III.6 Algoritma Pembanding

Sebagaimana ditetapkan pada variabel bebas jenis algoritma dan aktivitas *Design and Development*, CoDA-EDAA dibandingkan terhadap enam algoritma *baseline* yang telah dibahas landasan teoretisnya pada Bab II, yakni *compact-GA* dan *compact-PSO*, BOA dan iBOA, serta ACO dan UBK. Subbab ini menguraikan bagaimana keenam algoritma tersebut diimplementasikan dan dikonfigurasi agar perbandingan pada aktivitas *Evaluation* bersifat adil dan tidak bias oleh perbedaan kualitas implementasi antar-algoritma.

Compact-GA dan *compact-PSO* diimplementasikan mengikuti *template* algoritma *compact* umum (Khalfi dkk., 2023): satu PV univariat, skema kompetisi pemenang/pecundang, dan ukuran populasi virtual NP sebagai parameter utama. Representasi solusi menggunakan vektor biner yang identik dengan CoDA-EDA, sehingga perbedaan hasil semata-mata mencerminkan ada-tidaknya pemodelan dependensi dan bukan perbedaan *encoding*. Untuk *compact-PSO*, yang pada rancangan aslinya beroperasi pada ruang kontinu, representasi posisi-kecepatan diadaptasi ke ruang kebijakan ABAC yang diskret mengikuti pendekatan probabilistik yang sama dengan *compact-GA* sehingga sejalan dengan praktik pembineran algoritma metaheuristik berbasis kontinu yang lazim didokumentasikan pada literatur (Pan dkk., 2023), agar kedua *baseline compact* tetap dapat dibandingkan langsung pada representasi biner yang sama.

BOA dan iBOA diimplementasikan mengikuti mekanisme pembelajaran struktur berbasis BIC dan estimasi parameter kemungkinan maksimum (Pelikan dkk., 2008) tanpa pembatasan orde dependensi, berbeda dari struktur Chow-Liu orde-1 pada CoDA-EDA. BOA mempertahankan populasi eksplisit dan mempelajari ulang struktur secara penuh pada setiap generasi, sedangkan iBOA memperbarui struktur dan parameter secara inkremental melalui skema turnamen yang sama dengan *compact-GA*. Hal inilah yang menjadikan iBOA sebagai pembanding langsung yang paling relevan untuk menguji Hipotesis H3.

ACO (Shang dkk., 2024) dan UBK (Li dkk., 2026) diimplementasikan sebagai algoritma *population-based* murni tanpa model probabilistik eksplisit ala EDA. ACO menggunakan matriks feromon dengan parameter evaporasi dan bobot heuristik (α, β) , sedangkan UBK menggunakan mekanisme strategi berburu berbasis populasi sebagaimana diuraikan pada publikasi aslinya. Parameter kedua algoritma ini mengikuti rekomendasi/nilai *default* pada publikasi asli sebagai titik awal, dan dikalibrasi lebih lanjut pada dataset yang dikonstruksi apabila diperlukan agar kinerja *baseline*

tetap representatif, bukan sengaja dilemahkan.

Meskipun mekanisme internal keenam algoritma berbeda, kesetaraan perbandingan tetap terjamin melalui empat kontrol yang telah ditetapkan. Kontrol pertama adalah representasi solusi dan fungsi *fitness* yang identik bagi seluruh algoritma. Kontrol kedua adalah penyetaraan anggaran evaluasi *fitness* maksimum sebagai pengganti penyetaraan jumlah iterasi. Hal ini diperlukan karena algoritma berbasis populasi dan *compact* memiliki definisi iterasi yang berbeda. Pendekatan ini mengikuti konvensi perbandingan metaheuristik pada anggaran evaluasi fungsi yang setara, sebagaimana dikemukakan oleh Rajwar dkk. (2023) dan Omran dkk. (2022). Anggaran tersebut ditetapkan sebagai fungsi dari dimensi soal *D* dan bukan sebagai konstanta. Sebab, dimensi *D* berbeda 4,4 kali lipat antara dua dataset utama, yaitu 436 untuk *workforce* dan 1.907 untuk *e-document*. Jika anggaran berupa konstanta, maka dataset berdimensi kecil akan diuntungkan secara tidak adil. Kontrol ketiga adalah perhitungan anggaran untuk keseluruhan proses *separate-and-conquer*, bukan per aturan. Dengan demikian, algoritma yang menghasilkan lebih banyak aturan tidak akan memperoleh total evaluasi yang lebih besar. Kontrol keempat mencakup penggunaan perangkat keras dan dataset yang identik dalam satu skenario pengujian. Selain itu, replikasi berulang dilakukan dengan *random seed* yang berbeda untuk memperhitungkan variabilitas stokastik dari setiap algoritma.

Tabel III.8 Algoritma Pembanding dan Perlakuan Kesetaraan Implementasi

No	Algoritma	Sumber	Kelas	Parameter Kunci	Catatan Kesetaraan
1	<i>compact-GA</i>	(Khalfi dkk., 2023)	<i>Compact</i> , univariat (diskret)	Ukuran populasi virtual (NP)	Representasi biner identik dengan CoDA-EDA; tanpa pemodelan dependensi
2	<i>compact-PSO</i>	(Khalfi dkk., 2023)	<i>Compact</i> , univariat (kontinu, diadaptasi)	NP; parameter kecepatan-posisi	Representasi diadaptasi ke ruang diskret agar sebanding dengan <i>compact-GA</i>

Lanjut ke halaman berikutnya

Tabel III.8 – lanjutan dari halaman sebelumnya

No	Algoritma	Sumber	Kelas	Parameter Kunci	Catatan Kesetaraan
3	BOA	(Pelikan dkk., 2008)	<i>Population-based</i> , dependensi tak terbatas	Ukuran populasi; ambang BIC	Struktur dipelajari ulang penuh tiap generasi; representasi biner sama
4	iBOA	(Pelikan dkk., 2008)	Inkremental, dependensi tak terbatas	Ambang/ kriteria koneksi struktur	Skema turnamen sama dengan CoDA-EDA; pembanding langsung untuk H3
5	ACO	(Shang dkk., 2024)	<i>Population-based</i> , <i>swarm</i>	Jumlah semut; evaporasi feromon; α , β	Parameter mengikuti publikasi asli, dikalibrasi bila perlu
6	UBK	(Li dkk., 2026)	<i>Population-based</i> , <i>swarm</i>	Ukuran populasi; parameter strategi berburu	Parameter mengikuti publikasi asli, dikalibrasi bila perlu

Selain keempat kontrol di atas, kesetaraan juga dijaga para tataran operator bantu yang bersifat lintas algoritma. Operator perbaikan (*repair*) yang menjamin validitas aturan (sekurang-kurangnya satu aksi dan satu kondisi atribut terpilih) serta prosedur pembenihan dari contoh *permit* diterapkan secara identik pada ketujuh kondisi algoritma. Demikian pula seluruh solusi direpresentasikan memakai tipe wadah data yang sama, sebab perbedaan wadah dapat mengubah jejak memori terukur hingga satu orde besaran tanpa mencerminkan perbedaan algoritmik apa pun. Hal tersebut merupakan sebuah risiko yang secara langsung mengancam kesahihan pengujian H2.

Sebelum eksperimen penuh dijalankan, setiap implementasi wajib melewati uji monotonisitas. Kualitas kebijakan yang dihasilkan tidak boleh menurun ketika anggaran evaluasi dinaikkan. Uji ini diadopsi sebagai prosedur validasi baku karena terbukti mampu menyingkap ketidakselarasan antara fungsi objektif dan metrik evaluasi yang menjadi persoalan yang tidak terdeteksi pada anggaran tunggal, tetapi

tampak jelas ketika anggaran divariasikan. Implementasi yang gagal melewati uji ini mengindikasikan fungsi *fitness* yang salah spesifikasi dan harus diperbaiki sebelum hasilnya dipakai untuk menguji hipotesis.

III.7 Perangkat dan Lingkungan Eksperimen

Sejalan dengan Batasan Masalah yang membatasi pengujian pada satu *platform edge-IoT* spesifik, seluruh algoritma dijalankan pada perangkat dan lingkungan eksperimen yang identik, sebagaimana disyaratkan oleh variabel kontrol. Subbab ini menguraikan spesifikasi perangkat keras, perangkat lunak, serta prosedur pengukuran jejak memori dan waktu komputasi yang akan digunakan pada aktivitas *Demonstration* dan *Evaluation*.

Perangkat target penelitian ini adalah Raspberry Pi 5 dengan *System-on-Chip* Broadcom BCM2717 (quad-core ARM Cortex-A76 @2,4GHz dengan Cryptographic Extension, cache L2 512KB per-core, dan cache L3 bersama 2MB) serta varian RAM 8GB LPDDR4X-4267 (Raspberry Pi Ltd, 2026), mewakili kelas perangkat *edge-IoT* bertenaga menengah. Penyimpanan utama menggunakan microSD card slot berkecepatan tinggi bawaan perangkat. Sistem operasi menggunakan Raspberry Pi OS 64-bit berbasis Debian sebagai distribusi resmi yang direkomendasikan untuk *platform* ini.

Implementasi CoDA-EDA dan keenam algoritma pembandingan menggunakan bahasa Python. Hal ini konsisten dengan ekosistem perangkat lunak EDA yang telah mapan, Soloviev dkk. (2024) mencatat bahwa BOA dan varian EDA multivariat lainnya aktif diimplementasikan dalam paket Python seperti EDAspy, sehingga *baseline* BOA/iBOA dapat memanfaatkan atau merujuk pada implementasi referensi yang telah tersedia.

Jejak memori diukur melalui pemantauan *peak resident set size* proses masing-masing algoritma secara langsung pada perangkat target melalui dua instrumen yang saling melengkapi. Instrumen pertama adalah pelacak alokasi memori tingkat bahasa yang mengukur puncak alokasi struktur data algoritma secara bersih dari *overhead interpreter* dan dapat direset pada setiap pengukuran. Instrumen kedua adalah VmHWM pada `/proc/[pid]/status`, yang mengukur puncak RSS pada tataran sistem operasi sesuai definisi baku. Penggunaan kedua instrumen bersifat wajib karena masing-masing memiliki keterbatasan yang saling menutup. VmHWM pada perangkat target terukur sekitar 31 MB hanya untuk memuat interpreter beserta pustaka numeriknya, sementara jejak struktur data algoritma berskala *kilobyte* hingga

beberapa *megabyte*. Perbedaan antar-algoritma karenanya nyaris tidak terdeteksi bila hanya mengandalkan VmHWM. Selain itu, VmHWM bersifat *high-water mark* yang tidak pernah menurun sepanjang umur proses, sehingga beberapa pengukuran berurutan di dalam satu proses akan saling mencemari. Pada pelaksanaan awal, lima belas konfigurasi berbeda melaporkan nilai VmHWM yang praktis identik. Karena itu setiap konfigurasi pengukuran VmHWM dijalankan pada proses sistem operasi yang terpisah. Pengukuran dilakukan pada proses yang terisolasi (satu kombinasi algoritma-dataset dieksekusi pada satu waktu, tanpa proses lain yang bersaing memperebutkan sumber daya) agar hasil pengukuran mencerminkan konsumsi memori algoritma itu sendiri, bukan artefak dari beban sistem lain.

Pada setiap titik *NP*, jejak memori dicatat sebagai nilai puncak, bukan nilai rata-rata sepanjang eksekusi. Dua instrumen digunakan secara saling melengkapi. Metrik utama, *MemPeak_alloc*, merupakan puncak alokasi struktur data algoritma yang direkam menggunakan *tracemalloc*, sedangkan *MemPeak_sys* diperoleh dari VmHWM pada `/proc/[pid]/status` sebagai konteks penggunaan memori pada tingkat sistem operasi. Penggunaan nilai puncak dipilih karena kelayakan eksekusi pada perangkat *edge* ditentukan oleh kebutuhan memori maksimum yang harus dapat ditampung selama algoritma berjalan. Setiap konfigurasi dijalankan pada proses sistem operasi yang terpisah untuk mencegah sifat *high-water mark* VmHWM pada eksekusi sebelumnya memengaruhi pengukuran berikutnya.

Waktu komputasi diukur menggunakan *wall-clock time* yang dicatat langsung pada perangkat target, dipisahkan antara waktu pembelajaran awal (*initial training*) dan waktu satu siklus pembaruan inkremental. Untuk mengurangi variabilitas pengukuran akibat *thermal throttling* dan *dynamic frequency scaling* yang umum terjadi pada perangkat *edge* seperti Raspberry Pi 5, *CPU governor* ditetapkan pada mode *performance* yang konsisten sepanjang seluruh sesi pengujian, dan suhu perangkat dipantau agar tetap berada pada rentang suhu operasi resmi 0°C–70°C yang ditetapkan produsen (Raspberry Pi Ltd, 2026).

Tabel III.9 Spesifikasi Perangkat dan Lingkungan Eksperimen

Komponen	Spesifikasi/Prosedur
Perangkat	Raspberry Pi 5 (Raspberry Pi Ltd., 2026)
SoC	Broadcom BCM2712, quad-core ARM Cortex-A76 @ 2,4GHz, cache L2 512KB/core, cache L3 bersama 2MB

Lanjut ke halaman berikutnya

Tabel III.9 – lanjutan dari halaman sebelumnya

Komponen	Spesifikasi/Prosedur
RAM	8GB LPDDR4X-4267 SDRAM
Penyimpanan	microSD (mode SDR104); PCIe 2.0 x1 tersedia untuk SSD via M.2 HAT terpisah (tidak dipakai sebagai konfigurasi baku)
Suhu Operasi	0°C–70°C (spesifikasi produsen)
Sistem Operasi	Raspberry Pi OS 64-bit (berbasis Debian)
Bahasa Pemrograman	Python
Pengukuran Memori	Dual: puncak alokasi <code>tracemalloc</code> (metrik utama) + <code>VmHWM</code> pada <code>/proc/[pid]/status</code> (konteks sistem); satu proses terpisah per konfigurasi
Pengukuran Waktu	Wall-clock time langsung pada perangkat target; CPU governor mode <code>performance</code> ; suhu dipantau terhadap rentang operasi resmi

III.8 Desain Eksperimen

Berdasarkan variabel penelitian, dataset yang dikonstruksi, algoritma pembanding, serta perangkat dan prosedur pengukuran, subbab ini merancang empat skenario eksperimen yang masing-masing dipetakan langsung terhadap satu hipotesis sebagai operasionalisasi rinci aktivitas *Demonstration* dan *Evaluation*.

Eksperimen 1 menguji Hipotesis H1. Sesuai rumusan H1, eksperimen ini menguji dua faktor sekaligus, yakni tingkat korelasi atribut dan dimensi ruang solusi D . Ketujuh kondisi algoritma dijalankan pada kelima dataset ABAC Lab dalam kondisi stasioner (tanpa *drift*) pada tiga tingkat korelasi (rendah, sedang, tinggi) dengan akurasi *F-score*, WSC, dan *coverage* sebagai variabel yang diukur. *Workforce* ($D = 436$) dan *e-document* ($D = 1.907$) menjadi tumpuan vonis H1 karena keragaman poplanya memadai, sedangkan *healthcare*, *project-management*, dan *university* ($D = 46-54$) memberi taraf dimensi tambahan sebagai konteks pelengkap dengan interpretasi H1 yang dibatasi tingginya duplikasi. Dugaan awal penelitian memprediksi keunggulan CoDA-EDA menguat seiring meningkatnya tingkat korelasi.

Eksperimen 2a menguji Hipotesis H2. Eksperimen ini memanfaatkan eksekusi yang sama dengan Eksperimen 1, namun berfokus pada variabel *peak resident set size* yang dicatat sepanjang eksekusi. Dugaan awal adalah CoDa-EDA mempertahankan

jejak memori $O(D)$ yang secara konsisten lebih rendah dibanding keempat *baseline* berpopulasi tersebut, khususnya pada dataset berdimensi lebih besar.

Eksperimen 2b melengkapi Eksperimen 2a dengan menguji bentuk penskalaan jejak memori terhadap ukuran populasi, bukan hanya membandingkan nilai memori pada satu konfigurasi. Ukuran populasi NP divariasikan pada lima titik,

$$NP \in \{25, 50, 100, 200, 400\},$$

sehingga rentang pengujian mencakup kenaikan enam belas kali dari nilai terkecil ke nilai terbesar. Pengujian diterapkan pada dataset dengan dimensi berbeda dan menggunakan *MemPeak_alloc* sebagai metrik utama. Faktor eksperimen lain, seperti representasi solusi, fungsi *fitness*, dataset, dimensi D , dan prosedur pengukuran, dipertahankan sesuai protokol kesetaraan eksperimen. Setiap konfigurasi pengukuran juga dijalankan secara terisolasi agar nilai puncak memori suatu konfigurasi tidak tercampur dengan konfigurasi sebelumnya.

Makna NP terhadap penggunaan memori berbeda menurut mekanisme internal algoritma. Pada algoritma berpopulasi eksplisit seperti BOA, ACO, dan UBK, peningkatan NP berarti semakin banyak kandidat solusi berdimensi D yang harus dimaterialisasi selama proses optimasi. Komponen penyimpanan kandidat tersebut secara konseptual berukuran sebanding dengan

$$O(NP \times D),$$

di samping struktur tambahan yang spesifik terhadap masing-masing algoritma. Oleh karena itu, ketika NP dinaikkan, jumlah struktur data yang hidup secara bersamaan juga meningkat dan puncak alokasi memori diperkirakan ikut bertambah. Sebaliknya, CoDA-EDA tidak mematerialisasi populasi kandidat berukuran NP . Parameter NP berperan sebagai ukuran populasi virtual yang menentukan besar langkah pembaruan *Probability Vector*, yaitu sebesar $\pm 1/NP$. Kandidat baru dibangkitkan dan dievaluasi secara terbatas, sedangkan informasi pencarian yang dipertahankan secara persisten terdiri atas *Probability Vector* dan struktur dependensi orde satu berupa pohon Chow–Liu. Kedua struktur tersebut bertumbuh terhadap jumlah variabel D , bukan terhadap NP . Dengan demikian, peningkatan NP mengubah granularitas pembaruan probabilistik, tetapi tidak menyebabkan penyimpanan NP individu secara bersamaan. Mekanisme serupa berlaku pada keluarga *compact*, sedangkan iBOA mempertahankan statistik inkremental tanpa menyimpan populasi penuh.

Atas dasar perbedaan mekanisme tersebut, Hipotesis H2 tidak hanya diuji melalui

perbandingan nilai memori absolut, tetapi terutama melalui bentuk kurva penskalaan terhadap NP . Untuk merangkum sensitivitas tersebut digunakan rasio pertumbuhan

$$R_{NP} = \frac{MemPeak_{alloc}(NP = 400)}{MemPeak_{alloc}(NP = 25)}. \quad (III.3)$$

Nilai R_{NP} yang mendekati 1 menunjukkan bahwa jejak memori relatif tidak sensitif terhadap penambahan ukuran populasi, sedangkan nilai yang jauh lebih besar dari 1 menunjukkan pertumbuhan memori ketika NP diperbesar. Oleh karena itu, H2 memprediksi kurva CoDA-EDA, keluarga *compact*, dan iBOA relatif datar terhadap NP , sedangkan BOA, ACO, dan UBK menunjukkan pertumbuhan karena menyimpan populasi eksplisit. Pengujian bentuk penskalaan ini memberikan bukti yang lebih langsung bagi perbedaan $O(D)$ dan $O(NP \times D)$ dibandingkan perbandingan memori pada satu nilai NP saja.

Eksperimen 3 menguji Hipotesis H3. CoDA-EDAA dan iBOA dijalankan pada kelima dataset ABAC Lab dalam skenario *drift* rendah dan sedang dengan waktu komputasi per siklus pembaruan (dipisahkan antara siklus dengan pembaruan struktur dan siklus dengan pembaruan parameter saja) sebagai variabel yang diukur. Dugaan awal adalah rata-rata waktu per siklus CoDA-EDA lebih rendah karena pembaruan struktur hanya terjadi saat *drift* terdeteksi, berbeda dari iBOA yang berpotensi memperbarui struktur pada setiap siklus.

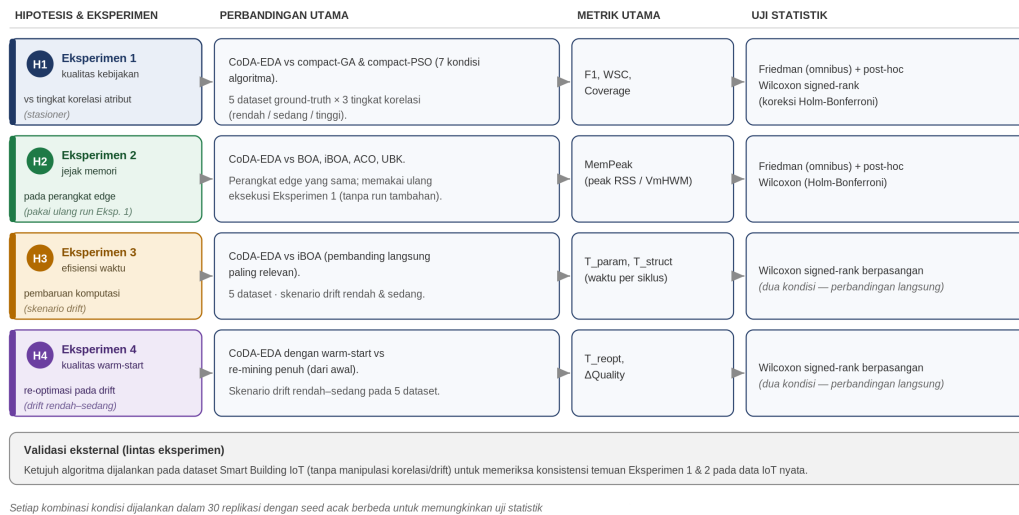
Eksperimen 4 menguji Hipotesis H4. Pada dataset dengan skenario *drift* rendah-sedang, CoDA-EDA dijalankan dengan dua strategi, yakni *warm-start* (melanjutkan dari model probabilistik sebelum *drift*) dan *re-mining* penuh (memulai ulang dari inisialisasi kosong setelah *drift* terdeteksi, sebagai kontrol internal). Waktu re-optimalisasi dan kualitas kebijakan pasca-*drift* dicatat untuk kedua strategi. Dugaan awal adalah strategi *warm-start* mencapai kualitas kebijakan yang setara dengan *re-mining* penuh dalam waktu yang jauh lebih singkat, khususnya pada skenario *drift* rendah.

Setiap kombinasi algoritma-dataset-skenario pada keempat eksperimen final dijalankan dengan skema enam *seed* pembangkitan dataset $\times$ lima replikasi algoritma (30 replikasi per kombinasi algoritma-dataset-skenario) pada eksperimen utama, dan lima *seed* pada validasi eksternal. Hal tersebut dilakukan guna menghasilkan data yang memadai untuk uji signifikansi statistik dan memperhitungkan variabilitas stokastik masing-masing algoritma.

Tabel III.10 Ringkasan Desain Eksperimen

No	Eksperimen	Hipotesis	Algoritma Dilibatkan	Dataset	Variabel Diukur
1	Kualitas kebijakan vs. korelasi dan dimensi	H1	7 kondisi (CoDA-EDA + 6 baseline)	Baris 1–5 (lima dataset; dua taraf D utama 436 dan 1.907 + tiga dataset kecil D=46–54 sebagai konteks), kondisi stasioner, 3 tingkat korelasi	Akurasi/F-score, WSC, coverage
2a	Jejak memori antar-algoritma	H2	CoDA-EDA vs BOA, iBOA, ACO, UBK	Sama dengan Eksperimen 1 (reuse eksekusi)	MemPeak_alloc (utama), MemPeak_sys
2b	Kurva penskalaan memori	H2	CoDA-EDA vs BOA, iBOA, ACO, UBK	Baris 1–2; NP divariasikan (25–400) pada dua nilai D	MemPeak_alloc sebagai fungsi NP dan D
3	Efisiensi waktu pembaruan	H3	CoDA-EDA vs iBOA	Baris 1–5, skenario drift rendah-sedang	Waktu komputasi per siklus pembaruan
4	Kualitas & efisiensi warm-start	H4	CoDA-EDA (warm-start vs re-mining penuh)	Baris 1–5, skenario drift rendah-sedang	Waktu re-optimasi, kualitas kebijakan pasca-drift
5	Validasi eksternal	H1-H4	7 kondisi (CoDA-EDA + 6 baseline)	Baris 6 (Smart Building IoT)	F-score relatif garis sepele, presisi, recall, coverage, memori, warm-start

Keterkaitan antara keempat hipotesis, eksperimen yang mengujinya, perbandingan yang dilakukan, metrik yang diukur, dan uji statistik yang digunakan divisualisasikan sebagai peta pada Gambar III.3.



Gambar III.3 Peta Eksperimen: Pemetaan Hipotesis H1-H4 terhadap Perbandingan, Metrik, dan Uji Statistik

Gambar III.3 menegaskan bahwa setiap hipotesis diuji oleh tepat satu eksperimen dengan perbandingan, metrik, dan uji statistik yang spesifik. Eksperimen 1 dan 2 membandingkan tujuh kondisi algoritma sekaligus sehingga memakai uji omnibus Friedman dengan post-hoc Wilcoxon berkoreksi Holm-Bonferroni, sedangkan Eksperimen 3 dan 4 hanya membandingkan dua kondisi sehingga cukup memakai Wilcoxon signed-rank berpasangan langsung. Peta ini juga memperlihatkan efisiensi rancangan pada Eksperimen 2 yang memakai ulang Eksperimen 1.

III.9 Metrik dan Instrumen Evaluasi

Definisi konseptual variabel terikat penelitian ini telah ditetapkan pada bagian sebelumnya. Bagian ini memformalkan definisi operasional, formula, serta instrumen pengukuran setiap metrik tersebut, yang merupakan kelanjutan langsung dari rancangan eksperimen dan prosedur pengukuran umum yang telah diuraikan.

Akurasi kebijakan diukur melalui *F-score* (F1), rata-rata harmonik antara presisi dan *recall* keputusan akses yang dihasilkan kebijakan hasil *mining* terhadap log akses uji/*holdout* (Díaz-Rodríguez dan Díaz-Pérez, 2025). Rumus *F1-score* didefinisikan

sebagai berikut:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}},$$

dengan $\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$ dan $\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$

Dengan *permit* sebagai kelas positif: TP (*true positive*) adalah permintaan akses yang sebenarnya diizinkan dan diklasifikasikan tepat sebagai *permit* oleh kebijakan hasil *mining*; FP (*false positive*) adalah permintaan yang sebenarnya ditolak namun diklasifikasikan sebagai *permit*; dan FN (*false negative*) adalah permintaan yang sebenarnya diizinkan namun diklasifikasikan sebagai *deny*.

Kompleksitas struktural kebijakan diukur melalui *Weighted Structural Complexity* (WSC) yang menjumlahkan banyaknya pasangan atribut-nilai yang digunakan pada seluruh aturan kebijakan (Perez-Haro dan Diaz-Perez, 2024). Definisi WSC dirumuskan sebagai berikut:

$$\text{WSC}(\pi) = \sum_{r \in \pi} |\text{attr}(r)|,$$

dengan π adalah kebijakan yang dihasilkan dan $|\text{attr}(r)|$ adalah banyaknya pasangan atribut-nilai yang menjadi syarat pada aturan r . Skema pembobotan tambahan per kategori atribut (subjek, objek, lingkungan), sebagaimana lazim pada literatur WSC, dapat diterapkan pada tahap implementasi apabila diperlukan; secara baku, seluruh pasangan atribut-nilai diberi bobot seragam (=1).

Cakupan (*coverage*) mengukur proporsi permintaan akses pada log uji yang tercakup oleh setidaknya satu aturan pada kebijakan hasil *mining*, alih-alih jatuh pada keputusan *default* (mis. *default-deny*). Kelengkapan cakupan sebagai dimensi kualitas kebijakan hasil mining (Perez-Haro dan Diaz-Perez, 2024):

$$\text{Coverage} = \frac{\text{jumlah permintaan akses yang cocok dengan } \geq 1 \text{ aturan}}{\text{total permintaan akses pada log uji}}.$$

Ketiga metrik ini (F1, WSC, dan *coverage*) bersama-sama merepresentasikan variabel kualitas kebijakan yang digunakan pada Eksperimen 1.

Jejak memori diformalkan sebagai nilai puncak RSS sepanjang eksekusi algoritma. MemPeak = nilai maksimum RSS(t) atas seluruh waktu *sampling* t, dengan RSS(t) adalah RSS proses pada waktu *sampling* ke-t. Sesuai prosedur dual, MemPeak dilaporkan dalam dua bentuk, yakni MemPeak_alloc, yaitu puncak alokasi struktur

data algoritma yang diukur *tracemalloc* dan menjadi metrik utama karena bersih dari *overhead* interpreter; dan *MemPeak_sys*, yaitu *VmHWM* pada tataran sistem operasi yang dilaporkan sebagai konteks penggunaan sumber daya nyata pada perangkat *edge*. Selain nilai pada satu titik konfigurasi, jejak memori juga dilaporkan sebagai kurva penskalaan terhadap ukuran populasi virtual NP dan terhadap dimensi soal D. Bentuk kurva inilah bukti terkuat bagi H2, karena klaim CoDA-EDA bersifat asimtotik, atau memori $O(D)$ yang tidak bergantung NP, sehingga yang perlu ditunjukkan adalah kedataran kurva terhadap NP dan kelinearan terhadap D, bukan selisih angka pada satu titik yang dapat tersamar oleh *offset baseline*. Metrik inilah yang digunakan pada Eksperimen 2 untuk menguji Hipotesis H2.

Waktu komputasi dipisahkan menjadi tiga komponen. T_{init} , waktu pembelajaran awal hingga elit pertama terbentuk. T_{param} , waktu satu siklus pembaruan yang hanya melibatkan pembaruan parameter (*Probability Vector*). Dan T_{struct} , waktu satu siklus pembaruan yang turut melibatkan pembelajaran ulang struktur dependensi saat *drift* terdeteksi. Ketiganya diukur sebagai *wall-clock time* langsung pada perangkat target dan digunakan pada Eksperimen 3 untuk menguji Hipotesis H3, khususnya melalui perbandingan rata-rata waktu per siklus CoDA-EDA terhadap iBOA.

Kualitas dan efisiensi strategi *warm-start* pada skenario *drift* diukur melalui dua metrik tambahan. Dua metrik tersebut adalah T_{reopt} , yaitu waktu yang dibutuhkan sejak *drift* terdeteksi hingga kebijakan mencapai kriteria konvergensi yang ditetapkan; dan $\Delta Quality$, yaitu selisih F1 dan WSC antara strategi *warm-start* dan *re-mining* penuh pada titik waktu yang setara, untuk menilai apakah *warm-start* mencapai kualitas kebijakan yang setara dengan *re-mining* penuh.

Tabel III.11 Definisi Operasional Metrik dan Instrumen Evaluasi

No	Metrik	Simbol	Definisi/Formula	Instrumen	Hipotesis
1	Akurasi	F1	Rata-rata harmonik presisi dan recall keputusan akses terhadap log uji	Skrip evaluasi Python terhadap log uji holdout	H1
2	Kompleksitas struktural	WSC	Jumlah pasangan atribut-nilai pada seluruh aturan kebijakan	Skrip evaluasi Python	H1

Lanjut ke halaman berikutnya

Tabel III.11 – lanjutan dari halaman sebelumnya

No	Metrik	Simbol	Definisi/Formula	Instrumen	Hipotesis
3	Cakupan	Coverage	Proporsi permintaan akses yang tercakup ≥ 1 aturan	Skrip evaluasi Python	H1
4	Jejak memori	MemPeak_alloc (utama); MemPeak_sys (konteks)	Puncak alokasi struktur data algoritma; dan puncak resident set size pada tataran sistem. Dilaporkan pula sebagai kurva penskalaan terhadap NP dan D	tracemalloc (utama) + VmH-WM pada /proc/[pid]/status, satu proses terpisah per konfigurasi (Subbab 3.7)	H2
5	Waktu inisialisasi	T_init	Waktu hingga elite pertama terbentuk	Wall-clock timer pada perangkat target	H3
6	Waktu siklus parameter	T_param	Waktu satu siklus pembaruan Probability Vector saja	Wall-clock timer pada perangkat target	H3
7	Waktu siklus struktur	T_struct	Waktu satu siklus dengan pembaruan struktur dependensi	Wall-clock timer pada perangkat target	H3
8	Waktu re-optimasi	T_reopt	Waktu sejak drift terdeteksi hingga konvergensi tercapai	Wall-clock timer pada perangkat target	H4
9	Selisih kualitas warm-start	Δ Quality	Selisih F1/WSC antara warm-start dan re-mining penuh	Skrip evaluasi Python	H4

Definisi operasional dan formula pada Tabel III.11 menjadi landasan pengolahan data dari pelaksanaan demonstrasi, sekaligus menjadi rujukan bagi teknik analisis statistik yang akan dilakukan.

III.10 Teknik Analisis Data

Data hasil pengukuran sembilan metrik pada Tabel III.11 yang dikumpulkan dari 30 replikasi ber-*seed* berbeda per kombinasi algoritma-dataset-skenario dianalisis secara statistik untuk menguji Hipotesis H1-H4 secara objektif. Subbab ini merancang teknik analisis data yang akan digunakan pada aktivitas *Evaluation*.

Kinerja algoritma metaheuristik pada replikasi berulang lazimnya tidak berdistribusi normal, mengingat sifat stokastik proses pencarian dan potensi hasil yang terpolarisasi (mis. sebagian replikasi konvergen cepat, sebagian lambat). Uji normalitas (Shapiro-Wilk) akan tetap dilakukan pada setiap kumpulan data sebagai langkah pemeriksaan awal, namun penelitian ini menetapkan uji statistik non-parametrik sebagai pendekatan baku, konsisten dengan konvensi umum pada literatur *benchmark* metaheuristik (Omran dkk., 2022) agar tidak bergantung pada asumsi normalitas yang berisiko dilanggar.

Untuk Eksperimen 1 dan 2 yang melibatkan perbandingan ketujuh kondisi algoritma sekaligus, digunakan uji Friedman sebagai uji omnibus non-parametrik untuk mendeteksi ada-tidaknya perbedaan signifikan di antara algoritma pada tiap kombinasi dataset-skenario. Apabila uji Friedman menunjukkan perbedaan signifikan ($p < \alpha$), dilanjutkan dengan uji post-hoc berpasangan, yakni uji Wilcoxon signed-rank antara CoDA-EDA dan tiap *baseline* yang relevan, dengan koreksi Holm-Bonferroni untuk mengendalikan inflasi galat Tipe I akibat perbandingan ganda. Unit blok pada uji Friedman/Wilcoxon adalah satu kombinasi (tingkat korelasi $\times$ *seed* pembangkitan dataset $\times$ replikasi algoritma) pada dataset tertentu untuk H1. Ketujuh kondisi algoritma dibandingkan berpasangan di dalam blok yang sama, sehingga variabilitas antar-blok tidak mencemari perbandingan antar-algoritma. Koreksi Holm-Bonferroni diterapkan per keluarga perbandingan, yakni satu keluarga = enam perbandingan post-hoc (CoDA-EDA vs tiap *baseline*) dalam satu dataset. Koreksi tidak dicampur lintas dataset agar keluarga uji tetap terdefinisi dan daya uji tidak terkuras berlebihan.

Perlu dicatat bahwa banyaknya replikasi bukan sekadar soal ketelitian, melainkan menentukan apakah pengujian dapat dilakukan sama sekali. Pada uji Wilcoxon signed-rank dua sisi, nilai p terkecil yang mungkin diperoleh dengan n pasangan adalah $\frac{2}{2^n}$, sehingga dengan lima replikasi nilai p tidak akan pernah turun di bawah 0,0625 dan taraf signifikansi 0,05 mustahil tercapai berapa pun besar perbedaannya. Diperlukan sekurang-kurangnya enam pasangan agar taraf tersebut dapat dicapai secara prinsip, dan ketetapan 30 replikasi memberi ruang daya uji yang memadai. Replikasi juga harus memisahkan dua sumber keacakan, yakni *seed* pembangkitan

dataset dan *seed* algoritma. Bila keduanya digerakkan bersama, ragam yang terukur mencampur variabilitas data dengan variabilitas algoritma, padahal yang hendak diuji adalah yang kedua. Karena itu replikasi disusun sebagai sejumlah pengulangan algoritma di atas setiap dataset yang tetap.

Untuk Eksperimen 3 dan 4 yang masing-masing membandingkan tepat dua kondisi, yakni CoDA-EDA terhadap iBOA dan strategi *warm-start* terhadap *re-mining* penuh pada CoDA-EDA sendiri, digunakan uji Wilcoxon signed-rank berpasangan secara langsung, tanpa memerlukan uji omnibus terlebih dahulu, mengingat hanya dua kondisi yang dibandingkan.

Tingkat signifikansi ditetapkan pada $\alpha = 0,05$ untuk seluruh uji. Mengingat nilai-p semata tidak merepresentasikan besaran praktis suatu perbedaan, hasil uji dilengkapi dengan ukuran efek yang sesuai untuk data non-parametrik, seperti *rank-biserial correlation* untuk uji Wilcoxon (Peres, 2026). Seluruh analisis statistik dilaksanakan menggunakan pustaka SciPy pada bahasa Python. Kualitas kebijakan dihitung terhadap keseluruhan log tempat kebijakan ditambang. Dengan demikian metrik dimaknai sebagai fidelitas rekonstruksi kebijakan, yakni sejauh mana himpunan aturan hasil penambangan mereproduksi keputusan izin/tolak yang tercatat pada log, dan bukan sebagai generalisasi prediktif ke permintaan yang belum pernah dilihat. Pemaknaan ini konsisten dengan tujuan *policy mining* ABAC, yaitu menyimpulkan kebijakan yang menjelaskan log akses yang tersedia. Selain ukuran efek, penelitian ini membedakan uji superioritas dari uji 'tidak lebih buruk'. Untuk klaim yang bersifat kesetaraan/non-inferioritas pada kualitas, ditetapkan margin non-inferioritas praktis $\delta = 0,02$ pada *F-score* (dan margin setara pada WSC) sebelum pengujian dilakukan. Suatu kondisi dinyatakan non-inferior bila batas atas selang kepercayaan 95/

Rancangan uji statistik untuk keempat hipotesis dirngkum pada Tabel III.12 yang memetakan tiap hipotesis terhadap eksperimen sumber datanya, kondisi yang dibandingkan, prosedur uji, dan ukuran efek pelengkapya.

Tabel III.12 Rancangan Uji Statistik per Hipotesis

Hipotesis	Eksperimen	Kondisi yang dibandingkan	Prosedur uji	Ukuran efek
H1 — Kualitas kebijakan	Eksperimen 1	Tujuh kondisi algoritma sekaligus, per dataset $\times$ tingkat korelasi	Friedman (omnibus) $\rightarrow$ post-hoc Wilcoxon signed-rank CoDA-EDA vs tiap baseline, koreksi Holm-Bonferroni	Rank-biserial
H2 — Jejak memori	Eksperimen 2	Tujuh kondisi algoritma sekaligus, per dataset	Friedman (omnibus) $\rightarrow$ post-hoc Wilcoxon signed-rank berkoreksi Holm-Bonferroni	Rank-biserial
H3 — Waktu pembaruan	Eksperimen 3	Dua kondisi (CoDA-EDA warm vs iBOA; serta warm vs always)	Wilcoxon signed-rank berpasangan langsung (tanpa omnibus)	Rank-biserial
H4 — Warm-start	Eksperimen 4	Dua kondisi (warm-start vs re-mining penuh)	Wilcoxon signed-rank berpasangan langsung (tanpa omnibus)	Rank-biserial

Tabel III.12 menjadi acuan pelaksanaan uji statistik atas hasil *Demonstration*. Dengan tersusunnya rancangan analisis ini, seluruh instrumen metodologi telah lengkap.

III.11 Ketertelusuran Rancangan Penelitian

Setelah seluruh variabel, rancangan algoritma, konstruksi dataset, desain eksperimen, metrik, dan teknik analisis ditetapkan, bagian ini menautkan kembali setiap keputusan rancangan ke akar masalah yang melatarinya dan ke bukti yang direncanakan untuk mengujinya. Tabel III.13 merangkum ketelusuran tersebut dari akar masalah, kebutuhan artefak, keputusan desain, hingga bukti evaluasi. Matriks ini memastikan tidak ada komponen CoDA-EDA yang ditetapkan tanpa justifikasi masalah, dan tidak ada klaim yang tidak memiliki rencana pembuktian.

Tabel III.13 Desain artefak dan pemetaan kebutuhan

ID	Masalah / akar penyebab	Kebutuhan artefak	Keputusan desain	Bukti evaluasi
M1	Populasi eksplisit membebani memori	Memori tidak bergantung pada NP	PV tanpa populasi eksplisit	Kurva memori vs NP ; analisis & regresi vs D (Gambar IV.x)
M2	Model univariat mengabaikan dependensi	Menangkap dependensi dengan biaya terbatas	Pohon Chow-Liu orde satu ($O(D)$ via Prim)	Perbandingan terhadap baseline compact univariat (compact-GA/PSO); kualitas per dimensi/korelasi (H1)
M3	Pembaruan struktur berulang mahal	Struktur diperbarui hanya saat diperlukan	Pemisahan pembaruan parameter–struktur + deteksi drift (JSD, τ)	Ablasi mekanisme pembaruan (warm vs always); trigger rate; overhead deteksi (H3)
M4	Kebijakan berubah setelah drift	Re-optimasi tanpa mining penuh	Warm-start dari model sebelumnya	Ablasi warm-start (warm vs cold); waktu menuju target kualitas + uji non-inferioritas (H4)
M5	Benchmark berpotensi tidak adil	Representasi, fitness, anggaran, operator setara	Kontrol eksperimen bersama (kesetaraan implementasi)	Tabel konfigurasi, sanity check, kontrol kesetaraan

Dengan matriks tersebut, alur argumentasi dari Bab I, Bab II, Bab III, hingga Bab IV menjadi satu kesatuan yang dapat ditelusuri. Setiap baris menautkan satu masalah pada konteks *edge-IoT* ke mekanisme desain yang menjawabnya dan ke pengujian yang menilainya.

III.12 Studi Pendahuluan

Studi pendahuluan diawali dengan analisis karakteristik dataset yang dilakukan sebelum eksperimen apa pun dijalankan. Analisis ini memeriksa properti tiap dataset yang menentukan kecocokannya dengan tiap hipotesis, seperti skala dan dimensi ruang solusi D , jumlah *permit* unik sebagai indikator risiko duplikasi, rasio *permit*, serta rantai derau divergensi Jensen-Shannon antar-jendela pada kondisi stasioner sebagai penentu kelayakan mekanisme pemicu-*drift*, sehingga keputusan penggunaan tiap dataset didasarkan pada bukti karakteristik dan bukan asumsi. Kelima dataset ABAC Lab beserta dataset validasi eksternal *Smart Building IoT* diprofilkan dengan prosedur yang sama yang hasilnya dirangkum pada Tabel III.14.

Tabel III.14 Karakteristik Dataset dan Kecocokannya dengan Hipotesis (Analisis Pra-Eksperimen)

Dataset	Dimensi D	Permit unik (risiko duplikasi)	Rasio permit	Lantai derau JSD (H3)	Kecocokan hipotesis
workforce	436	14.513 (rendah)	0,30	0,0066 (layak)	H1 baik; H2/H4 baik
e-document	1.907	32.961 (rendah)	0,30	0,0325 (tak layak)	H1 baik; H3 terbatas (rantai derau $> \tau$); H2/H4 baik
healthcare	46	14 (TINGGI)	0,30	0,0022 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik
project-management	51	17 (TINGGI)	0,30	0,0010 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik
university	54	90 (TINGGI)	0,30	0,0026 (layak)	H1 tumpul (duplikasi); H2/H3/H4 baik

Lanjut ke halaman berikutnya

Tabel III.14 – lanjutan dari halaman sebelumnya

Dataset	Dimensi <i>D</i>	Permit unik (risiko duplikasi)	Rasio permit	Lantai derau JSD (H3)	Kecocokan hipotesis
Smart Building IoT*	428*	5.729 (rendah)	0,75	0,0065 (layak)	berat-permit (sepele $F1=0,857$); H1 vs garis sepele; H2/H3/H4 baik

Tanda (*) menunjukkan dimensi efektif *Smart Building IoT* setelah subsampel terstratifikasi 8.000 baris dan penyatuan nilai ber-*support* rendah (berkas asalnya 5,7 juta permintaan, *permit* 4.273.814 dan *deny* 1.429.088). Rasio *permit* seragam 0,30 pada dataset ABAC Lab merupakan parameter konstruksi log, sedangkan 0,75 pada *Smart Building IoT* adalah rasio bawaan datanya.

Pembacaan Tabel III.14 menunjukkan bahwa keenam dataset memiliki karakteristik yang berbeda dan, karena itu, tidak seluruhnya memberikan kekuatan bukti yang sama untuk setiap hipotesis. Perbedaan paling penting terdapat pada dimensi ruang solusi *D*, jumlah *permit* unik, tingkat duplikasi efektif, rasio *permit*, serta lantai derau Jensen–Shannon divergence (JSD) pada kondisi stasioner. Karakteristik tersebut digunakan untuk menentukan peran masing-masing dataset dalam pengujian, sehingga interpretasi hasil tidak hanya didasarkan pada nilai metrik algoritma, tetapi juga mempertimbangkan tingkat kesulitan dan sifat statistik data yang dihadapi.

Dataset *healthcare*, *project-management*, dan *university* mewakili kelompok berdimensi rendah dengan *D* masing-masing 46, 51, dan 54. Ketiganya hanya memiliki 14, 17, dan 90 *permit* unik sehingga pengulangan pola akses relatif tinggi ketika log dibangkitkan dalam jumlah besar. Kondisi tersebut menyebabkan ruang pencarian cepat mengalami kejenuhan dan perbedaan kualitas antar-algoritma menjadi kurang tajam. Oleh karena itu, ketiga dataset ini tidak digunakan sebagai tumpuan utama untuk menilai H1. Meskipun demikian, ketiganya tetap relevan untuk H2 karena penskalaan memori merupakan properti algoritma, serta untuk H3 dan H4 karena lantai derau JSD-nya rendah, masing-masing sebesar 0,0022, 0,0010, dan 0,0026, sehingga mekanisme pemicu pembaruan struktur dapat diam ketika distribusi data memang stasioner.

Dataset *workforce* menempati kelompok dimensi menengah dengan $D = 436$ dan memiliki 14.513 *permit* unik. Jumlah pola unik yang jauh lebih besar dibanding

tiga dataset sebelumnya membuat *workforce* layak dijadikan salah satu dataset utama untuk pengujian kualitas kebijakan. Selain itu, rasio *permit* sebesar 0,30 dan rantai derau JSD stasioner sebesar 0,0066, yang berada di bawah ambang $\tau = 0,01$, menjadikannya lingkungan yang sesuai untuk menguji mekanisme pembaruan struktur berbasis *drift*. Pada kondisi stasioner, pergeseran distribusi akibat variasi sampel masih dapat dibedakan dari perubahan distribusi yang cukup besar untuk memicu pembelajaran ulang struktur. Karena karakteristik tersebut, *workforce* berperan penting baik pada evaluasi kualitas H1 maupun pada pengujian efisiensi pembaruan H3 dan H4.

Dataset *e-document* merupakan kasus berdimensi paling tinggi dengan $D = 1.907$ dan 32.961 *permit* unik. Besarnya ruang atribut menjadikan dataset ini kasus paling menantang bagi proses pencarian kebijakan sekaligus memberikan kondisi yang paling informatif untuk menguji manfaat pemodelan dependensi pada H1. Hasil eksperimen menunjukkan bahwa pada *e-document*, CoDA-EDA memperoleh F-score 0,452, sedangkan *compact-GA* dan *compact-PSO* masing-masing memperoleh 0,329 dan 0,260. Dengan demikian, keunggulan CoDA-EDA terhadap rerata kedua algoritma *compact* mencapai sekitar 0,16 F-score, jauh lebih besar daripada pada dataset berdimensi rendah maupun *workforce*. Pola ini menunjukkan bahwa manfaat struktur dependensi menjadi lebih nyata ketika ruang pencarian membesar dan model univariat harus melakukan pencuplikan tanpa informasi keterkaitan antar-variabel.

Namun, dimensi tinggi *e-document* juga menimbulkan konsekuensi terhadap mekanisme deteksi *drift*. Pada jendela analisis berukuran 800 entri, rantai derau JSD stasionernya mencapai 0,0325, lebih tinggi daripada ambang $\tau = 0,01$. Ketika ukuran jendela diperbesar menjadi 2.000 entri, nilainya menurun ke kisaran 0,013–0,016, tetapi tetap berada di atas ambang. Kondisi ini menunjukkan bahwa kenaikan jumlah sel kontingensi relatif terhadap jumlah sampel meningkatkan bias sampel-hingga estimator informasi. Akibatnya, mekanisme pemicu pembaruan struktur tidak dapat membedakan kondisi stasioner dari *drift* secara konsisten pada *e-document*, sehingga dataset ini tetap kuat untuk H1, H2, dan H4, tetapi bukti H3 harus ditafsirkan secara terbatas.

Dataset *Smart Building IoT* digunakan secara khusus sebagai validasi eksternal dan bukan sebagai dataset utama. Setelah subsampel terstratifikasi dan penyatuan nilai dengan *support* rendah, dataset ini memiliki dimensi efektif sekitar $D = 428$ dengan 5.729 *permit* unik dan rasio *permit* sekitar 0,75. Berbeda dari lima dataset ABAC Lab, korelasi maupun *drift* pada dataset ini tidak disuntikkan melalui generator penelitian. Nilai validasinya terletak pada asal generator yang independen, sehingga dataset

ini digunakan untuk memeriksa apakah pola temuan H1–H4 tetap muncul ketika CoDA-EDA diuji pada distribusi yang tidak dikendalikan oleh proses konstruksi dataset penelitian. Lantai derau JSD sebesar 0,0065 yang berada di bawah τ juga memungkinkan mekanisme H3 diam pada kondisi stasioner dan aktif ketika distribusi benar-benar berubah.

Secara keseluruhan, karakteristik keenam dataset memperlihatkan bahwa dimensi ruang solusi merupakan faktor penting dalam membaca hasil H1. Pada tiga dataset berdimensi rendah ($D = 46\text{--}54$), perbedaan CoDA-EDA terhadap keluarga *compact* relatif kecil karena ruang pencarian sederhana dan tingginya duplikasi menumpulkan perbedaan antar-algoritma. Pola serupa terlihat pada *workforce* ($D = 436$), tempat CoDA-EDA memperoleh F-score 0,889 dibanding 0,900 pada *compact-GA* dan 0,893 pada *compact-PSO*. Sebaliknya, pada *e-document* ($D = 1.907$), CoDA-EDA mempertahankan F-score 0,452 ketika *compact-GA* dan *compact-PSO* turun menjadi 0,329 dan 0,260. Dengan demikian, keunggulan pemodelan dependensi CoDA-EDA tidak muncul secara seragam pada seluruh dataset, tetapi terutama menjadi berarti pada ruang solusi berdimensi tinggi. Temuan validasi eksternal pada *Smart Building IoT* ($D \approx 428$) juga konsisten dengan pola tersebut karena CoDA-EDA tidak menunjukkan dominasi kualitas atas keluarga *compact* pada kelompok dimensi menengah.

Karakteristik dataset juga menjelaskan mengapa kekuatan bukti H3 berbeda antar-dataset. Mekanisme pembaruan struktur berbasis JSD dapat menunjukkan penghematan ketika lantai derau kondisi stasioner berada di bawah τ , seperti pada *workforce*, ketiga dataset berdimensi rendah, dan *Smart Building IoT*. Sebaliknya, pada *e-document*, tingginya lantai derau menyebabkan pemicu pembaruan struktur aktif meskipun perubahan distribusi yang sebenarnya kecil. Oleh karena itu, kegagalan menunjukkan penghematan H3 pada *e-document* tidak ditafsirkan sebagai kegagalan mekanisme pemisahan pembaruan itu sendiri, melainkan sebagai keterbatasan penggunaan ambang JSD tetap pada dataset dengan rasio jumlah sel kontingensi terhadap ukuran jendela yang tinggi.

Analisis ini dijalankan dengan instrumen tersendiri di luar rantai algoritma yang diuji, sehingga tidak memengaruhi anggaran evaluasi maupun kesetaraan algoritma. Instrumen ini berfungsi murni sebagai penyaring kecocokan pra-eksperimen. Setelah karakteristik dataset dipahami, rantai metodologi diuji melalui studi pendahuluan algoritmik yang dilaporkan pada paragraf-paragraf berikut.

Sebelum eksperimen penuh dilaksanakan, seluruh rantai metodologi telah diuji melalui studi pendahuluan pada perangkat target. Studi ini semula dimaksudkan

untuk memverifikasi kebenaran implementasi dan mengkalibrasi parameter eksekusi, tetapi menghasilkan temuan yang berimplikasi langsung pada temuan hipotesis. Subbab ini melaporkan pelaksanaan dan hasilnya secara lengkap, termasuk hasil yang tidak mendukung rumusan awal, agar dasar revisi hipotesis dapat ditelusuri dan dinilai secara terbuka. Dengan demikian studi pendahuluan ini diperlakukan sebagai tahap eksploratori yang menyempurnakan rumusan H1, H3, dan H4. Untuk mencegah hipotesis dibentuk dan diuji oleh data yang sama, eksperimen final pada Bab IV memakai kumpulan *seed* (baik *seed* pembangkitan dataset maupun *seed* algoritma) yang berbeda dari *seed* yang dipakai pada studi pendahuluan ini sehingga penyempurnaan rumusan pada tahap eksplorasi tidak mengambil manfaat dari data yang kemudian dipakai untuk mengujinya.

Studi pendahuluan membandingkan CoDA-EDA terhadap dua *baseline* keluarga *compact* (*compact-GA* dan *compact-PSO*) pada kedua dataset utama, mengikuti seluruh kontrol kesetaran, yakni representasi biner, fungsi *fitness*, operator perbaikan, prosedur pembenihan, dan anggaran evaluasi total yang identik. Empat dataset ber-*seed* berbeda dijalankan dengan sepuluh replikasi algoritma pada masing-masingnya dan menghasilkan 40 replikasi per kondisi. Anggaran evaluasi diskalakan menurut dimensi soal yakni 12.001 evaluasi untuk *workforce* dan 52.001 untuk *e-document*. Pengujian mengikuti protokol yang ada, yaitu uji Friedman sebagai uji omnibus, dilanjutkan uji Wilcoxon signed-rank berpasangan dengan koreksi Holm-Bonferroni bila uji omnibus signifikan, disertai pelaporan *effect size rank biserial*.

Tabel III.15 Hasil Studi Pendahuluan: Kualitas Kebijakan (F-score) pada Dua Dataset Utama

Dataset	Korelasi	CoDA-EDA	compact-GA	compact-PSO	Friedman	Post-hoc CoDA-EDA vs compact-GA
Workforce ($D = 436$)	Rendah	0,8354	0,8409	0,8123	$p = 0,025$	tidak signifikan ($p = 0,545$)
Workforce ($D = 436$)	Sedang	0,8644	0,8486	0,8471	$p = 0,592$	tidak dilanjutkan (omnibus tidak signifikan)
Workforce ($D = 436$)	Tinggi	0,9017	0,8852	0,8769	$p = 0,161$	tidak dilanjutkan (omnibus tidak signifikan)

Lanjut ke halaman berikutnya

Tabel III.15 – lanjutan dari halaman sebelumnya

Dataset	Korelasi	CoDA-EDA	compact-GA	compact-PSO	Friedman	Post-hoc CoDA-EDA vs compact-GA
E-document ($D = 1.907$)	Rendah	0,3754	0,2071	0,2395	$p = 8,5 \times 10^{-7}$	signifikan; $p_{\text{adj}} = 1,1 \times 10^{-4}$; $r = +0,751$
E-document ($D = 1.907$)	Sedang	0,3534	0,2308	0,2685	$p = 1,9 \times 10^{-4}$	signifikan; $p_{\text{adj}} = 4,9 \times 10^{-4}$; $r = +0,685$
E-document ($D = 1.907$)	Tinggi	0,4160	0,2944	0,3067	$p = 1,8 \times 10^{-3}$	signifikan; $p_{\text{adj}} = 7,9 \times 10^{-4}$; $r = +0,663$

Hasil pada Tabel III.15 memperlihatkan dua pola yang bertentangan dengan rumusan awal. Pertama, pada *workforce* keunggulan CoDA-EDA atas *compact-GA* hanya sebesar +0,016 pada korelasi sedang maupun tinggi dan tidak mencapai taraf signifikansi ($p = 0,197$ dan $p = 0,111$ pada uji langsung). Uji tren pertumbuhan keunggulan seiring naiknya korelasi juga tidak signifikan ($p = 0,073$). Kedua, pada *e-document* keunggulan CoDA-EDA justru sangat besar dan signifikan di seluruh tingkat korelasi dengan selisih *F-score* +0,12 hingga +0,17 dan *effect size* besar ($r = 0,66-0,75$), namun keunggulan tersebut tidak tumbuh seiring korelasi dan selisih terbesar justru teramati pada korelasi rendah ($p = 0,506$ untuk uji tren). Dengan demikian rumusan awal H1 yang memprediksi keunggulan muncul khusus pada korelasi tinggi dan mendekati *baseline* pada korelasi rendah tidak didukung data pada kedua dataset.

Faktor yang secara konsisten membedakan kedua dataset adalah dimensi ruang solusi. Pada *workforce* ($D = 436$), *compact-GA* yang murni univariat sudah mencapai *F-score* 0,885 pada korelasi tinggi sehingga ruang perbaikan yang tersisa sangat tipis. Pada *e-document* ($D = 1.907$), kualitas *compact-GA* turun ke 0,294 sementara CoDA-EDA bertahan pada 0,416. Penjelasan mekanistiknya sejalan dengan teori EDA, yakni ketika ruang pencarian membesar, *sampling* yang diarahkan struktur dependensi menjadi jauh lebih efektif dibanding *sampling* univariat yang harus menjelajahi ruang berdimensi tinggi tanpa panduan keterkaitan antar-variabel. Atas dasar inilah H1 dirumuskan ulang dengan dimensi ruang solusi sebagai faktor penentu dan

dimensi D ditambahkan sebagai variabel bebas.

Studi pendahuluan juga mengonfirmasi dua hal yang memperkuat rancangan. Pertama, kemampuan memanen korelasi memang berbeda antar algoritma. Kenaikan F -score dari korelasi rendah ke tinggi pada *workforce* mencapai +0,066 bagi CoDA-EDA ($p = 1,6 \times 10^{-6}$; $r = +0,873$) dibanding +0,044 bagi *compact-GA* ($p = 1,0 \times 10^{-3}$; $r = +0,598$), sehingga premis mekanistik mengenai manfaat pemodelan dependensi tetap didukung meskipun bentuk H1 semula tidak. Kedua, CoDA-EDA secara konsisten menghasilkan kebijakan yang lebih ringkas. WSC rata-rata 23,8 berbanding 28,9 (*compact-GA*) dan 38,2 (*compact-PSO*) pada *workforce*. Keringkasan ini merupakan dimensi kualitas kebijakan yang sah dan relevan bagi administrator, namun belum tercakup pada rumusan hipotesis awal.

Rangkaian tahap studi pendahuluan berikutnya (perluasan ke tujuh algoritma, pengukuran jejak memori, simulasi siklus pembaruan lintas jendela, pengujian pada *e-document*, eksplorasi perbaikan cakupan, dan validasi eksternal) berimplikasi langsung pada perumusan hipotesis. Ringkasnya H1 mengalami tiga kali revisi (dari bergantung tingkat korelasi, ke bergantung dimensi ruang solusi, hingga klaim terbatas terhadap keluarga *compact* univariat pada anggaran aturan memadai); H3 diberi syarat tingkat *drift*; dan H4 diperlonggar menjadi sekurang-kurangnya setara.

Tahap-tahap tersebut sekaligus menghasilkan kalibrasi rancangan dan pengukuran yang menjadi ketetapan bagi eksperimen. Penetapan ambang deteksi *drift* $\tau = 0,01$ dari pemisahan sinyal divergensi Jensen-Shannon terhadap lantai derau yang membesar seiring jumlah sel kontingensi; kalibrasi bobot fungsi *fitness* w_{fp} melalui uji monotonitas; serta optimasi implementasi perhitungan *mutual information* ke representasi terkemas-bit yang menurunkan puncak alokasi CoDA-EDA dari 2.195 KB menjadi 450 KB tanpa mengubah pohon dependensi yang dihasilkan maupun waktu komputasinya.

Bab IV Hasil dan Pembahasan

Bab ini melaporkan hasil aktivitas *Demonstration* dan *Evaluation*, antara lainnya adalah pengujian empiris Hipotesis H1-H4 pada kelima dataset beserta validasi eksternal pada *Smart Building IoT* yang mencakup keempat hipotesis. Pengujian kualitas (H1) dijalankan pada tiga *seed* dataset dikali lima replikasi algoritma per kombinasi dataset-korelasi dengan anggaran evaluasi diskalakan menurut dimensi soal. Seluruh eksekusi dilakukan pada Raspberry Pi 5. Penyajian disusun per hipotesis, dari kualitas kebijakan, jejak memori, efisiensi pembaruan, validasi eksternal, dan sintesis.

IV.1 Kualitas Kebijakan

Hipotesis H1 menyatakan CoDA-EDA menghasilkan kualitas kebijakan yang lebih tinggi dibanding keluarga *compact* univariat, dengan keunggulan yang menguat seiring dimensi ruang solusi, serta setara iBOA/BOA pada anggaran aturan memadai. Kualitas diukur melalui *F-score*, cakupan (*coverage*), dan kompleksitas struktural (WSC), dengan uji Friedman diikuti post-hoc Wilcoxon berkoreksi Holm-Bonferroni atas ketujuh kondisi algoritma. Gambaran menyeluruh rerata *F-score* pada kelima dataset disajikan pada Tabel IV.1.

Tabel IV.1 Rerata *F-score* Ketujuh Algoritma pada Lima Dataset ABAC Lab

Algoritma	workforce (436)	e- document (1.907)	healthcare (46)	project- mgmt (51)	university (54)
CoDA-EDA	0,889	0,452	0,779	0,898	0,830
compact-GA	0,900	0,329	0,765	0,883	0,822
compact-PSO	0,893	0,260	0,801	0,902	0,829
iBOA	0,888	0,452	0,802	0,909	0,824
BOA	0,832	0,633	0,835	0,927	0,831
ACO	0,870	0,661	0,761	0,886	0,835
UBK	0,853	0,630	0,782	0,853	0,869

Pada dua dataset utama, uji Friedman signifikan (*workforce* $p = 2,0 \times 10^{-14}$; *e-document* $p = 1,6 \times 10^{-42}$). Di *workforce* ($D = 436$) CoDA-EDA (0,889)

mengungguli secara signifikan BOA ($r = +0,56$) dan UBK ($r = +0,33$), sementara setara dengan *compact-GA*, *compact-PSO*, iBOA, dan ACO. Ia berada di papan atas tanpa dominan. Di *e-document* ($D = 1.907$), pola berbalik: CoDA-EDA (0,452) setara iBOA (0,452), mengungguli *compact-GA* ($r = +0,42$) dan *compact-PSO* ($r = +0,60$), namun kalah signifikan terhadap BOA, ACO, dan UBK dengan *effect size* besar ($r = -0,87$ hingga $-0,91$). Inilah defisit dimensi-besar yang menjadi ciri utama H1.

Ketiga dataset kecil berperan sebagai konteks pelengkap dan hasilnya menegaskan prediksi pra-eksperimen. Karena *permit* uniknya sedikit (14-90) sehingga duplikasi entri tinggi menjenuhkan kualitas, pembeda antar-algoritma tumpul. CoDA-EDA setara dengan mayoritas *baseline* pada ketiganya. Ia hanya kalah tipis-signifikan dari UBK pada *university* ($-0,039$), sekaligus mengungguli UBK pada *project-management* ($+0,045$). Karena duplikasi menjenuhkan ruang solusi dan menumpulkan pembeda antar-algoritma, hasil pada dataset kecil tidak dijadikan tumpuan vonis H1.

Perincian *e-document* (Tabel IV.2) memperlihatkan sumber mekanistik kekalahan pada dimensi besar. Kelemahan CoDA-EDA bukan pada presisi melainkan pada cakupan/*recall*. Dengan batas empat aturan, ACO, UBK, dan BOA menemukan aturan luas yang mencakup 47-62% permintaan, sedangkan CoDA-EDA dan iBOA menghasilkan aturan berpresisi tinggi namun sempit yang hanya mencakup sekitar 25%.

Tabel IV.2 Rincian Kualitas H1 pada *e-document* ($D = 1.907$; rerata lintas korelasi; post-hoc vs CoDA-EDA)

Algoritma	F-score	ΔF vs CoDA	Cakupan	Δ cakupan vs CoDA	WSC	Posisi terhadap CoDA-EDA (post-hoc Holm)
ACO	0,661	+0,209	0,572	+0,324	16,9	Unggul signifikan ($r = -0,91$)
BOA	0,633	+0,181	0,624	+0,376	11,2	Unggul signifikan ($r = -0,87$)
UBK	0,630	+0,178	0,468	+0,220	6,2	Unggul signifikan ($r = -0,91$)
iBOA	0,452	0,000	0,246	-0,002	13,8	Setara (n.s.)

Lanjut ke halaman berikutnya

Tabel IV.2 – lanjutan dari halaman sebelumnya

Algoritma	F-score	ΔF vs CoDA	Cakupan	Δ cakupan vs CoDA	WSC	Posisi terhadap CoDA-EDA (post-hoc Holm)
CoDA-EDA	0,452	—	0,248	—	11,8	— (acuan)
compact-GA	0,329	-0,123	0,156	-0,092	10,2	CoDA-EDA unggul ($r = +0,42$)
compact-PSO	0,260	-0,192	0,116	-0,132	11,0	CoDA-EDA unggul ($r = +0,60$)

Karena kelemahan bersumber dari cakupan, dua tuas hiperparameter diuji pada *e-document* untuk memeriksa apakah defisit dapat diperbaiki dan diterapkan seragam pada seluruh algoritma. Tuas bobot *false positive* w_{fp} tidak berpengaruh (cakupan CoDA-EDA 0,186 menjadi 0,187), menandakan persoalannya bukan pada pembobotan presisi. Sebaliknya, tuas anggaran jumlah aturan berdampak besar (Tabel IV.3). Dengan menaikkan dari empat ke delapan lalu dua belas aturan melonjakkan *F-score* CoDA-EDA dari 0,362 ke 0,571 dan menyempitkan jaraknya terhadap ACO dari -0,301 menjadi -0,100.

Tabel IV.3 Efek Anggaran Jumlah Aturan terhadap Kualitas CoDA-EDA pada *e-document*

Jumlah aturan	F-score	Cakupan	Δ cakupan vs 4-aturan	WSC	Waktu	Posisi terhadap baseline (post-hoc Holm)
4 (baku Tabel IV.2)	0,362	0,186	—	9,6	97 s	Kalah dari ACO, UBK, BOA; setara iBOA
8	0,571	0,391	+0,205	18,8	117 s	Kalah dari ACO, UBK; setara BOA & iBOA

Lanjut ke halaman berikutnya

Tabel IV.3 – lanjutan dari halaman sebelumnya

Jumlah aturan	F-score	Cakupan	Δ cakupan vs 4-aturan	WSC	Waktu	Posisi terhadap baseline (post-hoc Holm)
12	0,568	0,428	+0,242	26,8	136 s	Kalah dari ACO, UBK; setara BOA & iBOA

Pada dua belas aturan, post-hoc menempatkan CoDA-EDA setara iBOA ($p_{\text{adj}} = 0,52$), padahal pada empat aturan BOA mengungguli telak, sehingga CoDA-EDA hanya kalah moderat terhadap ACO dan UBK. Perbaikan ini memiliki harga pada waktu (97 menjadi 136 detik) dan kompleksitas (WSC 9,6 menjadi 26,8)

Vonis H1 adalah terdukung secara bersyarat. Hasil eksperimen tidak mendukung dugaan awal bahwa keunggulan CoDA-EDA meningkat secara konsisten seiring bertambahnya tingkat korelasi, dan juga tidak menunjukkan dominasi CoDA-EDA terhadap seluruh kelas algoritma pembanding. Sebaliknya, pola yang paling jelas adalah bahwa manfaat CoDA-EDA terhadap keluarga *compact* univariat bergantung pada dimensi ruang solusi. Pada *workforce* ($D = 436$), CoDA-EDA memperoleh F-score 0,889, yang secara praktis setara dengan *compact-GA* (0,900) dan *compact-PSO* (0,893). Sebaliknya, pada *e-document* ($D = 1.907$), CoDA-EDA mempertahankan F-score 0,452, sedangkan *compact-GA* dan *compact-PSO* masing-masing turun menjadi 0,329 dan 0,260. Dengan demikian, selisih CoDA-EDA terhadap rerata kedua algoritma *compact* meningkat dari sekitar $-0,008$ pada *workforce* menjadi sekitar $+0,158$ pada *e-document*.

Analisis menurut tingkat korelasi memperkuat interpretasi tersebut. Pada *e-document*, selisih F-score CoDA-EDA terhadap rerata *compact-GA* dan *compact-PSO* tetap berada pada kisaran sekitar $+0,15$ hingga $+0,17$ pada korelasi rendah, sedang, maupun tinggi. Selisih yang relatif stabil tersebut menunjukkan bahwa keunggulan CoDA-EDA pada dataset ini tidak semakin besar hanya karena tingkat korelasi dinaikkan. Faktor pembeda yang lebih konsisten adalah besarnya dimensi ruang solusi. Pada ruang berdimensi tinggi, pencuplikan berdasarkan probabilitas marginal secara independen memiliki peluang lebih besar menghasilkan kombinasi pasangan atribut-nilai yang kurang sesuai dengan struktur data. CoDA-EDA mengurangi keterbatasan tersebut dengan memanfaatkan dependensi orde satu yang dipelajari melalui *mutual information* dan direpresentasikan dalam pohon Chow–Liu sebagai informasi tambahan ketika membangkitkan kandidat aturan.

Temuan ini tidak berarti bahwa CoDA-EDA menjadi algoritma dengan kualitas penambangan statis terbaik pada dimensi tinggi. Pada *e-document*, CoDA-EDA tetap setara dengan iBOA, tetapi tertinggal dari BOA, ACO, dan UBK pada konfigurasi empat aturan. Perincian hasil menunjukkan bahwa kelemahan tersebut terutama berasal dari cakupan aturan: CoDA-EDA menghasilkan aturan dengan presisi tinggi tetapi cakupan relatif sempit, sedangkan BOA, ACO, dan UBK mampu menemukan aturan yang mencakup bagian log lebih besar. Ketika anggaran aturan ditingkatkan menjadi delapan dan dua belas aturan, jarak kualitas terhadap sebagian algoritma tersebut menyempit, tetapi dibayar dengan peningkatan waktu komputasi dan kompleksitas struktural. Oleh sebab itu, dukungan H1 dibatasi pada keunggulan pemodelan dependensi terhadap pendekatan *compact* univariat pada ruang solusi berdimensi tinggi, bukan pada dominasi terhadap seluruh metaheuristik.

Pola pada validasi eksternal juga konsisten dengan batas klaim tersebut. Dataset *Smart Building IoT* memiliki dimensi efektif sekitar $D = 428$, yang sebanding dengan *workforce*, dan tidak menggunakan korelasi hasil injeksi generator penelitian. Pada dataset ini CoDA-EDA memperoleh F-score 0,938, setara secara praktis dengan iBOA sebesar 0,943 dan sedikit di bawah *compact-GA* dan *compact-PSO* yang masing-masing memperoleh 0,961 dan 0,964. Hasil tersebut tidak menunjukkan keunggulan CoDA-EDA terhadap keluarga *compact* pada dimensi menengah, sehingga konsisten dengan temuan ABAC Lab bahwa manfaat pemodelan dependensi baru terlihat jelas pada kondisi berdimensi sangat tinggi.

Secara keseluruhan, H1 menunjukkan adanya pertukaran yang terkait langsung dengan keputusan rancangan CoDA-EDA. Representasi probabilistik yang diperkaya dengan pohon Chow–Liu memberikan informasi struktural yang tidak tersedia pada model *compact* univariat dan memberikan keuntungan nyata ketika ruang solusi menjadi sangat besar. Namun, penggunaan model ringkas tanpa populasi eksplisit juga membatasi keragaman eksplorasi dibanding algoritma berpopulasi seperti ACO dan UBK. Dengan demikian, posisi CoDA-EDA bukan sebagai penambang kebijakan statis yang selalu menghasilkan F-score tertinggi, melainkan sebagai algoritma *dependency-aware* yang mempertahankan kualitas kompetitif, menunjukkan keunggulan terhadap pendekatan *compact* univariat pada dimensi tinggi, dan sekaligus menyediakan karakteristik efisiensi memori serta pembaruan yang menjadi fokus utama penelitian ini.

Sebagai ilustrasi keluaran konkret proses penambangan, Tabel IV.4 menyajikan cuplikan aturan yang dihasilkan CoDA-EDA pada *workforce* dan *Smart Building IoT* beserta pembandingnya dari ACO. Semantik pencocokan mengikuti bahasa

ABAC Lab yang menyatakan bahwa beberapa nilai pada atribut yang sama bersifat disjungtif dan antar-atribut bersifat konjungtif.

Tabel IV.4 Contoh Kebijakan Hasil Mining (cuplikan aturan terbaca)

Sumber / algoritma	Metrik	Contoh aturan (cuplikan)	Cakupan aturan
workforce / CoDA-EDA	F1 0,885; WSC 20; 5 aturan	IF contractStatus=active AND action=view THEN permit	TP=1.291; FP=339
workforce / CoDA-EDA	(aturan berpresisi)	IF provider in {externalHelpdeskSupplier, telco} AND tenant=telco AND associatedContract in {contract001,003,005} AND action=modify THEN permit	TP=402; FP=3
workforce / ACO	F1 0,820; WSC 19; 5 aturan	IF tenantType=primary AND action in {modify,view} THEN permit	TP=2.879; FP=1.297 (luas, presisi rendah)
Smart Building IoT / CoDA-EDA	F1 0,998; WSC 276; 5 aturan	IF type in {TV, Security Cameras, Smart locks} AND action in {arm, control} THEN permit	TP=3.417; FP=0

Cuplikan ini memperlihatkan sifat keluaran yang berbeda antar-algoritma. CoDA-EDA cenderung menghasilkan aturan berpresisi tinggi (mis. FP=3 dari 405 pada aturan *modify workforce*, FP=0 pada IoT), sedangkan ACO cenderung menghasilkan aturan lebih luas bercakupan tinggi namun berpresisi lebih rendah (FP=1.297). Hal tersebut konsisten dengan analisis mekanistik cakupan-presisi pada Tabel IV.2. Perlu dicatat pula bahwa pada *Smart Building IoT* sebagian aturan CoDA-EDA membengkak WSC-nya (total 276) karena mengenumerasi nilai atribut bersifat pengenalan (uname/rname) sebagai disjungsi panjang.

IV.2 Jejak Memori

Hipotesis H2 menyatakan jejak memori CoDA-EDA tetap datar terhadap ukuran populasi virtual NP, sementara algoritma berpopulasi eksplisit tumbuh terhadap NP. Pengujian mengukur puncak alokasi (*tracemalloc*) pada NP = 25 hingga 400 dengan isolasi proses per konfigurasi, lalu memeriksa bentuk kurva penskalaan yang menjadi bukti terkuat bagi klaim asimtotik $O(D)$ versus $O(NP \times D)$. Hasil rerata lintas lima dataset disajikan pada Tabel IV.5.

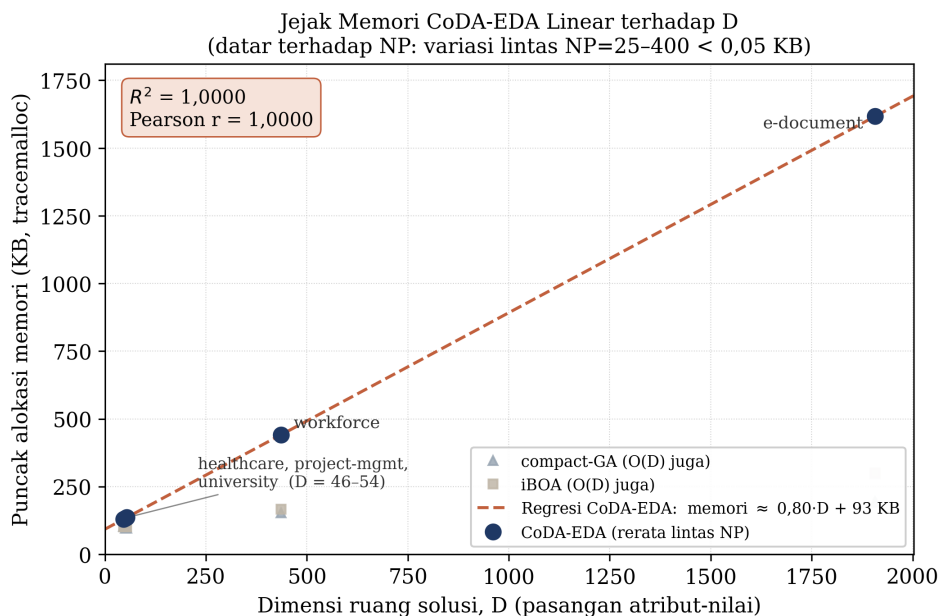
Tabel IV.5 Penskalaan Jejak Memori terhadap Ukuran Populasi NP (rerata lintas lima dataset)

Algoritma	alloc NP=25 (KB)	alloc NP=400 (KB)	Rasio 400/25	Keterangan
CoDA-EDA	492	492	1,00×	DATAR — memori tak bergantung NP ($O(D)$)
compact-GA	169	103	0,77×	Datar (compact univariat)
compact-PSO	172	106	0,76×	Datar (compact univariat)
iBOA	190	134	0,82×	Datar (statistik inkremental)
ACO	212	2.036	6,22×	Tumbuh — populasi/feromon
BOA	349	4.028	7,68×	Tumbuh — populasi + jaringan Bayes
UBK	387	5.169	9,20×	Tumbuh — populasi swarm

Pola pada Tabel IV.5 sesuai dengan mekanisme penyimpanan masing-masing kelas algoritma. Kenaikan NP dari 25 menjadi 400 berarti peningkatan enam belas kali ukuran populasi, tetapi pada CoDA-EDA peningkatan tersebut tidak menghasilkan enam belas kali lebih banyak kandidat yang disimpan secara bersamaan karena NP hanya mengatur granularitas pembaruan probabilistik. Akibatnya, *MemPeak_alloc* tetap sebesar 492 KB dan menghasilkan $R_{NP} = 1,00$. Sebaliknya, ACO, BOA, dan UBK harus mempertahankan struktur yang berkaitan dengan populasi eksplisit sehingga puncak alokasinya meningkat masing-masing sebesar 6,22, 7,68, dan 9,20

kali. Dengan demikian, hasil H2 bukan sekadar menunjukkan bahwa CoDA-EDA menggunakan sejumlah memori tertentu, tetapi menunjukkan bahwa struktur memori yang dipertahankannya tidak bertambah ketika ukuran populasi virtual diperbesar.

Vonis H2 terdukung kuat dan konsisten pada konfigurasi yang diuji. Rasio penskalaan $\text{alloc}(\text{NP} = 400)/\text{alloc}(\text{NP} = 25)$ CoDA-EDA tepat $1,00\times$. Jejak memorinya sama sekali tidak tumbuh ketika ukuran populasi dinaikkan enam belas kali lipat, dan setara dengan keluarga *compact* ($0,76\text{--}0,77\times$) serta iBOA ($0,82\times$). Sebaliknya, ketiga algoritma berpopulasi eksplisit tumbuh 6,2 hingga 9,2 kali lipat, yaitu ACO $6,22\times$, BOA $7,68\times$, dan UBK $9,20\times$. Sedangkan CoDA-EDA, yang justru memodelkan dependensi antar-atribut, tidak menanggung pertumbuhan memori itu sama sekali. Sebagai bukti empiris pelengkap bagi sifat linear terhadap D, Gambar IV.1 memetakan puncak alokasi memori CoDA-EDA terhadap D pada kelima dataset (rerata lintas NP). Regresi linear menghasilkan memori $\approx 0,80 \cdot D + 93 \text{ KB}$ dengan $R^2 = 1,000$ (Pearson $r = 1,000$), sedangkan variasi memori lintas NP = 25-400 berada di bawah 0,05 KB. Dengan demikian kedataran terhadap NP terbukti secara empiris langsung dan linearitas terhadap D turut terkonfirmasi secara empiris lintas dataset. Perlu dicatat, titik-titik D berasal dari lima dataset berdomain berbeda, sehingga bukti ini bersifat lintas-dataset dan bukan *sweep* D terkontrol pada satu dataset.



Gambar IV.1 Jejak Memori CoDA-EDA Linear Terhadap Dimensi Ruang Solusi D

Kedataran ini adalah inti kontribusi rancangan dari CoDA-EDA yang menyatukan

pembaruan tanpa populasi ala *compact* dengan pemodelan dependensi berorde-terbatas sehingga memperoleh manfaat struktur pada kualitas (di atas keluarga *compact*) sekaligus jejak memori yang tidak tumbuh dengan NP. Perlu dicatat bahwa alokasi mutlak CoDA-EDA pada satu titik (492 KB) lebih tinggi daripada *compact* (103-172 KB) dan iBOA (134-190 KB) karena beban komputasi pohon dependensi. Klaim H2 adalah tentang bentuk penskalaan terhadap NP, yang menentukan kelayakan pada perangkat terbatas saat populasi virtual diperbesar untuk kualitas, bukan tentang selisih pada satu titik. Untuk *gateway edge* yang perlu menaikkan NP demi kualitas tanpa membengkakkan RAM, sifat datar inilah keunggulan operasional CoDA-EDA.

IV.3 Efisiensi Pembaruan Kebijakan

Dua hipotesis diuji bersama pada subbab ini karena menyangkut siklus pembaruan yang sama. H3 menyatakan pemisahan frekuensi pembaruan (struktur hanya dipelajari ulang saat *drift* terdeteksi sedangkan parameter tiap siklus) mengurangi waktu tanpa penurunan kualitas signifikan dan bersyarat pada tingkat *drift*. H4 menyatakan strategi *warm-start* mencapai kualitas sekurang-kurangnya setara *re-mining* penuh dalam waktu jauh lebih singkat. Pengujian menyimulasikan siklus pembaruan dengan membagi log menjadi lima jendela. Struktur Chow-Liu hanya dipelajari ulang bila divergensi Jensen-Shannon melampaui $\tau = 0,01$. Empat kondisi dibandingkan: *warm* (struktur dipicu *drift*), *always* (struktur tiap jendela), *cold* ([re-mining] penuh), dan iBOA *cold*. Hasil ringkas pada Tabel IV.6.

Tabel IV.6 Hasil Pengujian H3 dan H4 pada Lima Dataset (rerata; warm/always/cold)

Dataset / skenario	Pemicu relearn	warm (s)	always (s)	cold (s)	Vonis
workforce / stasioner	0 dari 4	3,46	4,58	9,05	H3 TERDUKUNG ($p = 6 \times 10^{-5}$); H4 $2,6\times$, F1 $+0,095$
workforce / drift sedang	4 dari 4	4,52	4,53	8,95	H3 tak berlaku (struktur memang perlu); H4 $2,0\times$, F1 $+0,061$

Lanjut ke halaman berikutnya

Tabel IV.6 – lanjutan dari halaman sebelumnya

Dataset / skenario	Pemicu relearn	warm (s)	always (s)	cold (s)	Vonis
e-document / stasioner	4 dari 4	22,3	22,3	28,2	H3 tak dapat ditunjukkan (lantai derau $> \tau$); H4 $1,27\times$, F1 $+0,102$
project- management / stasioner	0 dari 4	0,17	0,21	4,45	H4 $25,9\times$, F1 $-0,020$ (kecil, jenuh)
university / stasioner	0 dari 4	0,08	0,11	3,76	H4 $49\times$, F1 $+0,007$

Vonis H3 terdukung secara bersyarat. Pada *workforce* stasioner, pemicu *drift* tidak menyala sama sekali sepanjang empat siklus sementara kondisi *always* menyegarkan struktur tiap siklus. Akibatnya *warm-start* lebih cepat (3,46 vs 4,58 detik) pada seluruh replikasi ($p = 6,1 \times 10^{-5}$) tanpa penurunan kualitas. Pada *workforce drift* rendah/sedang, pemicu menyala tiap jendela sehingga *warm* menjadi identik dengan *always*, yang mana ini adalah perilaku yang memang dikehendaki dan bukan merupakan kegagalan. Pada *e-document*, H3 tetap tidak dapat ditunjukkan karena lantai derau divergensi melampaui τ sehingga pemicu selalu menyala. Klaim H3 karenanya bersyarat pada tingkat *drift* dan lantai derau dataset.

Vonis H4 terdukung penuh dan pada banyak skenario melampaui rumusannya. *Warm-start* menyelesaikan re-optimasi lebih cepat dari *re-mining* penuh pada seluruh dataset: $2,0 - 2,6\times$ pada *workforce*, $1,27\times$ pada *e-document*, dan $21 - 52\times$ pada tiga dataset kecil, unggul pada seluruh replikasi. Rumusan awal hanya memprediksi kualitas setara, namun pada dua dataset utama kualitas *warm-start* justru lebih tinggi (F1 $+0,06$ hingga $+0,17$), karena *warm-start* hanya menambal *permit* yang belum tercakup alih-alih membangun ulang seluruh kebijakan. Pada tiga dataset kecil, selisih kualitas *warm-vs-cold* kecil dan bertanda campur ($-0,057$ hingga $+0,031$) akibat kejenuhan duplikasi sehingga vonis kualitas H4 bertumpu pada dua dataset utama.

Kedua hipotesis menjawab kebutuhan operasional yang memotivasi penelitian, yakni *gateway edge* yang menyegarkan kebijakan berkala. Selisih satu siklus dari 9 ke 3,5 detik (*workforce*) atau dari 28 ke 22 detik (*e-document*) menentukan anggaran daya dan lamanya jendela pemeliharaan. H4 juga menuntaskan posisi kontekstual

CoDA-EDA terhadap iBOA. Pada penambangan statis iBOA setara/sedikit unggul, tetapi pada pembaruan berkala *warm-start* CoDA-EDA lebih cepat sekaligus berkualitas lebih tinggi. Bersama H2, temuan ini menegaskan keunggulan pembeda CoDA-EDA terletak pada efisiensi sumber daya dan pembaruan, bukan pada kualitas penambangan sekali jalan.

IV.4 Validasi Eksternal

Seluruh pengujian di atas memakai dataset dari generator ABAC Lab yang diperluas sendiri. Validasi eksternal menguji apakah temuan bertahan pada data yang dibangkitkan oleh pihak lain (Díaz-Rodríguez dan Díaz-Pérez, 2025) dengan generator, kebijakan, dan distribusi yang tidak dikendalikan penelitian ini. Dua sifat dataset ini menuntut penyesuaian, antara lain rasio *permitnya* berat (0,75) sehingga kebijakan sepele yang mengizinkan semua permintaan sudah beroleh *F-score* 0,858. Pembandingan sah adalah garis dasar sepele itu dan berkas aslinya juga besar (5,7 juta permintaan) sehingga disubsampel terstratifikasi menjadi 8.000 barus berdimensi efektif 428. Hasil kualitas (H1) disajikan pada Tabel IV.7.

Tabel IV.7 Validasi Eksternal Smart Building IoT — Kualitas ($D = 428$; garis sepele $F1=0,858$; 5 seed)

Algoritma	F-score	Presisi	Cakupan	>sepele	Keterangan
BOA	0,998	1,000	0,747	5/5 seed	Kebijakan hampir dipulihkan sempurna
ACO	0,994	1,000	0,742	5/5 seed	Setara BOA
compact-PSO	0,964	1,000	0,704	4/5 seed	Di atas garis sepele
compact-GA	0,961	1,000	0,697	5/5 seed	Di atas garis sepele
iBOA	0,943	1,000	0,674	4/5 seed	Setara CoDA-EDA
CoDA-EDA	0,938	1,000	0,665	5/5 seed	Melampaui sepele di seluruh seed
UBK	0,834	1,000	0,539	2/5 seed	Satu-satunya yang tak konsisten

Validasi eksternal H1 berhasil. CoDA-EDA melampaui garis dasar sepele pada seluruh lima *seed* (*F-score* 0,938 berbanding 0,858), demikian pula lima algoritma lain, hanya UBK yang tidak konsisten (2 dari 5 *seed*). Uji Friedman signifikan

($p = 1,6 \times 10^{-3}$). Presisi seluruh algoritma 1,000 sehingga pembedanya semata cakupan dan posisi CoDA-EDA setara iBOA serta sedikit di bawah keluarga *compact*/BOA/ACO. Hal ini konsisten dengan pola dimensi dan anggaran aturan yang ada.

Keempat hipotesis turut diuji pada dataset ini. H2 (memori) menyatakan urutan jejak memori antar-algoritma konsisten dengan temuan ABAC Lab. H3 dan H4 (pembaruan) menyatakan siklus pembaruan lima jendela dijalankan pada dua mode pembentukan jendela. Pada model jendela sepadan (stasioner), rantai derau IoT yang rendah ($0,0065 < \tau$) membuat pemicu *drift* CoDA-EDA menyala hanya 1 dari 5 jendela, sehingga *warm-start* lebih cepat dari *always* (6,7 vs 7,9 detik) dengan kualitas setara (F1 0,977 vs 0,973). H3 yang gagal ditunjukkan pada *e-document* justru dapat diperlihatkan di sini. Pada mode jendela ber-*drift* (diurut menurut atribut lokasi), pemicu menyala 5 dari 5 jendela sesuai rancangan. H4 terkonfirmasi pada kedua mode. *warm-start* 2,1-2,5 kali lebih cepat dari *re-mining* penuh (6,7 vs 16,9 detik) dengan kualitas lebih tinggi (F1 0,977 vs 0,914).

Dengan demikian, temuan menunjukkan pola yang konsisten dengan keempat hipotesis pada satu dataset sintesis lintas-generator. Hal ini menunjukkan ketahanan lintas-generator teruji, bukan generalisasi ke data operasional *edge-IoT*. Kerangka penambangan bekerja, CoDA-EDA menghasilkan kebijakan bermakna jauh di atas garis sepele, jejak memorinya berperilaku sesuai H2, dan mekanisme H3/H4 terlihat. Bahkan H3 yang terbatas pada *e-document* dapat ditunjukkan pada IoT berkat rantai derau yang lebih rendah, memberi bukti bahwa keterbatasan H3 bersifat spesifik dataset (bias sampel-hingga), bukan cacat mekanisme.

IV.5 Sintesis, Keterbatasan, dan Jawaban Rumusan Masalah

Subbab ini menyintesis hasil keempat hipotesis lintas seluruh dataset, memosisikannya terhadap kebaruan yang diklaim, menjawab rumusan masalah, dan menyatakan keterbatasan. Status ringkas H1-H4 disajikan pada Tabel IV.8.

Tabel IV.8 Sintesis Status Hipotesis H1-H4 Lintas Dataset

Hipotesis	Dua dataset utama ($D = 436$ & 1.907)	Tiga dataset kecil ($D = 46-54$)	Smart Building IoT (eksternal)
H1 — Kualitas	Unggul keluarga compact; setara iBOA; kalah moderat ACO/UBK pada D besar	Setara mayoritas (tumpul; duplikasi tinggi)	Melampaui sepele 5/5; setara iBOA
H2 — Memori	Terkonfirmasi (datar $1,00\times$ vs $6,2-9,2\times$)	Konsisten (properti algoritma)	Konsisten
H3 — Pembaruan	Terdukung (workforce stasioner); tak tampil di e-document (lantai derau)	Terbatas (kejenuhan)	Terdukung (mode sepadan; relearn 1/5)
H4 — Warm-start	Terdukung penuh ($2,0-2,6\times$, kualitas naik)	Terdukung ($21-52\times$; kualitas campur)	Terdukung ($2,1-2,5\times$, kualitas naik)

Dua pola menonjol dan menjadi tumpuan kontribusi. Pertama, H2 (memori datar terhadap NP) terkonfirmasi menyeluruh. CoDA-EDA datar $1,00\times$ sementara algoritma berpopulasi eksplisit (BOA, ACO, UBK) tumbuh 6,2 hingga 9,2 kali lipat. Kedua, H4 *warm-start* terkonfirmasi konsisten lintas lima dataset dan pada data eksternal. Sebaliknya, H1 (kualitas statis) bersifat terbatas yang bergantung pada dimensi dan anggaran aturan, dan H3 bersyarat pada tingkat *drift* serta lantai derau. *Trade-off* inti menjadi jelas. CoDA-EDA menukar *recall* penambangan statis pada dimensi besar demi jejak memori yang tidak tumbuh dengan ukuran populasi dan pembaruan berkala yang jauh lebih cepat. Untuk *gateway edge* bersumber daya terbatas yang menyegarkan kebijakan berkala, pertukaran itu menguntungkan. Pusat kontribusi bertumpu pada H2 dan H4, bukan pada klaim keunggulan penambangan sekali jalan. Terhadap tida unsur kebaruan, hasil memberi dukungan yang terukur. Kebaruan pertama, yakni EDA *compact* yang sadar dependensi pada ABAC *policy mining*

terkonfirmasi bekerja. CoDA-EDA menghasilkan kebijakan bermakna pada seluruh dataset sambil mempertahankan memori datar. Kebaruan kedua, yakni pembatasan dependensi orde satu juga didukung. BOA yang tak membatasi orde tidak unggul kualitas pada ruang ABAC jarang dan jauh lebih boros memori, sementara pada anggaran aturan memadai CoDA-EDA setara BOA namun dengan memori tak tumbuh. Orde satu memadai untuk keuntungan struktur tanpa ongkos penuh. Kebaruan ketiga, yakni pemisahan frekuensi pembaruan juga terbukti pada H3 dan menjadi fondasi keunggulan *warm-start* H4. Posisi CoDA-EDA terhadap iBOA yang bergantung konteks memperjelas posisi yang diisi.

Dengan demikian rumusan masalah terjawab. Masalah pertama tentang bagaimana merancang algoritma *compact* yang memodelkan dependensi tanpa membengkakkan memori dijawab rancangan CoDA-EDA dan dikonfirmasi H2. Masalah kedua tentang kualitas kebijakan dijawab pada H1 yang menunjukkan bahwa algoritma yang dirancang kompetitif dan hemat, mengungguli keluarga *compact* serta setara iBOA/BOA pada anggaran memadai, dengan keterbatasan jujur pada dimensi besar terhadap ACO/UBK. Masalah yang ketiga tentang pembaruan inkremental mengurangi biaya dijawab pada H3-H4. Pemicu *drift* menghemat pada data stabil, *warm-start* 2-2,6 kali lebih cepat dengan kualitas sekurang-kurangnya setara. Masalah keempat tentang jejak memori dan waktu pada *edge* dijawab pengukuran langsung pada Raspberry Pi 5, sehingga semua tujuan penelitian tercapai.

Keterbatasan hasil dinyatakan secara terbuka. Untuk validitas internal, kesetaraan antar kondisi tetap dijaga. Meskipun demikian, implementasi BOA membatasi kandidat induk pada skala besar sebagai bentuk aproksimasi yang lazim. Mengenai validitas eksternal, generalisasi diuji hanya pada satu dataset pihak lain. Penetapan ukuran subsampel dan ambang batas dukungan minimum turut memengaruhi dimensi efektif yang teramati. Pada validitas konstruk, kualitas diringkas dengan *F-score*, WSC, dan *coverage*. Metrik ini tidak menangkap seluruh aspek keterbacaan kebijakan bagi administrator. Hal ini terlihat pada aturan CoDA-EDA berpengenal-banyak di *Smart Building IoT* yang WSC-nya membengkak akibat enumerasi disjungtif. Terkait validitas simpulan, uji non-parametrik dipilih untuk menghindari asumsi normalitas. Perbandingan utama memiliki daya uji yang memadai (45 blok). Sementara itu, beberapa perbandingan pada dataset kecil dan validasi eksternal dengan jumlah ulangan kecil (5 *seed*) hanya dilaporkan secara deksriptif.

Secara keseluruhan, Bab IV menegaskan kontribusi CoDA-EDA terletak pada efisiensi sumber daya dan efisiensi pembaruan, bukan pada dominasi kualitas penambangan statis, dan bahwa temuan tersebut bertahan pada data yang dibangkitkan pihak lain.

Bab V Kesimpulan dan Saran

V.1 Kesimpulan

Penelitian ini merancang dan menguji CoDA-EDA, sebuah *Estimation of Distribution Algorithm compact* yang sadar dependensi untuk penambangan kebijakan ABAC pada perangkat *edge-IoT* bersumber daya terbatas. Berdasarkan hasil pengujian yang telah dilakukan, ditarik kesimpulan berikut sesuai tujuan penelitian, rumusan masalah, dan hipotesis H1-H4 yang diajukan.

Pertama, terkait rancangan artefak yang menjawab rumusan masalah 1, sekaligus mendasari hipotesis H2. Algoritma CoDA-EDA berhasil dirancang dengan menggabungkan *Probability Vector* univariat, struktur dependensi pohon Chow-Liu berorde satu, dan pemisahan frekuensi pembaruan struktur dari parameter. Rancangan ini menjawab bagaimana dependensi antar-atribut dapat dimodelkan tanpa membengkakkan memori, sifat yang selanjutnya diuji secara empiris pada Hipotesis H2.

Kedua, terkait kualitas kebijakan yang menjadi hipotesis H1. CoDA-EDA menghasilkan kebijakan yang kompetitif namun dengan keunggulan yang terbatas. CoDA-EDA secara signifikan mengungguli keluarga *compact* univariat, dengan keunggulan yang menguat seiring dimensi, setara dengan iBOA dan BOA pada anggaran yang memadai, tetapi tertinggal moderat dari ACO dan UBK pada penambangan statis berdimensi besar. Sebagian besar defisit pada dimensi besar terbukti merupakan artefak anggaran jumlah aturan yang tidak proporsional, dan ini bukan merupakan kelemahan struktural mutlak. Dengan demikian H1 didukung secara bersyarat.

Ketiga, terkait jejak memori (Hipotesis H2). CoDA-EDA terkonfirmasi mempertahankan memori yang datar terhadap ukuran populasi (rasio 1,00), sementara algoritma berpopulasi eksplisit (BOA, ACO, dan UBK) tumbuh 6,2 hingga 9,2 kali lipat ketika populasi virtual dinaikkan enam belas kali lipat. Hasil ini konsisten pada kelima dataset. Kedataran serupa juga dimiliki iBOA dan keluarga *compact*, sehingga keunggulan memori CoDA-EDA berlaku terhadap ketiga algoritma berpopulasi eksplisit. H2 terkonfirmasi menyeluruh.

Keempat, terkait efisiensi pembaruan berkala (Hipotesis H3 dan H4). Dua hipotesis dijawab sekaligus pada sisi ini. Pemisahan frekuensi pembaruan yang dipicu *drift* menghemat total waktu komputasi ketika data relatif stabil, dengan kualitas non-

inferior (Hipotesis H3), dan strategi *warm-start* menyelesaikan re-optimalisasi 2,0 hingga 2,6 kali lebih cepat dibanding *re-mining* penuh dengan kualitas sekurang-kurangnya setara (Hipotesis H4). H4 terdukung konsisten, sedangkan H3 bersifat bersyarat pada tingkat *drift* dan rantai derau sehingga tidak dapat ditunjukkan pada *e-document*.

Kelima, validasi lintas-generator (konsistensi Hipotesis H1-H4). Pengujian pada dataset *Smart Building IoT* menunjukkan pola yang konsisten dengan empat hipotesis: kebijakan melampaui garis sepele (mendukung H1), jejak memori berperilaku datar sesuai H2, serta mekanisme H3-H4 terlihat, dengan H3 yang tidak tampak pada *e-document* karena rantai deraunya lebih tinggi. Temuan ini bertahan lintas-generator sintetis, meskipun belum digeneralisasi ke data operasional *edge-IoT*.

Dengan demikian, kontribusi utama penelitian ini bukan terletak pada dominasi kualitas penambangan statis, melainkan pada pertukaran rancangan yang koheren dan menguntungkan bagi konteks target. CoDA-EDA menukar sebagian *recall* penambangan statis demi jejak memori yang tidak tumbuh dengan ukuran populasi dan pembaruan berkala yang jauh lebih efisien. Untuk *gateway edge* bersumber daya terbatas yang menuntut pembaruan kebijakan berulang, pertukaran ini menjadikan CoDA-EDA pilihan yang sesuai.

V.2 Saran

Saran disusun dalam dua kelompok, yakni saran penerapan praktis dan saran pengembangan lanjutan.

Untuk penerapan praktis, CoDA-EDA disarankan dipilih ketika prioritas utama adalah kehematan memori yang dapat diprediksi dan efisiensi pembaruan kebijakan berkala pada perangkat terbatas, misalnya *gateway* kontrol akses di tepi jaringan yang menyegarkan kebijakan secara berkala tanpa mengirim log ke *server*. Bila prioritasnya adalah kualitas penambangan sekali jalan pada dataset statis berdimensi besar, algoritma berpopulasi seperti ACO dan UBK lebih sesuai. Jumlah aturan pada *separate-and-conquer* sebaiknya diperlakukan sebagai parameter yang disetel menurut kebutuhan. Anggaran aturan yang lebih besar menaikkan cakupan dengan biaya waktu dan kompleksitas kebijakan yang lebih tinggi.

Untuk pengembangan lanjutan, pertama, ambang deteksi *drift* (τ) perlu dibuat adaptif terhadap jumlah sel kontingensi agar mekanisme H3 dapat bekerja pada dataset berdimensi besar tempat rantai derau divergensi informasi meningkat. Ini akan mengatasi keterbatasan H3 pada *e-document*.

Kedua, kelemahan cakupan pada dimensi besar dapat ditangani lebih mendasar melalui perancangan fungsi *fitness* atau mekanisme pencarian yang sadar cakupan, sehingga cakupan tidak bergantung pada anggaran jumlah aturan. Ini berpotensi menutup jarak terhadap algoritma berpopulasi tanpa mengorbankan kehematan memori dan dapat menjadi kontribusi lanjutan tersendiri.

Ketiga, perluasan model dependensi ke orde lebih tinggi secara selektif dan hemat memori layak dieksplorasi untuk menguji apakah keuntungan struktur dapat ditingkatkan tanpa kembali ke biaya penuh BOA.

Keempat, validasi eksternal perlu diperluas ke lebih banyak dataset dari pihak lain dan, bila memungkinkan, ke data operasional nyata dari perangkat IoT guna memperkuat klaim generalisasi di luar dataset sintetis. Pengujian pada ragam perangkat *edge* yang lebih luas serta integrasi CoDA-EDA ke dalam *pipeline* pengelolaan kebijakan yang berjalan juga disarankan untuk menilai kelayakan penerapannya secara menyeluruh.

DAFTAR PUSTAKA

- Abu Jabal, A., Bertino, E., Lobo, J., Law, M., Russo, A., Calo, S., dan Verma, D. (2020): Polisma - A Framework for Learning Attribute-Based Access Control Policies (Lecture Notes in Computer Science), 523–544 *dalam* L. Chen, N. Li, K. Liang, dan S. Schneider, ed., *Computer Security – ESORICS 2020*, Springer International Publishing, Cham. https://doi.org/10.1007/978-3-030-58951-6_26.
- Ahsan, M. S., dan Pathan, A.-S. K. (2025) : A Comprehensive Survey on the Requirements, Applications, and Future Challenges for Access Control Models in IoT: The State of the Art, *IoT*, **6**, 9.
- Bamberger, A., dan Fernández, M. (2024) : Automated Generation and Update of Structured ABAC Policies, *Proceedings of the 2024 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems*, 31–40.
- Batra, G., Atluri, V., Vaidya, J., dan Sural, S. (2021) : Incremental Maintenance of ABAC Policies, *Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy*, 185–196.
- Bui, T., Matricia, A., Contreras, E., Mauvais, R., Medina, L., dan Serrano, I. (2025) : ABAC Lab: An Interactive Platform for Attribute-based Access Control Policy Analysis, Tools, and Datasets [Dataset/Tool Paper], *Proceedings of the 30th ACM Symposium on Access Control Models and Technologies*, 111–116.
- Canavese, D., Mannella, L., Regano, L., dan Basile, C. (2024) : Security at the Edge for Resource-Limited IoT Devices, *Sensors*, **24**, 590.
- Chow, C., dan Liu, C. (1968) : Approximating discrete probability distributions with dependence trees, *IEEE Transactions on Information Theory*, **14**, 462–467.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022). *Introduction to algorithms*. Fourth edition. Cambridge, Massachusetts London, England: The MIT Press. 1 p.
- Cotrini Jimenez, C., Weghorn, T., dan Basin, D. (2018) : Mining ABAC Rules from Sparse Logs, 31–46.
- Dauda, A., Flauzac, O., dan Nolot, F. (2024) : A Survey on IoT Application Architectures, *Sensors*, **24**, 5320.
- Díaz-Rodríguez, H., dan Díaz-Pérez, A. (2025) : ABAC Policy Mining Using Complex Network Analysis Techniques, *Applied Sciences*, **15**, 12571.
- Dritsas, E., dan Trigka, M. (2025) : A Survey on Cybersecurity in IoT, *Future Internet*, **17**, 30.

- Gudyś, A., Sikora, M., dan Wróbel, Ł. (2024) : Separate and conquer heuristic allows robust mining of contrast sets in classification, regression, and survival data, *Expert Systems with Applications*, **248**, 123376.
- Guo, Y., Wang, X., Yu, M., Li, F., Pang, Z., dan Fang, L. (2025) : An on-the-fly framework for usable access control policy mining, *Computers & Security*, **150**, 104211.
- Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., dan Scarfone, K. (2014): *Guide to Attribute Based Access Control (ABAC) Definition and Considerations* (NIST SP 800-162), National Institute of Standards dan Technology, diperoleh 11 Mei 2026 melalui situs internet: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-162.pdf>.
- Iyer, P., dan Masoumzadeh, A. (2018) : Mining Positive and Negative Attribute-Based Access Control Policy Rules, *Proceedings of the 23rd ACM on Symposium on Access Control Models and Technologies*, 161–172.
- Jakšić, Z., Devi, S., Jakšić, O., dan Guha, K. (2023) : A Comprehensive Review of Bio-Inspired Optimization Algorithms Including Applications in Microelectronics and Nanophotonics, *Biomimetics*, **8**, 278.
- Khalifi, S., Caraffini, F., dan Iacca, G. (2023) : Metaheuristics in the Balance: A Survey on Memory-Saving Approaches for Platforms with Seriously Limited Resources, *International Journal of Intelligent Systems*, **2023**, 5708085.
- Khosrowshahi, H. N., Aghdasi, H. S., dan Salehpour, P. (2025) : A refined Greylag Goose optimization method for effective IoT service allocation in edge computing systems, *Scientific Reports*, **15**, 15729.
- Larrañaga, P., dan Bielza, C. (2024) : Estimation of Distribution Algorithms in Machine Learning: A Survey, *IEEE Transactions on Evolutionary Computation*, **28**, 1301–1321.
- Lemtenneche, S., Bensayah, A., dan Cheriet, A. (2023) : An Estimation of Distribution Algorithm for Permutation Flow-Shop Scheduling Problem, *Systems*, **11**, 389.
- Li, S., Liu, W., Wu, Y., Wu, X., dan Li, L. (2026) : Attribute based access control of geographic spatial data sharing using blockchain and smart contracts, *Scientific Reports*, **16**, 9132.
- Lim, Y. T., Cheng, P. C., Clark, J. A., dan Rohatgi, P. (2008): Policy Evolution with Grammatical Evolution, 71–80 *dalam* X. Li, M. Kirley, M. Zhang, D. Green, V. Ciesielski, H. Abbass, Z. Michalewicz, T. Hendtlass, K. Deb, K. C. Tan, J. Branke, dan Y. Shi, ed., *Simulated Evolution and Learning*, Springer Berlin Heidelberg, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-89694-4_8.

- Lim, Y. T., Cheng, P.-C., Rohatgi, P., dan Clark, J. A. (2009) : Dynamic security policy learning, *Proceedings of the first ACM workshop on Information security governance*, 39–48.
- Lin, J. (1991) : Divergence Measures Based on the Shannon Entropy, *IEEE Transactions on Information Theory*, **37**, 145–151.
- Medvet, E., Bartoli, A., Carminati, B., dan Ferrari, E. (2015): Evolutionary Inference of Attribute-Based Access Control Policies, 351–365 *dalam* A. Gaspar-Cunha, C. Henggeler Antunes, dan C. C. Coello, ed., *Evolutionary Multi-Criterion Optimization*, Springer International Publishing, Cham. https://doi.org/10.1007/978-3-319-15934-8_24.
- Molloy, M., dan Reed, B. (1995) : A critical point for random graphs with a given degree sequence, *Random Structures & Algorithms*, **6**, 161–180.
- Omran, M. G. H., Clerc, M., Ghaddar, F., Aldabagh, A., dan Tawfik, O. (2022) : Permutation Tests for Metaheuristic Algorithms, *Mathematics*, **10**, 2219.
- Pan, J.-S., Hu, P., Snášel, V., dan Chu, S.-C. (2023) : A survey on binary metaheuristic algorithms and their engineering applications, *Artificial Intelligence Review*, **56**, 6101–6167.
- Peffer, K., Tuunanen, T., Rothenberger, M. A., dan Chatterjee, S. (2007) : A Design Science Research Methodology for Information Systems Research, *Journal of Management Information Systems*, **24**, 45–77.
- Pelikan, M., Sastry, K., dan Goldberg, D. E. (2008) : iBOA: the incremental bayesian optimization algorithm, *Proceedings of the 10th annual conference on Genetic and evolutionary computation*, 455–462.
- Peres, F. F. (2026) : Effect sizes for nonparametric tests, *Biochemia medica*, **36**, 5–16.
- Perez-Haro, A., dan Diaz-Perez, A. (2024) : ABAC Policy Mining through Affiliation Networks and Biclique Analysis, *Information*, **15**, 45.
- Quan, S., Zhao, Y., dan Helil, N. (2023) : Attribute-Based Access Control Policy Generation Approach from Access Logs Based on the CatBoost, *Computing and Informatics*, **42**, 615–650.
- Ragothaman, K., Wang, Y., Rimal, B., dan Lawrence, M. (2023) : Access Control for IoT: A Survey of Existing Research, Dynamic Policies and Future Directions, *Sensors*, **23**, 1805.
- Rajwar, K., Deep, K., dan Das, S. (2023) : An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges, *Artificial Intelligence Review*, **56**, 13187–13257.

- Raspberry Pi Ltd (2026): *Raspberry Pi 5 Product Brief* (Raspberry Pi Technical Documentation RP-008348-DS-6), Raspberry Pi Ltd, Cambridge, UK, diperoleh 7 Agustus 2026 melalui situs internet: <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>.
- Shang, S., Wang, X., dan Liu, A. (2024) : ABAC policy mining method based on hierarchical clustering and relationship extraction, *Computers & Security*, **139**, 103717.
- Siyuan, S., Aodi, L., Xuehui, D., Xiaohan, W., dan Ming, T. (2025) : ABAC policy mining method for heterogeneous access control system, *The Journal of Supercomputing*, **81**, 1065.
- Soloviev, V. P., Larrañaga, P., dan Bielza, C. (2024) : EDAspy: An extensible python package for estimation of distribution algorithms, *Neurocomputing*, **598**, 128043.
- Xu, Z., dan Stoller, S. D. (2015) : Mining Attribute-Based Access Control Policies, *IEEE Transactions on Dependable and Secure Computing*, **12**, 533–545.
- Yang, W., Su, R., Cheng, Y., dan Guo, J. (2022) : A Concept Drift Detection Approach Based on Jensen–Shannon Divergence for Network Traffic Classification, *Proceedings of the 2022 5th International Conference on Artificial Intelligence and Pattern Recognition*, 982–987.

LAMPIRAN

Lampiran A Contoh Kebijakan Hasil Mining Lengkap

Lampiran ini menampilkan kebijakan ABAC lengkap yang dihasilkan oleh proses penambangan (*policy mining*), sebagai bukti keluaran konkret yang ada pada Bab IV hanya disajikan cuplikannya. Semantik pencocokan mengikuti bahasa kebijakan ABAC Lab yang menyebutkan bahwa beberapa nilai pada atribut yang sama bersifat disjungtif dan ditulis sebagai “atribut $\in \{\text{nilai}_1, \text{nilai}_2, \dots\}$ ” (operator keanggotaan “in”), sedangkan antar atribut bersifat konjungtif (DAN). Untuk keterbacaan, himpunan nilai yang sangat panjang pada atribut berpengenal (mis. *uname/rname*) dipangkas setelah 25 nilai dengan catatan jumlah sisanya; jumlah syarat atribut penuh tetap tercermin pada nilai WSC di baris cakupan. TP dan FP menyatakan banyaknya permintaan izin yang benar tercakup dan permintaan tolak yang keliru diizinkan oleh aturan tersebut.

A.1 CoDA-EDA pada Dataset *Workforce* (D = 436)

Sumber data: ABAC Lab / *workforce* (*corr* = sedang, *drift* = stasioner)

Ukuran log: $n = 10000$ permintaan, $D = 436$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,885$; $\text{precision} = 0,862$; $\text{recall} = 0,910$; $\text{coverage} = 0,317$; WSC = 20; jumlah aturan = 5

- **Aturan 1:** JIKA $\text{provider} \in \{\text{eWorkforce}, \text{externalWorkforceSupplier}\}$ DAN $\text{position} \in \{\text{workforceManager}, \text{technician}\}$ DAN $\text{action} = \text{view}$ MAKA izinkan (permit)
Cakupan: TP=1651 FP=113 WSC=4
- **Aturan 2:** JIKA $\text{provider} = \text{eWorkforce}$ DAN $\text{assignedTenant} = \text{powerProtection}$ DAN $\text{associatedContract} = \text{contract006}$ DAN $\text{action} = \text{complete}$ MAKA izinkan (permit)
Cakupan: TP=28 FP=36 WSC=3
- **Aturan 3:** JIKA $\text{provider} \in \{\text{externalHelpdeskSupplier}, \text{telco}\}$ DAN $\text{tenant} = \text{telco}$ DAN $\text{tenantType} = \text{primary}$ DAN $\text{associatedContract} \in \{\text{contract001}, \text{contract003}, \text{contract005}\}$ DAN $\text{isAppointment} = \text{False}$ DAN $\text{action} = \text{modify}$ MAKA izinkan (permit)
Cakupan: TP=402 FP=3 WSC=8
- **Aturan 4:** JIKA $\text{contractStatus} = \text{active}$ DAN $\text{action} = \text{view}$ MAKA

izinkan (permit)

Cakupan: TP=1291 FP=339 WSC=1

- **Aturan 5:** JIKA `provider = externalHelpdeskSupplier` DAN `tenant = powerProtection` DAN `recurrence ∈ {True,False}` DAN `action = modify` MAKA izinkan (permit)

Cakupan: TP=82 FP=2 WSC=4

A.2 ACO pada Dataset *Workforce* (Pembanding)

Sumber data: ABAC Lab / *workforce* (*corr* = sedang, *drift* = stasioner)

Ukuran log: $n = 10000$ permintaan, $D = 436$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,820$; $precision = 0,697$; $recall = 0,995$; $coverage = 0,428$; WSC = 19; jumlah aturan = 5

- **Aturan 1:** JIKA `tenantType = primary` DAN `action ∈ {modify,view}` MAKA izinkan (permit)

Cakupan: TP=2879 FP=1297 WSC=1

- **Aturan 2:** JIKA `provider = eWorkforce` DAN `position = helpdeskOperator` DAN `action = delete` MAKA izinkan (permit)

Cakupan: TP=57 FP=0 WSC=2

- **Aturan 3:** JIKA `assignedRegion = north` DAN `certifications = powerProtectionSpecialist` DAN `action = complete` MAKA izinkan (permit)

Cakupan: TP=19 FP=0 WSC=2

- **Aturan 4:** JIKA `managedStaff ∈ {tech009,tech011,tech014,tech025,tech036,tech046,tech060,tech062,tech065}` DAN `action = complete` MAKA izinkan (permit)

Cakupan: TP=21 FP=2 WSC=9

- **Aturan 5:** JIKA `department = workforce` DAN `assignedRegion = north` DAN `assignedTechnician ∈ {tech005,tech014,tech046}` DAN `action = complete` MAKA izinkan (permit)

Cakupan: TP=12 FP=0 WSC=5

A.3 CoDA-EDA pada Dataset *Smart Building IoT* (D = 428)

Sumber data: *Smart Building IoT*

Ukuran log: $n = 8000$ permintaan, $D = 428$ pasangan atribut-nilai

Metrik kebijakan: $F1 = 0,998$; $precision = 1,000$; $recall = 0,996$; $coverage = 0,747$;

WSC = 276; jumlah aturan = 5

- **Aturan 1:** JIKA `area` $\in \{\text{backyard, bedroom}\}$ DAN `action` = `arm` MAKA izinkan (permit)
Cakupan: TP=1770 FP=0 WSC=2
- **Aturan 2:** JIKA `uname` $\in \{116, 104, 110, 108, 118, 119, 107, 84, 77, 114, 38, 97, 117, 109, 87, 106, 54, 88, 115, 99, 93, 36, 2, 113, 111\}$... (+34 nilai lain) DAN `action` $\in \{\text{arm, access, control}\}$ MAKA izinkan (permit)
Cakupan: TP=4111 FP=0 WSC=59
- **Aturan 3:** JIKA `rname` $\in \{1100, 1109, 1068, 1106, _\text{rare}_, 1091, 1108, 629, 1067, 1052, 1105, 254, 961, 1097, 1038, 1031, 1090, 1098, 1102, 1069, 1041, 1033, 1008, 1107, 1077\}$... (+87 nilai lain) DAN `action` $\in \{\text{arm, access, control}\}$ MAKA izinkan (permit)
Cakupan: TP=4794 FP=0 WSC=112
- **Aturan 4:** JIKA `rname` $\in \{1100, 1092, 1065, 1043, 1097, 1096, 1032, 1073, 711, 1078, 1060, 1087, 1041, 1051, 1008, 1071, 843, 1057, 837, 980, 899, 849, 1076, 1086, 1055\}$... (+75 nilai lain) DAN `action` $\in \{\text{access, control}\}$ MAKA izinkan (permit)
Cakupan: TP=854 FP=0 WSC=100
- **Aturan 5:** JIKA `type` $\in \{\text{TV, Security Cameras, Smart locks}\}$ DAN `action` $\in \{\text{arm, control}\}$ MAKA izinkan (permit)
Cakupan: TP=3417 FP=0 WSC=3

Lampiran B Tabel Hasil Eksperimen Lengkap

Lampiran ini memuat rincian numerik hasil pengujian keempat hipotesis yang ada pada Bab IV disajikan dalam bentuk ringkas. Seluruh angka dihitung dari keluaran eksekusi di Raspberry Pi 5.

B.1 Kualitas Kebijakan (H1) – Rerata Per Dataset dan Tingkat Korelasi

Tabel B.1 F-score, cakupan, dan WSC pada workforce (D=436)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,860	0,890	0,918	0,337	33,3
compact- GA	0,872	0,902	0,925	0,325	41,7
compact- PSO	0,864	0,897	0,918	0,320	51,7
iBOA	0,862	0,883	0,919	0,341	29,2
BOA	0,823	0,831	0,840	0,418	9,0
ACO	0,861	0,862	0,887	0,388	13,9
UBK	0,837	0,861	0,860	0,376	6,4

Tabel B.2 F-score, cakupan, dan WSC pada e-document (D=1.907)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,453	0,442	0,459	0,248	11,8
compact- GA	0,355	0,277	0,356	0,156	10,2
compact- PSO	0,220	0,287	0,273	0,116	11,0
iBOA	0,413	0,460	0,483	0,246	13,8
BOA	0,605	0,643	0,652	0,624	11,2
ACO	0,648	0,663	0,671	0,572	16,9

Tabel B.2 F-score, cakupan, dan WSC pada e-document (D=1.907) (lanjutan)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
UBK	0,627	0,617	0,647	0,468	6,2

Tabel B.3 F-score, cakupan, dan WSC pada healthcare (D=46)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,731	0,802	0,804	0,479	2,0
compact- GA	0,747	0,771	0,778	0,490	2,1
compact- PSO	0,747	0,818	0,838	0,459	3,0
iBOA	0,760	0,823	0,824	0,455	2,5
BOA	0,792	0,833	0,881	0,433	2,6
ACO	0,731	0,774	0,778	0,494	1,4
UBK	0,739	0,793	0,812	0,478	2,0

Tabel B.4 F-score, cakupan, dan WSC pada project-management (D=51)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA- EDA	0,852	0,918	0,923	0,372	7,0
compact- GA	0,832	0,900	0,918	0,384	6,4
compact- PSO	0,854	0,920	0,930	0,370	6,7
iBOA	0,857	0,922	0,947	0,366	6,7
BOA	0,918	0,927	0,935	0,349	6,2
ACO	0,834	0,901	0,924	0,382	5,5
UBK	0,815	0,855	0,889	0,404	7,7

Tabel B.5 F-score, cakupan, dan WSC pada university (D=54)

Algoritma	F1 rendah	F1 sedang	F1 tinggi	Cakupan	WSC
CoDA-EDA	0,824	0,834	0,832	0,425	6,6
compact-GA	0,786	0,833	0,848	0,432	6,4
compact-PSO	0,803	0,829	0,854	0,426	9,7
iBOA	0,779	0,829	0,863	0,431	7,6
BOA	0,809	0,826	0,856	0,424	6,9
ACO	0,791	0,832	0,884	0,420	7,9
UBK	0,860	0,870	0,876	0,385	6,6

B.2 Jejak Memori (H2) – Rasio Penskalaan

$\text{alloc}(NP = 400)/\text{alloc}(NP = 25)$ Per Dataset

Tabel B.6 Rasio penskalaan jejak memori terhadap ukuran populasi (per dataset dan rata-rata)

Algoritma	workforce	e-document	healthcare	project-mgmt	university	Rata-rata
CoDA-EDA	1,00	1,00	1,00	1,00	1,00	1,00x
compact-GA	0,47	0,36	1,00	1,00	1,00	0,77x
compact-PSO	0,48	0,39	1,00	1,00	0,95	0,76x
iBOA	0,53	0,58	1,00	1,00	1,00	0,82x
BOA	12,09	13,80	3,94	4,21	4,37	7,68x
ACO	9,47	13,24	2,63	2,82	2,94	6,22x
UBK	12,08	15,64	5,64	6,17	6,49	9,20x

Catatan: rasio mendekati 1,00 menandakan memori datar terhadap ukuran populasi. Nilai per-dataset pada algoritma berpopulasi lebih tinggi pada dua dataset besar karena skala populasi absolutnya lebih besar.

B.3 Efisiensi Pembaruan (H3 dan H4) – Waktu Per Siklus dan Kualitas

Tabel B.7 Waktu total pembaruan (warm/always/cold) dan F-score akhir per dataset dan skenario drift

Dataset / skenario	Relearn (dari 4)	Waktu pembaruan			F-score akhir	
		warm (s)	always (s)	cold (s)	F1 warm	F1 cold
workforce / stasioner	0,0	3,46	4,58	9,05	0,901	0,805
workforce / rendah	4,0	4,50	4,49	9,12	0,891	0,812
workforce / sedang	4,0	4,52	4,53	8,95	0,877	0,816
edocument / stasioner	4,0	22,26	22,27	28,24	0,399	0,297
edocument / rendah	4,0	22,38	22,39	28,36	0,465	0,341
edocument / sedang	4,0	22,37	22,37	28,31	0,433	0,267
healthcare / stasioner	0,0	0,00	0,00	2,00	0,814	0,807
healthcare / rendah	3,3	0,00	0,00	2,81	0,789	0,838
healthcare / sedang	4,0	0,00	0,00	2,67	0,832	0,890
project-management / stasioner	0,0	0,17	0,21	4,45	0,909	0,928
project-management / rendah	3,0	0,16	0,20	3,98	0,928	0,896
project-management / sedang	3,3	0,20	0,24	4,13	0,936	0,928

Tabel B.7 Waktu total pembaruan (warm/always/cold) dan F-score akhir per dataset dan skenario drift (lanjutan)

Dataset / skenario	Relearn (dari 4)	Waktu pembaruan			F-score akhir	
		warm (s)	always (s)	cold (s)	F1 warm	F1 cold
university / stasioner	0,0	0,08	0,11	3,76	0,810	0,803
university / rendah	4,0	0,07	0,07	3,88	0,808	0,819
university / sedang	3,7	0,08	0,08	3,99	0,798	0,850

H_3 (*warm* vs *always*) tampak ketika $Relearn < 4$ (pemicu drift diam); H_4 (*warm* vs *cold*) tampak dari waktu *warm* yang jauh lebih kecil dari *cold* dengan $F1_{warm} \geq F1_{cold}$.

B.4 Validasi Eksternal *Smart Building IoT* (H1-H4)

Tabel B.8 Tabel Kualitas pada Smart Building IoT (D=428; garis sepele F1=0,858; 5 seed)

Algoritma	F-score	Presisi	Cakupan	Memori (KB)	>sepele
BOA	0,998	1,000	0,747	960	5/5
ACO	0,994	1,000	0,742	513	5/5
compact-PSO	0,964	1,000	0,704	989	4/5
compact-GA	0,961	1,000	0,697	729	5/5
iBOA	0,943	1,000	0,674	505	4/5
CoDA-EDA	0,938	1,000	0,665	949	5/5
UBK	0,834	1,000	0,539	1122	2/5

Tabel B.9 Siklus pembaruan pada Smart Building IoT (total waktu, F1 akhir, jumlah relearn)

Mode jendela	Algoritma	Kondisi	Total waktu (s)	F1 akhir	Relearn
acak (sepadan)	CoDA-EDA	always	nan	0,973	5,0
acak (sepadan)	CoDA-EDA	cold	nan	0,914	5,0
acak (sepadan)	CoDA-EDA	warm	nan	0,977	1,0
acak (sepadan)	iBOA	cold	nan	0,934	5,0
attr (drift)	CoDA-EDA	always	nan	0,958	5,0
attr (drift)	CoDA-EDA	cold	nan	0,839	5,0
attr (drift)	CoDA-EDA	warm	nan	0,958	5,0
attr (drift)	iBOA	cold	nan	0,952	5,0

Lampiran C Hasil Uji Statistik Lengkap

Lampiran ini memuat hasil uji statistik non-parametrik yang mendasari vonis Bab IV. Untuk perbandingan banyak algoritma digunakan uji Friedman (omnibus) diikuti post-hoc Wilcoxon signed-rank antara CoDA-EDA dan tiap *baseline* dengan koreksi Holm-Bonferroni (p_{adj}). Ukuran efek dilaporkan sebagai korelasi rank-biserial (r), positif bila menguntungkan CoDA-EDA. Taraf signifikansi $\alpha = 0,05$. $\Delta F1$ adalah selisih rerata F-score CoDA-EDA terhadap *baseline*.

C.1 Uji H1 (Kualitas Kebijakan) Per Dataset

Tabel C.1 Post-hoc CoDA-EDA vs baseline pada workforce (D=436) (Friedman $\chi^2 = 50,5$; $p = 3,8 \times 10^{-9}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	-0,011	0,306	-0,29	setara
compact-PSO	-0,004	1,000	-0,11	setara
iBOA	+0,001	1,000	-0,16	setara
BOA	+0,058	$8,6 \times 10^{-5}$	+0,56	CoDA-EDA unggul
ACO	+0,019	0,266	+0,24	setara
UBK	+0,036	0,011	+0,33	CoDA-EDA unggul

Tabel C.2 Post-hoc CoDA-EDA vs baseline pada e-document (D=1.907) (Friedman $\chi^2 = 186,8$; $p = 1,2 \times 10^{-37}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,122	0,003	+0,42	CoDA-EDA unggul
compact-PSO	+0,192	$2,2 \times 10^{-7}$	+0,60	CoDA-EDA unggul
iBOA	-0,001	0,763	-0,16	setara

Tabel C.2 Post-hoc CoDA-EDA vs baseline pada e-document (D=1.907) (lanjutan)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
BOA	-0,181	$9,5 \times 10^{-8}$	-0,87	CoDA-EDA kalah
ACO	-0,209	$7,1 \times 10^{-12}$	-0,91	CoDA-EDA kalah
UBK	-0,178	$6,5 \times 10^{-12}$	-0,91	CoDA-EDA kalah

Tabel C.3 Post-hoc CoDA-EDA vs baseline pada healthcare (D=46) (Friedman $\chi^2 = 21,6$; $p = 0,001$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,014	1,000	+0,00	setara
compact-PSO	-0,022	0,679	-0,41	setara
iBOA	-0,023	1,000	-0,45	setara
BOA	-0,056	0,122	-0,56	setara
ACO	+0,018	1,000	+0,43	setara
UBK	-0,002	1,000	+0,11	setara

Tabel C.4 Post-hoc CoDA-EDA vs baseline pada project-management (D=51) (Friedman $\chi^2 = 47,6$; $p = 1,4 \times 10^{-8}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,014	0,241	+0,41	setara
compact-PSO	-0,004	1,000	+0,06	setara
iBOA	-0,011	1,000	-0,03	setara
BOA	-0,029	0,143	-0,24	setara
ACO	+0,011	1,000	+0,10	setara
UBK	+0,045	0,002	+0,49	CoDA-EDA unggul

Tabel C.5 Post-hoc CoDA-EDA vs baseline pada university (D=54) (Friedman $\chi^2 = 41,5$; $p = 2,3 \times 10^{-7}$; $n = 45$ blok)

Pembanding	$\Delta F1$	p_{adj} (Holm)	r (rank-biserial)	Vonis
compact-GA	+0,007	1,000	+0,11	setara
compact-PSO	+0,001	1,000	+0,21	setara
iBOA	+0,006	1,000	+0,05	setara
BOA	-0,001	1,000	-0,09	setara
ACO	-0,005	1,000	-0,12	setara
UBK	-0,039	$2,3 \times 10^{-4}$	-0,73	CoDA-EDA kalah

C.2 Uji H3 dan H4 (Efisiensi Pembaruan) Per Dataset dan Skenario

Tabel C.6 Uji Wilcoxon berpasangan waktu pembaruan — H3 (warm vs always) dan H4 (warm vs cold)

Dataset / skenario	H3: p (warm vs always)	r	H4: p (warm vs cold)	r
workforce / stasioner	$6,1 \times 10^{-5}$	+1,00	$6,1 \times 10^{-5}$	+1,00
workforce / rendah	0,389	+0,20	$6,1 \times 10^{-5}$	+1,00
workforce / sedang	0,053	+0,60	$6,1 \times 10^{-5}$	+1,00
edocument / stasioner	0,489	-0,07	$6,1 \times 10^{-5}$	+1,00
edocument / rendah	0,389	+0,33	$6,1 \times 10^{-5}$	+1,00
edocument / sedang	0,762	-0,07	$6,1 \times 10^{-5}$	+1,00
healthcare / stasioner	—	—	$6,5 \times 10^{-4}$	+1,00
healthcare / rendah	—	—	$6,1 \times 10^{-5}$	+1,00
healthcare / sedang	—	—	$6,1 \times 10^{-5}$	+1,00
project-management / stasioner	0,715	+0,00	$6,1 \times 10^{-5}$	+1,00
project-management / rendah	0,068	+1,00	$6,1 \times 10^{-5}$	+1,00
project-management / sedang	0,144	+0,50	$6,1 \times 10^{-5}$	+1,00
university / stasioner	0,225	+0,60	$6,1 \times 10^{-5}$	+1,00
university / rendah	0,285	-0,33	$6,1 \times 10^{-5}$	+1,00

Tabel C.6 Uji Wilcoxon berpasangan waktu pembaruan — H3 (warm vs always) dan H4 (warm vs cold) (lanjutan)

Dataset / skenario	H3: p (warm vs always)	r	H4: p (warm vs cold)	r
university / sedang	0,655	+0,00	$6,1 \times 10^{-5}$	+1,00

Keterangan: $p = \text{“—”}$ berarti kedua kondisi identik (mis. warm = always ketika pemicu drift menyala di setiap jendela) atau jumlah pasangan terlalu kecil untuk uji. H3 didukung pada baris ber- p signifikan dengan r positif; H4 konsisten didukung (warm lebih cepat dari cold) di seluruh dataset.

C.3 Uji Validasi Eksternal *Smart Building IoT* (H1)

Tabel C.7 Post-hoc CoDA-EDA vs baseline pada Smart Building IoT (Friedman $\chi^2 = 21,4$; $p = 0,002$; $n = 5$ seed)

Pembandingan	$\Delta F1$	p (Wilcoxon)	r (rank-biserial)
compact-GA	-0,023	0,438	-0,60
compact-PSO	-0,026	0,625	-0,60
iBOA	-0,005	1,000	-0,20
BOA	-0,057	0,062	-1,00
ACO	-0,056	0,062	-1,00
UBK	+0,104	0,062	+1,00

Catatan: Dengan hanya lima seed, nilai p terkecil yang mungkin pada uji Wilcoxon dua sisi adalah $2/2^5 = 0,0625$, sehingga taraf 0,05 tidak dapat dicapai berapa pun besar perbedaannya. Hasil eksternal karena itu ditafsirkan secara deskriptif (arah r dan posisi relatif), bukan sebagai vonis signifikansi; uji Friedman omnibus tetap signifikan ($p = 0,002$).

Lampiran D Analisis Karakteristik Dataset (Pra-Eksperimen)

Lampiran ini memuat profil karakteristik seluruh dataset yang dijalankan sebelum eksperimen utama, sebagai dasar penentuan kecocokan tiap dataset dengan tiap hipotesis. Kolom skala menyatakan jumlah *user/resource/aksi*; *permit* unik menandai risiko duplikasi (makin sedikit makin berisiko); rantai derau JSD stasioner menentukan kelayakan mekanisme H3 (layak bila di bawah ambang $\tau = 0,01$).

Tabel D.1 Profil karakteristik dataset

Dataset	Skala (U/R/A)	D	Permit unik	Duplikasi	Rasio permit	Lantai derau JSD	H3
workforce	353/250/9	436	14.513	rendah	0,30	0,0066	layak
e-document	500/300/4	1907	32.961	rendah	0,30	0,0325	tak layak
healthcare	21/16/3	46	14	TINGGI	0,30	0,0022	layak
project-management	19/40/4	51	17	TINGGI	0,30	0,0010	layak
university	22/34/9	54	90	TINGGI	0,30	0,0026	layak
Smart Building IoT	—/—/4	428	5.729	rendah	0,75	0,0065	layak

Tabel D.2 Matriks kecocokan dataset $\times$ hipotesis

Dataset	H1 (kualitas)	H2 (memori)	H3 (pembaruan)	H4 (warm-start)
workforce	Baik	Baik	Layak ($JSD < \tau$)	Baik
e-document	Baik	Baik	Tak layak ($JSD > \tau$)	Baik

Tabel D.2 Matriks kecocokan dataset $\times$ hipotesis (lanjutan)

Dataset	H1 (kualitas)	H2 (memori)	H3 (pembaruan)	H4 (warm-start)
healthcare	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
project-management	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
university	Terbatas (duplikasi)	Baik	Layak ($JSD < \tau$)	Baik
Smart Building IoT	Kontras garis-sepele	Baik	Layak ($JSD < \tau$)	Baik

Ringkasan implikasi: hanya *workforce* dan *e-document* yang layak menjadi tumpuan vonis H_1 (permit unik memadai); ketiga dataset kecil terbatas untuk H_1 karena duplikasi tinggi namun tetap sah untuk $H_2/H_3/H_4$; *e-document* tidak layak untuk H_3 karena rantai derau melampaui τ , sedangkan *Smart Building IoT* justru layak — menjadi sarana menunjukkan mekanisme H_3 yang tidak dapat diperlihatkan pada *e-document*.

Lampiran E Konfigurasi dan Kalibrasi Algoritma

Lampiran ini merinci parameter tiap algoritma, anggaran evaluasi, nilai ambang dan bobot fungsi fitness, serta ringkasan uji kalibrasi (monotonisitas), untuk menjamin keterulangan eksperimen. Seluruh algoritma dijalankan pada kerangka *separate-and-conquer* bersama dan fungsi fitness yang identik:

$$f(x) = \frac{TP^2}{TP + w_{fp} \cdot FP} - w_{wsc} \cdot WSC,$$

dengan $w_{fp} = 1, 0$ dan $w_{wsc} = 0, 1$.

E.1 Parameter Tiap Algoritma

Tabel E.1 Parameter kunci CoDA-EDA dan enam baseline

Algoritma	Parameter kunci (nilai)	Catatan
CoDA-EDA	$NP = 100; p_0 = 3/D;$ $mi_rows = 1500; w_{fp} = 1, 0;$ $w_{wsc} = 0, 1$	Pohon Chow-Liu order-1 disusun via algoritma Prim; struktur hanya dipicu drift
compact-GA	$NP = 100; p_0 = 3/D;$ laju $\pm 1/NP$	Univariat; representasi biner identik dengan CoDA-EDA
compact-PSO	$NP = 100; w = 0, 7;$ $c_1 = c_2 = 1, 5; p_0 = 3/D$	Posisi-kecepatan diadaptasi ke ruang kebijakan diskret
BOA	$NP = 100; cand = 8;$ $top_vars = 400$	Jaringan Bayes via BIC; kandidat induk dibatasi 400 variabel paling informatif agar layak pada D besar
iBOA	$NP = 100; k_{order} = 2$	Pohon dependensi inkremental (Chow-Liu); pembanding langsung H3
ACO	$NP = 100$ (semut); $\rho = 0, 1$ (evaporasi feromon)	Parameter mengikuti nilai baku publikasi asli
UBK	$NP = 100; \alpha$ adaptif $= 0, 8 \cdot \max(0, 05; 1 - t/200);$ $derau = 0, 3$	Strategi berburu berbasis populasi

Prior kejarangan $p_0 = 3/D$ bersifat sadar-dimensi (aturan ABAC nyata umumnya 2–5 pasangan atribut-nilai), sehingga jumlah bit aktif di awal pencarian tetap konstan meskipun D berbeda antar-dataset.

E.2 Anggaran Evaluasi

Kesetaraan perbandingan dijaga dengan menyetarakan anggaran evaluasi fungsi fitness (bukan jumlah iterasi) antar-algoritma, dihitung untuk keseluruhan proses *separate-and-conquer*. Pada eksperimen kualitas (H_1) semua-dataset, anggaran ditetapkan seragam: maksimum 20.000 evaluasi total, 4.000 evaluasi per aturan, dan paling banyak 12 aturan; pencarian berhenti lebih awal ketika seluruh permit telah tercakup atau tidak ada aturan yang menambah cakupan. Akibatnya, dataset berdimensi besar (*workforce*, *e-document*) memakai anggaran mendekati penuh, sedangkan dataset kecil berhenti lebih awal karena permit uniknya sedikit.

Tabel E.2 Anggaran evaluasi terpakai per dataset (eksperimen H1)

Dataset	D	Evals terpakai (min–maks)	Rata-rata	Jumlah aturan
workforce	436	16.400–20.100	20.003	4–5
e-document	1907	16.400–20.100	20.003	4–5
healthcare	46	4.001–12.300	4.656	1–3
project- management	51	4.001–20.001	9.328	1–5
university	54	4.001–20.100	11.413	1–5

E.3 Parameter Siklus Pembaruan (H3 dan H4)

Tabel E.3 Parameter simulasi siklus pembaruan lintas jendela

Parameter	Nilai	Keterangan
Ambang pemicu drift (τ)	0, 01	Struktur Chow-Liu dipelajari ulang bila divergensi Jensen-Shannon $> \tau$

Tabel E.3 Parameter simulasi siklus pembaruan lintas jendela (lanjutan)

Parameter	Nilai	Keterangan
Jumlah jendela	5	Log dibagi menjadi lima jendela berurutan (empat siklus pembaruan)
Anggaran pembelajaran awal	9.000 evaluasi	Jendela pertama / kondisi cold
Anggaran per pembaruan	3.000 evaluasi	Tiap siklus pembaruan (warm/always)
Anggaran per aturan	3.000 evaluasi	Batas per aturan pada siklus pembaruan
Ukuran populasi virtual (NP)	100	Sama seperti eksperimen H1

E.4 Ringkasan Uji Monotonisitas (Kalibrasi)

Uji monotonisitas dipakai untuk memverifikasi bahwa *baseline* diimplementasikan secara sah (kualitas tidak menurun ketika anggaran dinaikkan) sekaligus mengidentifikasi tuas hiperparameter yang benar-benar menggerakkan cakupan. Dua tuas diuji seragam pada seluruh algoritma.

Tabel E.4 Efek tuas hiperparameter terhadap kualitas (dataset e-document)

Tuas	Nilai diuji	Efek terhadap CoDA-EDA	Kesimpulan
Bobot false positive (w_{fp})	1, 0/2, 0/4, 0	Cakupan $0,186 \rightarrow 0,187$ (praktis datar)	Bukan tuas cakupan; ditetapkan $w_{fp} = 1,0$
Anggaran jumlah aturan	4/8/12	F-score $0,362 \rightarrow 0,571 \rightarrow 0,568$ (monoton naik)	Tuas cakupan sesungguhnya; jumlah aturan diperlakukan sebagai parameter rekayasa

Karena kenaikan anggaran aturan menaikkan F-score secara monoton (tanpa penurunan), implementasi kerangka penambangan dinyatakan sah. Nilai baku eksperimen

ditetapkan $w_{fp} = 1, 0$; jumlah aturan efektif pada anggaran baku berkisar 4–5.

Lampiran F Rincian Konstruksi Dataset

Dataset penelitian dikonstruksi dengan memperluas generator log sintetis *ABAC Lab*: korelasi antar-atribut disuntikkan hingga mencapai target *normalized mutual information (NMI)*, sedangkan pergeseran (*drift*) antar-jendela disuntikkan hingga mencapai target *Jensen-Shannon divergence (JSD)*. Karena estimator *NMI* dan *JSD* memiliki bias sampel-hingga yang membesar seiring rasio sel/sampel, tingkat *drift* dilaporkan sebagai nilai bersih (divergensi terukur dikurangi lantai derau kondisi stasioner). Lampiran ini merinci parameter konstruksi, struktur manifest, dan laporan validasi.

F.1 Parameter Konstruksi

Tabel F.1 Parameter injeksi korelasi, drift, dan pembangkitan log

Parameter	Nilai
Tingkat korelasi – target <i>NMI</i>	rendah = 0,05; sedang = 0,35; tinggi = 0,65
Skenario drift – target <i>JSD</i> (bersih)	stasioner = 0,00; rendah = 0,10; sedang = 0,25
Entri per jendela	2.000
Jumlah jendela	5 (empat transisi/siklus pembaruan)
Rasio permit	0,30 (proporsi keputusan izin pada log)
Seed dataset	0, 1, 2
Total konfigurasi	$5 \text{ dataset} \times 3 \text{ korelasi} \times 3 \text{ drift} \times 3 \text{ seed} = 135$

Kalibrasi korelasi bersifat *closed-loop*: parameter penggandeng distribusi bersama dicari melalui pencarian biner hingga *NMI* yang terukur pada log final (bukan pada distribusi teoretis) mendekati target, dengan ukuran log kalibrasi disamakan dengan ukuran log final untuk menekan bias sampel-hingga.

F.2 Struktur Manifest

Tabel F.2 Medan (field) manifest per konfigurasi dataset

Medan	Keterangan
tag	Pengenal konfigurasi, mis. workforce_corr-sedang_drift-rendah_seed-0
dataset, corr, drift, seed	Identitas dataset dasar, tingkat korelasi, skenario drift, dan seed
uattr, rattr	Pasangan atribut subjek dan objek yang dipilih untuk injeksi korelasi
alpha	Parameter penggandeng distribusi bersama hasil kalibrasi biner
target_nmi, observed_nmi	Target <i>NMI</i> dan <i>NMI</i> yang benar-benar terukur pada log final
target_jsd, observed_jsd	Target <i>JSD</i> dan <i>JSD</i> terukur per jendela (untuk perhitungan drift bersih)
permit_ratio duplication	Rasio permit teramati pada log
_rate	Tingkat duplikasi entri (indikator keragaman efektif data)

F.3 Laporan Validasi — Kalibrasi Korelasi

Tabel F.3 NMI teramati per dataset dan tingkat korelasi (rerata lintas drift dan seed; target dalam kurung)

Dataset	<i>NMI</i> @ rendah (0,05)	<i>NMI</i> @ sedang (0,35)	<i>NMI</i> @ tinggi (0,65)
workforce	0,050 (0,05)	0,351 (0,35)	0,638 (0,65)
e-document	0,057 (0,05)	0,348 (0,35)	0,653 (0,65)
healthcare	0,048 (0,05)	0,354 (0,35)	0,477 (0,65)
project-management	0,057 (0,05)	0,349 (0,35)	0,652 (0,65)
university	0,044 (0,05)	0,335 (0,35)	0,653 (0,65)

NMI teramati mendekati target di seluruh dataset, menegaskan mekanisme injeksi korelasi bekerja sebagaimana dirancang; simpangan kalibrasi terbesar pada satu konfigurasi sebesar 0,34 (akibat bias sampel-hingga pada rasio sel/sampel yang tinggi), sedangkan rerata per dataset tetap dekat target.

F.4 Laporan Validasi — Lantai Derau, Drift Bersih, dan Duplikasi

Tabel F.4 Lantai derau JSD, drift bersih terkalibrasi, tingkat duplikasi, dan status validasi per dataset

Dataset	Lantai derau JSD	Drift bersih ($\rightarrow 0, 10$)	Drift bersih ($\rightarrow 0, 25$)	Duplikasi	Status (OK/total)
workforce	0,017	0,094	0,236	16–57%	11/27 OK
e-document	0,132	0,091	0,228	5–37%	24/27 OK
healthcare	0,003	0,102	0,172	95–99%	0/27 OK
project-management	0,001	0,099	0,175	92–94%	0/27 OK
university	0,009	0,089	0,181	68–88%	0/27 OK

Dua pola menegaskan temuan pra-eksperimen. Pertama, lantai derau JSD *e-document* sangat tinggi (0,132) — jauh melampaui ambang $\tau = 0,01$ — sehingga pelaporan drift bersih wajib dan mekanisme H_3 tidak dapat ditunjukkan pada dataset ini; nilai absolut lantai derau ini bergantung pada ukuran jendela pengukuran. Kedua, ketiga dataset kecil berduplikasi 68–99% dan seluruh konfigurasinya ditandai peringatan (0/27 OK), mengonfirmasi status mereka sebagai konteks pelengkap dan bukan tumpuan vonis H_1 .

Lampiran G Kutipan Kode Kunci

Lampiran ini menampilkan kutipan kode dari paket `codaeda` untuk tiga mekanisme inti *CoDA-EDA*. Kutipan disederhanakan agar terbaca (bagian aritmetika padat diringkas dengan tanda "...").

G.1 Fungsi Fitness Selaras-F1

Skor satu aturan berbentuk

$$\frac{TP^2}{TP + w_{fp} \cdot FP} - w_{wsc} \cdot WSC,$$

yang setara dengan $TP \times$ presisi sehingga menghargai cakupan dan presisi sekaligus (selaras F1); bentuk penalti linier ditolak karena optimumnya aturan bercakupan nol (modul `codaeda/problem.py`).

```
def fitness(self, x, uncovered):

    self.evals += 1

    # aturan tanpa aksi tidak pernah cocok
    if x[self.act_idx].sum() == 0:

        return -1e9

    m = self.matches(x)

    tp = int((m & self.label & uncovered).sum())

    if tp == 0:

        # aturan tanpa permit baru tidak berguna
        # (separate-and-conquer); tanpa
        # dominasi ini aturan KOSONG mengalahkan
        # aturan bermanfaat ber-FP kecil.

        return -1e9 - self.w_wsc * self.wsc(x)
```

```

fp = int((m & ~self.label).sum())

# Skor = TP x presisi = TP^2 / (TP + w_fp*FP):
# selaras F1 (cakupan DAN presisi).*

# Penalti linier (TP - w_fp*FP) keliru:
# optimumnya aturan super-presisi
# mencakup nol, sehingga pencarian lebih kuat
# justru MENURUNKAN F1 kebijakan.

return (tp * tp) / (tp + self.w_fp * fp)
- self.w_wsc * self.wsc(x)

```

G.2 Pohon Chow-Liu order-1 via Algoritma Prim (Memori $O(D)$)

Struktur dependensi disusun sebagai pohon rentang maksimum menggunakan algoritma Prim, yang hanya menyimpan larik sepanjang D (bukan matriks mutual information $D \times D$); mutual information dihitung baris-demi-baris atas representasi terkemas-bit (modul `codaeda/algos.py`).

```

def chow_liu_prim(A, rng, max_rows=1536, eps=1e-6):
    n, D = A.shape
    idx = rng.choice(n, max_rows, replace=False)
    if n > max_rows else np.arange(n)

    # Bit-packing: kolom biner dikemas 8 baris/byte;
    # ko-okurensi via AND bitwise + tabel popcount,
    # tanpa membentuk matriks MI  $D \times D$  maupun salinan
    # float data.

    # (disusun per blok baris)
    P = np.packbits(A[idx], axis=0)
    cnt1 = _POPCNT[P].sum(axis=0).astype(float);
    p1 = cnt1 / n

    # mutual information baris-ke-i terhadap
    # semua variabel
    def mi_row(i):

```

```

        j11 = _POPCNT[P & P[:, i:i+1]].sum(axis=0) / n
        # ... hitung j10, j01, j00
        # lalu MI = SUM p*log2(p / (p_i * p_j)) ...
        return mi

# Algoritma Prim: HANYA larik sepanjang D
# (bukan matriks D x D) -> memori O(D)
parent = np.full(D, -1); best = np.full(D, -np.inf);
intree = np.zeros(D, bool)
cur = 0; intree[cur] = True

for _ in range(D - 1):
    # satu baris MI dihitung saat perlu
    m = mi_row(cur)
    upd = (~intree) & (m > best);
    best[upd] = m[upd]; bestfrom[upd] = cur
    nxt = int(np.argmax(np.where(intree,
    -np.inf, best)))
    parent[nxt] = bestfrom[nxt]; intree[nxt] = True;
    cur = nxt

return parent, cp0, cp1, wgt
# induk + tabel probabilitas bersyarat per variabel

```

G.3 Mekanisme Deteksi Drift

Pergeseran distribusi antar-jendela diukur sebagai Jensen-Shannon divergence atas marginal pasangan atribut-nilai; struktur dependensi hanya dipelajari ulang ketika pergeseran melampaui ambang τ , sedangkan parameter (*Probability Vector*) tetap diperbarui tiap siklus.

```

# distribusi frekuensi pasangan atribut-nilai
# pada satu jendela
def av_marginal(A):
    m = A.mean(axis=0).astype(np.float64); s = m.sum()
    return m / s if s > 0 else np.full(len(m),
    1.0 / max(1, len(m)))

```

```

# Jensen-Shannon divergence
def jsd(p, q, eps=1e-12):
    m = 0.5 * (p + q)
    kl = lambda a: float(np.sum(np.where(a > 0,
    a * np.log2((a+eps)/(m+eps)), 0.0)))
    return max(0.0, min(1.0,
    0.5 * kl(p) + 0.5 * kl(q)))

# Pemicu (di run_sequential): struktur dipelajari
# ulang HANYA bila drift > tau

# distribusi jendela berjalan
cur      = av_marginal(prob.A)

# pergeseran thd jendela referensi
delta    = 0.0 if ref is None else jsd(ref, cur)

# tau = 0,01 (baku)
relearn  = (mode == "always") or (delta > tau)

if relearn:
    # perbarui struktur; jika tidak: PV saja
    T = chow_liu_prim(prob.A, rng)

```